

Open-FDD Documentation

Generated by scripts/build_docs_pdf.py

June 9, 2026

- 1 Home
- 2 Open-FDD
 - 2.1 Who it is for
 - 2.2 Start here
 - 2.3 v3 edge stack
 - 2.4 Distribution
 - 2.5 License
- 3 Run with Docker images
- 4 Run with Docker images
 - 4.1 Images
 - 4.2 1. Install and validate Docker
 - 4.3 2. Bootstrap the site (one script)
 - 4.3.1 Manual start (optional)
 - 4.4 Long-term operation
 - 4.4.1 Survive power cycles
 - 4.4.2 Start / stop / restart
 - 4.4.3 LAN access
 - 4.5 Deployment choices
 - 4.6 Next steps
- 5 First login and health check
- 6 First login and health check
 - 6.1 Public health
 - 6.2 Login
 - 6.3 Smoke checklist
 - 6.4 Stack detail (authenticated)
 - 6.5 Troubleshooting
- 7 Quick Start
- 8 Quick Start
- 9 Updating the stack
- 10 Updating the stack
 - 10.1 Before you start
 - 10.2 Fetch helper scripts (no git clone)
 - 10.3 One-command upgrade (recommended)
 - 10.4 Step by step
 - 10.4.1 1. Backup site state
 - 10.4.2 2. Verify images (optional)
 - 10.4.3 3. Pull and recreate

- 10.4.4 4. Verify
- 10.5 Dashboard UI updates
- 10.6 Rollback
- 10.7 Safe cleanup
- 10.8 Next steps
- 11 Local development
- 12 Local development
 - 12.1 Clone and auth
 - 12.2 Start stack
 - 12.3 BACnet bench overlay (optional)
 - 12.4 Production-like local stack (Caddy, LAN scans)
 - 12.5 Docs preview
- 13 Build Docker images
- 14 Build Docker images
 - 14.1 Local build
 - 14.2 Publish to GHCR (maintainers)
- 15 Developer Guide
- 16 Developer Guide
- 17 Tests and CI
- 18 Tests and CI
 - 18.1 Local tests
 - 18.2 CI (GitHub Actions)
 - 18.3 Docs build (same as CI)
 - 18.4 Security-related CI
- 19 Contributing
- 20 Contributing
 - 20.1 Expectations
 - 20.2 Pull request checklist
 - 20.3 Repository layout (short)
 - 20.4 Security issues
 - 20.5 Agent-assisted development
- 21 Health checks (developer)
- 22 Health checks (developer)
 - 22.1 Clone and local stack
 - 22.2 Stack health script
 - 22.3 Compose overlays
 - 22.4 BACnet poll pipeline
 - 22.5 pytest (bridge security / BACnet)
 - 22.6 Pentest production stack (LAN ZAP)
 - 22.7 Post-deploy insurance (inventory hosts)
- 23 Security testing cycle
- 24 Security testing cycle
 - 24.1 Who owns what
 - 24.2 Per-revision workflow
 - 24.2.1 1. Local functional tests
 - 24.2.2 2. Pentest production stack (on Open-FDD host)
 - 24.2.3 3. Host + exposure checks (on edge VM)
 - 24.2.4 4. LAN security scan (from laptop)
 - 24.2.5 5. Pull request + automated review
 - 24.2.6 6. Release
 - 24.2.7 7. Lab edge validation
 - 24.2.8 8. Next cycle

- 24.3 What each scan layer covers
- 24.4 Open-source contributors
- 25 Python recipes (full Arrow library)
- 26 Python recipes — full Arrow library
 - 26.1 Shared helpers (paste once per rule file)
 - 26.2 GL36 AHU Rules A-M
 - 26.2.1 Rule A — duct static low at full fan speed (AHU-A)
 - 26.2.2 Rules B & C — blended air outside OAT/RAT band (AHU-D)
 - 26.2.3 Rule D — discharge cold when heating commanded (AHU-B)
 - 26.2.4 Rule E — SAT too low with full heating (AHU-C)
 - 26.2.5 Rule F — SAT/MAT mismatch in economizer mode (AHU-E)
 - 26.2.6 Rule G — ambient too warm for free cooling (AHU-E)
 - 26.2.7 Rule H — OAT/MAT mismatch (econ + mech cooling) (AHU-E)
 - 26.2.8 Rule I — OAT/MAT mismatch (economizer only) (AHU-E)
 - 26.2.9 Rule J — discharge above blended in cooling (AHU-B)
 - 26.2.10 Rule K — discharge above setpoint in full cooling (AHU-C)
 - 26.2.11 Rule L — cooling coil ΔT when inactive (CH-C)
 - 26.2.12 Rule M — heating coil ΔT when inactive (AHU-B)
 - 26.3 Starter pack (VAV / AHU baseline)
 - 26.3.1 01_vav_zone_temp_bounds_occupied (VAV-C)
 - 26.3.2 02_vav_zone_temp_flatline_occupied (VAV-C)
 - 26.3.3 03_vav_damper_command_extreme_flatline (VAV-D)
 - 26.3.4 04_ahu_runtime_outside_schedule (BLD-C)
 - 26.3.5 05 / 06 / 07 — see Rules A, sensor_bounds_mask on sa-t / sensor_flatline_mask on sa-t
 - 26.4 Economizer starters
 - 26.4.1 ahu_econ_100oa_temp_tracking_fault (AHU-E)
 - 26.4.2 ahu_mech_cooling_when_free_cooling_available (AHU-E)
 - 26.4.3 ahu_oa_damper_excess_open_extreme_ambient (AHU-E)
 - 26.5 VAV zones
 - 26.5.1 zone_reheat_warm_ambient (VAV-A)
 - 26.5.2 zone_damper_valve_full_open (VAV-D)
 - 26.5.3 Zone / IAQ bounds
 - 26.6 Central plant
 - 26.6.1 dp_below_sp_pump_max (CH-E)
 - 26.6.2 flow_high_pump_max (CH-B)
 - 26.6.3 plant_supply_temp_deadband (CH-D)
 - 26.6.4 chiller_excessive_runtime (CH-F)
 - 26.7 Heat pumps
 - 26.7.1 hp_discharge_cold_when_heating (HP-D)
 - 26.8 Opportunistic / ventilation
 - 26.8.1 econ_active_warm_ambient (AHU-E)
 - 26.8.2 mech_cool_when_econ_available (AHU-E)
 - 26.8.3 low_oa_fraction_estimated (VAV-B)
 - 26.8.4 preheat_excess_temp (AHU-B)
 - 26.9 Weather station
 - 26.9.1 weather_temp_spike (BLD-B)

- 26.9.2 weather_gust_lt_wind (BLD-B)
 - 26.9.3 RH bounds
- 26.10 Schedule + weather gating (occupied hours)
- 26.11 Next steps
- 27 System overview
- 28 System overview
 - 28.1 Docker edge stack (production)
 - 28.2 Components
 - 28.3 Deployment
 - 28.4 Optional PyPI-only path
- 29 Expression cookbook (Arrow-native)
- 30 Expression cookbook (Arrow-native)
 - 30.1 Legacy → modern translation
 - 30.2 How Arrow rules are structured
 - 30.2.1 Command scaling (0-1 vs 0-100 %)
 - 30.2.2 Occupied hours (schedule gating)
 - 30.3 Sensor validation (bounds, flatline, rate of change)
 - 30.3.1 Bounds (out of range) — oob_rolling
 - 30.3.2 Flatline (stuck sensor) — flatline_1h
 - 30.3.3 Rate of change (spike) — rate_of_change
 - 30.3.4 Mixing envelope (MAT vs OAT/RAT) — mixing_envelope
 - 30.4 GL36-inspired AHU rules (legacy expression → fault code)
 - 30.5 Starter pack (VAV / AHU baseline)
 - 30.5.1 Economizer starters
 - 30.6 VAV, central plant, heat pump
 - 30.7 Opportunistic / ventilation
 - 30.8 Data quality vs equipment faults
 - 30.9 Metric (°C) equivalents
 - 30.10 Binding rules to the fault catalog
 - 30.11 Pandas YAML → Arrow checklist (commissioning)
- 31 Deployment modes
- 32 Deployment modes
 - 32.1 Mode summary
 - 32.2 v3 container stack (all edge modes)
 - 32.3 Caddy HTTP (typical edge)
 - 32.4 Caddy TLS (OT LAN)
 - 32.5 BACnet data path
 - 32.6 What not to do on production edges
 - 32.7 Related
- 33 Arrow recipes
- 34 Arrow recipes (default)
 - 34.1 Simple threshold
 - 34.2 Supply air temp high
 - 34.3 Fan commanded but no airflow
 - 34.4 Flatline (1 h rolling window)
 - 34.5 Sensor out of range (catalog defaults)
 - 34.6 Sensor flatline + spike (catalog)
- 35 Python recipes (Arrow)
- 36 Python recipes (Arrow)
- 37 Containers
- 38 Containers
 - 38.1 GHCR images

- 38.2 Stack diagram
- 38.3 Responsibilities
- 38.4 Network and ports (typical edge)
- 38.5 Long-lived deployment
- 38.6 Persistent data
- 38.7 Optional host services
- 39 Lookback window helper
- 40 Lookback window helper
 - 40.1 Recommended pattern (copy into `rule.py`)
 - 40.2 Why not hide this behind a library call yet?
 - 40.3 Lookback vs rolling samples
 - 40.4 Batch intervals
- 41 Data flow
- 42 Data flow
 - 42.1 Telemetry ingest
 - 42.1.1 BACnet
 - 42.1.2 Modbus TCP
 - 42.1.3 JSON API (HTTP/HTTPS)
 - 42.2 Rule evaluation
 - 42.3 Model sync
 - 42.4 Retention
- 43 Architecture
- 44 Architecture
- 45 Rule Cookbook
- 46 Rule Cookbook
- 47 Windowing & debugging
- 48 Windowing & debugging
 - 48.1 Lookback vs rolling window
 - 48.2 Rolling windows (Arrow)
 - 48.3 Rule Lab console
- 49 GL36 algorithm stubs
- 50 GL36 algorithm stubs (doc-only)
 - 50.1 Shared lookback helper
 - 50.2 VAV zone cooling requests (0-3)
 - 50.3 AHU duct static trim & respond (advisory fault)
 - 50.4 Chiller plant enable (valve request counting)
 - 50.5 Hot water plant HWST trim & respond (advisory)
 - 50.6 Chilled water plant (DP + CHWST reset advisory)
 - 50.7 Testing later
- 51 Central plants (CHW / CTW / BLR)
- 52 Central plant Arrow recipes
 - 52.1 Starter fault codes
 - 52.2 Required point roles (CHW)
 - 52.3 Synthetic test pattern
- 53 Dashboard overview
- 54 Dashboard overview
 - 54.1 Primary areas
 - 54.2 Live updates
 - 54.3 Roles
- 55 Auth and roles
- 56 Auth and roles
 - 56.1 Production (LAN / edge)

- 56.2 Localhost dev
- 57 Rule Lab
- 58 Rule Lab
 - 58.1 Workflow
 - 58.2 Dev kit zip (download)
 - 58.3 Lookback windows
 - 58.4 Algorithms (supervisory — coming soon)
 - 58.5 Related
- 59 JSON API driver
- 60 JSON API driver
 - 60.1 Supported features
 - 60.2 Secrets: `json_api.env.local`
 - 60.3 OpenWeatherMap showcase (recommended demo)
 - 60.3.1 1. Configure env
 - 60.3.2 2. Register from the UI
 - 60.3.3 3. Headless / script
 - 60.3.4 4. FDD: local OA-T vs web weather
 - 60.4 Commissioning (UI)
 - 60.4.1 API for agents
 - 60.4.2 Bench preset
 - 60.5 Authentication examples
 - 60.5.1 Query-string API key via env (OpenWeather)
 - 60.5.2 Bearer token (API key)
 - 60.5.3 HTTP Basic
 - 60.5.4 POST with JSON body
 - 60.6 REST API
 - 60.7 Typical OT URLs
 - 60.8 Limitations
 - 60.9 Disable polling (save API quota)
 - 60.10 Smoke test
- 61 Algorithms
- 62 Algorithms (coming soon)
 - 62.1 GL36 Trim & Respond reference
 - 62.2 Planned workflow (same kit shape as Rule Lab)
 - 62.3 Lookback windows
 - 62.4 AHU supervisory checks (doc-only stubs)
 - 62.4.1 Duct static pressure too high
 - 62.4.2 Supply air temperature too cold (cooling coil over-delivering)
 - 62.5 Plant supervisory checks (doc-only)
 - 62.6 Related
- 63 Model workflow
- 64 Model workflow
 - 64.1 Brick model
 - 64.2 Typical integrator flow
- 65 Driver framework
- 66 Driver framework
 - 66.1 Drivers
 - 66.2 Data layout
 - 66.3 Historian and FDD
 - 66.4 Rule Lab (Arrow upload/download)
 - 66.5 Smoke validation

- 66.6 Related docs
- 67 Operator Bridge
- 68 Operator Bridge
- 69 Ollama and analytics
- 70 Ollama and analytics
 - 70.1 What runs without login
 - 70.2 Ollama setup
 - 70.3 Automatic analytics (deterministic, always on)
 - 70.4 When Ollama is used
 - 70.5 AI Agent (authenticated)
 - 70.6 Code map
- 71 FDD and assignments
- 72 FDD and assignments
 - 72.1 What the product actually does
 - 72.1.1 BRICK modeling (integrator + AI-assisted)
 - 72.1.2 Rule assignment (three binding kinds)
 - 72.1.3 CLASS vs device
 - 72.2 Commissioning JSON shape
 - 72.2.1 Rule names vs ids (Rule Lab alignment)
 - 72.3 Related
- 73 Network setup
- 74 Network setup
 - 74.1 Bind address
 - 74.2 Discovery range
 - 74.3 Verify
- 75 Discover and read
- 76 Discover and read
 - 76.1 Operator workflow
- 77 Polling
- 78 Polling
 - 78.1 Commission poll loop (default)
 - 78.2 Bridge ingest worker
 - 78.3 Health
- 79 Write safety
- 80 Write safety
 - 80.1 Defaults
 - 80.2 Enable writes (commission role)
 - 80.3 Operator rules
- 81 BACnet
- 82 BACnet
- 83 Convention
- 84 Fault code convention
 - 84.1 Format (Grade-A, 3.0.18+)
 - 84.2 Categories (Grade-A)
 - 84.3 Severity
 - 84.4 Linking rules
 - 84.5 Cookbook mapping
- 85 Live site update
- 86 Live site update
 - 86.1 Layout on the edge host
 - 86.2 Concepts
 - 86.3 1. Verify new images on GHCR

- 86.4 2. Update dashboard UI (when release changed React)
- 86.5 3. Pull and recreate (edge host)
- 86.6 4. Ansible image-only (control machine)
- 86.7 5. Validate
- 86.8 Rollback
- 86.9 Related
- 87 AHU faults
- 88 AHU faults
- 89 Standards crosswalk
- 90 Standards crosswalk
 - 90.1 Example
- 91 Backup and restore
- 92 Backup and restore
 - 92.1 Quick backup (edge host)
 - 92.2 Manual backup (SSH)
 - 92.3 What to protect
 - 92.4 Restore after a bad upgrade
 - 92.5 Safe cleanup
 - 92.6 Related
- 93 VAV faults
- 94 VAV faults
- 95 RTU faults
- 96 RTU faults
- 97 Deployment validation
- 98 Deployment validation
 - 98.1 Edge host smoke (5 minutes)
 - 98.2 Browser checklist
 - 98.3 Control-machine insurance check
 - 98.4 LAN security smoke (optional)
 - 98.5 Arrow / FDD rules (3.0+)
 - 98.6 Related
- 99 Sensor and data quality
- 100 Sensor and data quality
 - 100.1 Catalog codes
 - 100.2 Default bounds (imperial, occupied)
 - 100.3 Communication quality
 - 100.4 Operator workflow
- 101 Logging and audit
- 102 Logging and audit
 - 102.1 Log types
 - 102.2 Audit record schema
 - 102.2.1 Auth events (industry-standard login trail)
 - 102.2.2 OT / commissioning events
 - 102.2.3 Secret redaction
 - 102.3 Integrator API (read logs in the UI stack)
 - 102.4 Retention and rotation (defaults)
 - 102.5 Docker edge: json-file caps (not journald for app audit)
 - 102.6 Local dev (run_local.sh)
 - 102.7 Quick checks
 - 102.8 Comparison to typical web apps
 - 102.9 Related
- 103 Fault Codes

- 104 Fault Codes
- 105 Operations
- 106 Operations
- 107 LAN hardening
- 108 LAN hardening
 - 108.1 Auth and bind
 - 108.2 Network exposure
 - 108.3 Rule Lab
 - 108.4 TLS and LAN security scans
 - 108.5 Ollama on the edge
- 109 ZAP baseline (local bench)
- 110 ZAP baseline (local bench)
 - 110.1 Run
 - 110.2 Expected results (HTTP bench mode)
 - 110.3 Header ownership
 - 110.4 Authenticated scan (later)
 - 110.5 After fixes
- 111 TLS and certificates
- 112 TLS and certificates
 - 112.1 Ownership
 - 112.2 Generate bench certs (local)
 - 112.3 ZAP / Nmap with TLS bench
 - 112.4 Renewal
 - 112.5 Enterprise CA on Caddy (edge)
 - 112.6 Caddy internal CA mode (lab)
 - 112.7 Secrets
- 113 Authenticated scanning (roadmap)
- 114 Authenticated scanning (roadmap)
 - 114.1 Current state (3.0.x)
 - 114.2 Why baseline is not enough
 - 114.3 Planned approach (contributors welcome)
 - 114.4 Manual authenticated ZAP today
 - 114.5 Related
- 115 Secrets and auth
- 116 Secrets and auth
 - 116.1 Files (gitignored)
 - 116.2 Required variables (production)
 - 116.3 GHCR pulls
 - 116.4 Backup
- 117 BACnet write allowlists
- 118 BACnet write allowlists
- 119 Linux host hardening
- 120 Linux host hardening
 - 120.1 Scope
 - 120.2 Quick operator workflow
 - 120.3 Ubuntu version and EOL
 - 120.4 Kernel and package updates
 - 120.5 Host Python (pip, wheel, pyOpenSSL)
 - 120.6 Ollama — no LAN exposure
 - 120.7 OpenSSH Terrapin (CVE-2023-48795)
 - 120.8 ICMP timestamp (Low)
 - 120.9 TLS and certificate scans

120.10	Maintainer security audits (CI parity)
121	Tenable / Nessus remediation
122	Tenable / Nessus remediation
122.1	Prioritized action plan
122.2	Finding classification table
122.3	Operator commands (edge VM)
122.3.1	Validate (read-only)
122.3.2	Remediate host (maintenance window)
122.3.3	Remediate Open-FDD containers
122.3.4	Rescan
122.4	Ollama unauthenticated access (Critical)
122.5	pyOpenSSL and pip in containers
122.6	TLS — expected vs production
122.7	Residual findings after remediation
122.8	Notes for IT / security team
122.9	Repo-owned fixes (this document's release)
123	Security
124	Security
125	API routes
126	API routes
126.1	Health & audit
126.2	Auth
126.3	Rules & FDD
126.4	Rule Lab playground
126.5	BRICK / data model
126.6	BACnet
126.7	Faults (check-engine)
126.8	Timeseries & sites
126.9	Building & host
126.10	Local agent (Ollama)
126.11	Modbus
126.12	JSON API (HTTP/HTTPS)
126.13	PyPI package (separate)
126.14	Errors
127	Python package
128	Python package (open-fdd)
128.1	Modules
128.2	When to use package-only
128.3	Versioning
128.4	Offline Arrow example
129	Configuration reference
130	Configuration reference
130.1	Bridge / auth
130.2	BACnet
130.3	Docker edge
131	Glossary
132	Glossary
133	Workspace env files
134	Workspace env files
134.1	Ansible / edge extras
134.2	Related docs
135	Appendix

- 136 Appendix
- 137 GL36 lab site (Acme)
- 138 GL36 lab site (Acme)
 - 138.1 BACnet poll scope
 - 138.2 Example FDD rules (Arrow)
 - 138.2.1 Duct static pressure high (AHU)
 - 138.2.2 SAT too cold vs setpoint
 - 138.2.3 Site rule stubs in the repo
 - 138.2.4 Local OA-T vs web weather (JSON API)
 - 138.3 Deploy / upgrade (Ansible, this lab only)
 - 138.4 AI-assisted data modeling (lab workflow)
- 139 Examples & lab notes
- 140 Examples & lab notes
- 141 Central collection
- 142 Portfolio central collection
 - 142.1 Interval summary schema
 - 142.2 Edge export formats
 - 142.3 Collect today (rollup API)
 - 142.4 Tuning proposals
- 143 Arrow-native runtime
- 144 Arrow-native runtime
 - 144.1 Authoring contract
 - 144.2 Package layout

Generated June 9, 2026

1 Home

2 Open-FDD

Open-FDD is an open-source **building edge platform** for BACnet integration, feather historian storage, Arrow-native fault detection, and operator dashboards — designed for **trusted LAN / OT edge** deployment behind Caddy, not direct public internet exposure.

2.1 Who it is for

Role	What you get
IT / controls engineer	Pull three GHCR images, log in, poll BACnet, view faults
Commissioning integrator	Discover points, bind BRICK model, pin FDD rules in Rule Lab

Role	What you get
Developer / maintainer	Extend the Operator Bridge, publish images, run CI and LAN security scans

2.2 Start here

Path	Go to
Try it	Quick Start — Run with Docker
Operate it	Updating the stack · Operations
Develop it	Developer Guide · Architecture

2.3 v3 edge stack

Three primary containers on a Linux edge host:

Container	Role
openfdd-bridge	Operator Bridge — API, dashboard, feather ingest
openfdd-commission	BACnet discover/read/write + poll loop
openfdd-mcp-rag	Doc-search sidecar for agent tools

Host **Caddy** on :80 or :443 is the normal LAN front door. BACnet UDP :47808 is bound by **commission** only — the bridge ingests samples.csv into the feather historian.

Details: [Containers](#) · [Deployment modes](#)

LAN / OT edge only. Do not expose the Operator Bridge to the public internet without TLS, strong auth, and site review. BACnet writes are disabled by default — see [BACnet write guard](#).

2.4 Distribution

Channel	Use when
GHCR ghcr.io/bbartling/openfdd-*	Production or trial edge deploy
This repository	Custom builds, commissioning, Rule Lab development

Channel	Use when
PyPI open-fdd	Arrow runtime lint/test without full UI

2.5 License

MIT — see repository LICENSE.

3 Run with Docker images

4 Run with Docker images

Deploy Open-FDD on a Linux edge host using **three published GHCR images**. No git clone required.

4.1 Images

GHCR image	Service	Role
ghcr.io/bbartling/openfdd-bridge	bridge	Operator API, dashboard, historian ingest
ghcr.io/bbartling/openfdd-commission	commission	BACnet discover/ read/write and poll loop
ghcr.io/bbartling/openfdd-mcp-rag	mcp-rag	Doc-search sidecar

Edge hosts pull **latest** from GHCR (three images: bridge, commission, mcp-rag). The retired **openfdd-bacnet-poll** image is no longer published.

4.2 1. Install and validate Docker

```
sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"
newgrp docker    # or log out/in
```

```
docker --version
docker compose version
docker ps
docker run --rm hello-world
```

Expect: active / enabled for docker, docker in groups, hello-world succeeds.

4.3 2. Bootstrap the site (one script)

Downloads docker-compose.yml, creates workspace/, generates auth.env.local, and sets BACNET_BIND from the host LAN NIC (e.g. ens192 → 10.200.200.185/24:47808):

```
curl -fsSL -o /tmp/openfdd_edge_bootstrap.sh \
https://github.com/bbartling/open-fdd/raw/refs/heads/master/
scripts/openfdd_edge_bootstrap.sh
bash /tmp/openfdd_edge_bootstrap.sh
```

Bootstrap + pull + start in one go:

```
bash /tmp/openfdd_edge_bootstrap.sh --start
```

Auth — bootstrap writes ~/open-fdd/workspace/auth.env.local (gitignored). This file holds the API signing secret and one username/password per role. Use it for your first dashboard login:

```
cat ~/open-fdd/workspace/auth.env.local
```

Variable	Role
OFDD_AUTH_SECRET	Signs session tokens (do not share)
OFDD_OPERATOR_*	Read-only operator
OFDD_INTEGRATOR_*	Default dashboard login — commissioning + BACnet
OFDD_AGENT_*	Agent / automation

Example shape (passwords are **random on first bootstrap only** — existing file is kept on re-runs):

```
OFDD_AUTH_SECRET=<random>
OFDD_OPERATOR_USER=operator
OFDD_OPERATOR_PASSWORD=<random>
OFDD_INTEGRATOR_USER=integrator
OFDD_INTEGRATOR_PASSWORD=<random>
```

```
OFDD_AGENT_USER=agent
OFDD_AGENT_PASSWORD=<random>
```

Sign in as **integrator** with the password from that file → First login and health check.

Action	Touches auth.env.local?
First bash ... bootstrap.sh --start	Creates file with random passwords (if missing)
bash ... bootstrap.sh --start again	Keeps your file — does not overwrite
bash ... bootstrap.sh --restart	Keeps your file — only restarts bridge to reload it
nano auth.env.local + --restart	Uses your edited passwords
--force-auth	Regenerates random passwords (opt-in only)

The **bridge** container reads auth.env.local at startup only. After you change passwords (manual edit or --force-auth), recreate bridge (env files are not reloaded by restart alone):

```
cd ~/open-fdd && docker compose up -d --force-recreate bridge
```

Regenerate passwords and reload (stack already running):

```
bash /tmp/openfdd_edge_bootstrap.sh --force-auth --restart --show-secrets
```

Manual edit then reload:

```
nano ~/open-fdd/workspace/auth.env.local
bash /tmp/openfdd_edge_bootstrap.sh --restart
```

Options:

Flag	Purpose
--start	docker compose pull && up -d after layout; curls http://127.0.0.1:8765/health
--image-tag TAG	default latest
--repo-ref BRANCH	branch for compose.edge.yml if not on master yet
--force-auth	regenerate auth.env.local (restarts bridge if stack is up)
--restart	

Flag	Purpose
	recreate bridge to reload auth.env.local; curls /health after
--show-secrets	print passwords at end (lab only)

From a repo checkout: `./scripts/openfdd_edge_bootstrap.sh --start`

The script prints **BACnet NIC**, **BACNET_BIND**, and file paths — **validate** `commission.env` before polling OT devices:

```
nano ~/open-fdd/workspace/bacnet/commissioning/commission.env
```

If you used `--start`, the stack is already up. Next step → [First login and health check](#).

4.3.1 Manual start (optional)

Use this only if you ran bootstrap **without** `--start` and want to bring the stack up yourself:

```
cd ~/open-fdd
docker compose pull
docker compose up -d
docker compose ps
curl -sf http://127.0.0.1:8765/health && echo
```

Expected: **bridge**, **commission**, **mcp-rag** — all Up. Then → [First login and health check](#).

4.4 Long-term operation

4.4.1 Survive power cycles

All services use restart: `unless-stopped`. After reboot:

```
cd ~/open-fdd && docker compose ps
curl -sf http://127.0.0.1:8765/health
```

4.4.2 Start / stop / restart

```
cd ~/open-fdd
docker compose up -d           # idempotent
docker compose stop
docker compose restart commission
docker compose logs -f --tail 100 commission
```

Never run `docker compose down -v` or delete workspace/ on a live site.

4.4.3 LAN access

Put **Caddy** (or nginx) on port 80 → 127.0.0.1:8765, or expose 8765 through the firewall.

4.5 Deployment choices

Mode	Compose services	When to use
Standard OT / BACnet	bridge + commission + mcp-rag	Real buildings — commission polls BACnet into samples.csv; bridge ingests to feather
Non-BACnet / demo	bridge + mcp-rag (omit commission)	JSON API, Modbus-only, or lab without field BACnet — see Driver framework
TLS on OT LAN	Same stack + host Caddy OFDD_CADDY_MODE=tls	Encrypt browser↔edge; trust self-signed CA on operator PCs — not for public internet

Do **not** run the retired openfdd-bacnet-poll image or legacy openfdd-bacnet-poll systemd unit alongside Docker **commission** — that double-polls BACnet.

4.6 Next steps

→ [First login and health check](#)

→ [Updating the stack](#) — ./scripts/openfdd_site_update.sh

5 First login and health check

6 First login and health check

Quick checks after docker compose up -d on the edge host. Services use restart: unless-stopped — after a reboot, run docker compose ps once Docker is up (see [Run with Docker](#)).

6.1 Public health

```
curl -s http://127.0.0.1:8765/health
```

Through Caddy on port 80:

```
curl -s http://127.0.0.1/health
```

Expect "ok": true and "auth_required": true when auth is configured.

6.2 Login

1. Open the dashboard ([http://<host-lan-ip>/ or :8765](http://<host-lan-ip>/or :8765)).
2. Sign in with credentials from `workspace/auth.env.local`.

```
curl -s -X POST http://127.0.0.1:8765/api/auth/login \
-H 'Content-Type: application/json' \
-d '{"username":"integrator","password":"<from-auth-env>"}'
```

6.3 Smoke checklist

Check	Pass
docker compose ps	bridge, commission, mcp-rag Up
GET /health	200, ok: true
Login	Token returned
BACnet poll	samples.csv growing (tail -1 workspace/bacnet/ polls/samples.csv)
Historian	Files under workspace/data/ feather_store/

6.4 Stack detail (authenticated)

Integrator token required:

```
TOKEN=$(curl -sf -X POST http://127.0.0.1:8765/api/auth/login \
-H 'Content-Type: application/json' \
-d '{"username":"integrator","password":"..."}' | jq -r .token)
curl -sf http://127.0.0.1:8765/health/stack -H "Authorization:
Bearer $TOKEN" | jq .
```

Look for `bacnet_poll` green/yellow in the services list.

6.5 Troubleshooting

Symptom	Fix
401 everywhere	Configure workspace/ auth.env.local
Empty BACnet discover	Fix BACNET_BIND in commission.env — network setup
No poll rows	Commission container running? points.csv has enabled rows?
LAN browser timeout	Open firewall port 80 or 8765

Advanced probes (model graph, Rule Lab, compose profiles):
[Developer health check](#).

7 Quick Start

8 Quick Start

Run Open-FDD on a Linux edge host using **three GHCR Docker images**. No git clone on the host.

Step	Page
1	Run with Docker images — bootstrap script + docker compose up
2	First login and health check
3	Updating the stack — backup + pull

One-liner bootstrap (edge host):

```
curl -fsSL -o /tmp/openfdd_edge_bootstrap.sh \  
https://github.com/bbartling/open-fdd/raw/refs/heads/master/  
scripts/openfdd_edge_bootstrap.sh  
bash /tmp/openfdd_edge_bootstrap.sh --start
```

Prerequisites: Linux, Docker enabled on boot, BACnet LAN if
polling OT equipment.

| **BACnet writes are disabled by default.** See [Write safety](#).

Stack: bridge + commission (BACnet + poll) + mcp-rag. Details: [Containers](#).

9 Updating the stack

10 Updating the stack

Upgrade GHCR images on the edge host: **backup workspace/, pull new tags, recreate containers**. No git pull on the host.

10.1 Before you start

1. SSH to the edge host (cd ~/open-fdd).
2. Pick a tag from [GitHub Packages](#) — default is **latest**.
3. Short maintenance window (containers restart briefly;
restart: unless-stopped brings them back after reboot).

10.2 Fetch helper scripts (no git clone)

If you bootstrapped with openfdd_edge_bootstrap.sh, scripts are already under ~/open-fdd/scripts/. Otherwise:

```
mkdir -p ~/open-fdd/scripts
curl -fsSL -o ~/open-fdd/scripts/openfdd_site_backup.sh \
    https://github.com/bbartling/open-fdd/raw/refs/heads/master/
scripts/openfdd_site_backup.sh
curl -fsSL -o ~/open-fdd/scripts/openfdd_site_update.sh \
    https://github.com/bbartling/open-fdd/raw/refs/heads/master/
scripts/openfdd_site_update.sh
chmod +x ~/open-fdd/scripts/openfdd_site_*.sh
```

10.3 One-command upgrade (recommended)

```
cd ~/open-fdd
./scripts/openfdd_site_backup.sh
./scripts/openfdd_site_update.sh
```

Pulls **:latest** by default, verifies all three images exist on GHCR, recreates containers, and hits /health.

10.4 Step by step

10.4.1 1. Backup site state

```
cd ~/open-fdd
./scripts/openfdd_site_backup.sh
```

Archives workspace/ (feather, BACnet CSVs, model, auth) under ~/openfdd-backups/<timestamp>/.

Custom backup location:

```
BACKUP_ROOT=~/openfdd-backups/manual ./scripts/
openfdd_site_backup.sh
```

10.4.2 2. Verify images (optional)

```
export NEW_TAG="${NEW_TAG:-latest}"

docker manifest inspect ghcr.io/bbartling/openfdd-bridge:$
{NEW_TAG} >/dev/null &&
docker manifest inspect ghcr.io/bbartling/openfdd-commission:$
{NEW_TAG} >/dev/null &&
docker manifest inspect ghcr.io/bbartling/openfdd-mcp-rag:$
{NEW_TAG} >/dev/null &&
echo "All images exist for ${NEW_TAG}"
```

10.4.3 3. Pull and recreate

```
cd ~/open-fdd
./scripts/openfdd_site_update.sh
```

Manual equivalent (same as the script):

```
cd ~/open-fdd
docker compose pull
docker compose up -d --force-recreate
docker compose ps
curl -sf http://127.0.0.1:8765/health && echo
```

10.4.4 4. Verify

```
docker compose ps
curl -sf http://127.0.0.1:8765/health | jq .
tail -1 workspace/bacnet/polls/samples.csv
ls -lah workspace/data/feather_store/
```

→ Health check

10.5 Dashboard UI updates

If the release changed the React UI, copy a new workspace/api/static/app/ build onto the host before or after the image pull. Image pull alone may not change the browser bundle hash.

10.6 Rollback

Restore docker-compose.yml from backup and pull again, or re-run bootstrap from a known-good master commit. GHCR keeps only the **three newest** versions per image; early sites should rely on **latest** plus workspace/ backups.

workspace/ is untouched by image-only upgrades.

10.7 Safe cleanup

```
docker image prune -f
```

Never run docker compose down -v, docker volume prune, or delete workspace/ on a live site.

10.8 Next steps

- [Run with Docker images](#) — bootstrap a new host
- [Health check](#)

11 Local development

12 Local development

12.1 Clone and auth

```
git clone https://github.com/bbartling/open-fdd.git && cd open-fdd
cp workspace/auth.env.example workspace/auth.env.local
# Edit integrator/operator passwords and OFDD_AUTH_SECRET (32+
chars)
```

12.2 Start stack

```
./scripts/build_and_test.sh           # dashboard build + pytest
./scripts/docker_build.sh
./scripts/openfdd_stack.sh up
./scripts/stack_health_check.sh
```

URL	Service
http://127.0.0.1:8765	Bridge (direct)
http://127.0.0.1:8765/docs	OpenAPI

12.3 BACnet bench overlay (optional)

```
docker compose -f docker/compose.dev.yml -f docker/
compose.bench.yml up -d
# commission.env – set BACNET_BIND to your OT interface
```

12.4 Production-like local stack (Caddy, LAN scans)

Same shape as an Acme edge deploy: **prod React UI, Caddy on :80**, bridge on loopback **8765**, auth enabled. Use for OWASP ZAP or nmap from another machine on the LAN.

```
./scripts/pentest_production_stack.sh start    # build prod UI +
Caddy + auth.pentest.local
./scripts/pentest_production_stack.sh status   # bridge,
commission, caddy, mcp-rag
./scripts/pentest_production_stack.sh verify   # LAN IP, /health,
ZAP target URL
./scripts/pentest_production_stack.sh stop     # when done
```

Credentials: workspace/auth.pentest.local (generated on first start). ZAP target is usually http://<lan-ip>/ (benserver example: http://192.168.204.18/). If LAN clients cannot reach :80, run `sudo ./scripts/open_lan_port.sh 80`.

See also [Health checks \(developer\)](#) for probe details and [ZAP baseline expectations](#) for accepted Medium/Low findings.

12.5 Docs preview

```
cd docs && bundle install && bundle exec jekyll serve
# http://127.0.0.1:4000/open-fdd/
```

13 Build Docker images

14 Build Docker images

14.1 Local build

```
./scripts/docker_build.sh
# Optional tar for air-gap:
./scripts/docker_build.sh --save
```

Images: openfdd-bridge, openfdd-commission, openfdd-mcp-rag.

14.2 Publish to GHCR (maintainers)

GitHub Actions workflow **Publish Docker addons** — manual dispatch; publishes **:latest** only and prunes GHCR (retired openfdd-bacnet-poll removed; keep 3 versions per image).

Maintainer checklist:

1. Green CI on master
2. Run **Publish Docker addons** workflow
3. Verify ghcr.io/bbartling/openfdd-{bridge,commission,mcp-rag}:latest
4. Edge hosts: docker compose pull && docker compose up -d

Details live in .github/workflows/ and scripts/docker_build.sh — not duplicated here.

15 Developer Guide

16 Developer Guide

Clone, build, test, and contribute to Open-FDD.

Topic	Page
Local setup	Local development
Docker builds	Build Docker images
Health checks	Health checks (developer)

Topic	Page
Tests & CI	Tests and CI
Security testing	Release & LAN scan cycle
PRs	Contributing

AI-assisted contributors: see repository AGENTS.md for workspace layout, skill routing, and agent policy (one paragraph in public docs; details stay internal).

17 Tests and CI

18 Tests and CI

18.1 Local tests

```
./scripts/build_and_test.sh # UI + workspace bridge
pytest
python3 -m pytest open_fdd/tests -q # PyPI package tests
cd workspace/dashboard && npm test # dashboard unit tests (if configured)
```

18.2 CI (GitHub Actions)

Workflow	Trigger	Checks
ci.yml	PR / push	pytest, bridge security audit, dashboard build, Jekyll docs
docs-pages.yml	push master	GitHub Pages site
publish-open-fdd.yml	tag open-fdd-v*	PyPI wheel
publish-docker-addons.yml	manual	GHCR images

18.3 Docs build (same as CI)

```
cd docs && bundle exec jekyll build -d /tmp/open-fdd-site
```

Fix any template or link errors before opening a PR that touches docs/.

18.4 Security-related CI

ci.yml includes bridge security tests (tests/workspace_bridge/test_security.py). That complements — but does not replace — LAN ZAP + Nmap scans on each patch revision. See Security testing cycle.

19 Contributing

20 Contributing

20.1 Expectations

- Open an issue or discuss large changes before big refactors.
- Keep PRs focused; include test or docs updates when behavior changes.
- Do not commit secrets (auth.env.local, Ansible secrets/*.env.local, commissioning CSVs with real site data).
- Follow existing code style in workspace/api/ and workspace/dashboard/.

20.2 Pull request checklist

☐

Tests pass locally (./scripts/build_and_test.sh or scoped pytest)

☐

Docs updated if user-facing behavior changed

☐

docs/ Jekyll build succeeds

☐

No credentials in diff

☐

Auth/Caddy/header changes: re-run pentest verify + LAN ZAP and update ZAP baseline if expectations changed

20.3 Repository layout (short)

Path	Contents
workspace/api/	Operator Bridge (FastAPI)
workspace/dashboard/	React SPA
open_fdd/	PyPI package (arrow_runtime, playground)
docker/	Compose files and image contexts
infra/ansible/	Edge deploy playbooks
docs/	This site (Jekyll + Just the Docs)

20.4 Security issues

Do not file public issues for vulnerabilities. Contact the maintainer or use GitHub private security advisories.

20.5 Agent-assisted development

Cursor/Claude contributors: read AGENTS.md and skills/ for automation boundaries. Public docs intentionally avoid agent playbook detail.

21 Health checks (developer)

22 Health checks (developer)

Extended validation for engineers with a **full git checkout** — CI, compose overlays, and pre-release gates.

IT operators on edge hosts should use Quick Start — health check and Deployment validation instead.

22.1 Clone and local stack

```
git clone https://github.com/bbartling/open-fdd.git && cd open-fdd
cp workspace/auth.env.example workspace/auth.env.local #
loopback dev only
./scripts/docker_build.sh # or: ./
scripts/openfdd_stack.sh prod
./scripts/openfdd_stack.sh up
```

GHCR production pull (no local build):

```
OPENFDD_IMAGE_TAG=2026.06.07-edge ./scripts/openfdd_stack.sh prod
```

22.2 Stack health script

```
./scripts/stack_health_check.sh
OPENFDD_BASE_URL=http://192.168.204.18:8765 ./scripts/
stack_health_check.sh
./scripts/stack_health_check.sh --require-ollama
```

Probes: containers up, /health, /api/model/health, sites, tree, SPARQL graph (with integrator auth when configured).

22.3 Compose overlays

Overlay	Use
docker/compose.dev.yml	Local dev bind-mount
docker/compose.bench.yml	Host-network BACnet lab
docker/compose.prod.yml	GHCR images instead of local build
docker/compose.edge.yml	IT edge template (copy to ~/open-fdd/ docker-compose.yml)

```
docker compose -f docker/compose.dev.yml -f docker/
compose.bench.yml ps
docker compose -f docker/compose.dev.yml config --images
```

22.4 BACnet poll pipeline

```
tail -1 workspace/bacnet/polls/samples.csv
cat workspace/data/bacnet_ingest_state.json
docker compose logs --since 10m commission | grep -i poll
docker compose logs --since 10m bridge | grep -i ingest
```

22.5 pytest (bridge security / BACnet)

```
PYTHONPATH=workspace/api pytest tests/workspace_bridge/
test_security.py -q
```

22.6 Pentest production stack (LAN ZAP)

```
./scripts/pentest_production_stack.sh start
./scripts/pentest_production_stack.sh verify
```

Then from a **LAN workstation**, run the packaged scan:

- Windows: scripts/security/Run-OpenFddSecurityScan.ps1
- macOS/Linux: scripts/security/run_openfdd_security_scan.sh

See [scripts/security/README.md](#), [Security — ZAP baseline](#), and the full [security testing cycle](#). Host-side notes: workspace/deploy/PENTEST.md.

22.7 Post-deploy insurance (inventory hosts)

For automated checks against named inventory hosts, see `infra/ansible/scripts/post_deploy_check.sh` (maintainer tooling — not required for IT docker-pull sites).

23 Security testing cycle

24 Security testing cycle

How Open-FDD validates each **patch revision** before it ships to the live Acme HVAC bench — local automation, AI-assisted PR review, LAN passive scans, and edge post-deploy checks.

Related security docs: [ZAP baseline](#), [TLS and certificates](#), [LAN hardening](#), [Linux host hardening](#), [Tenable remediation](#), [Authenticated scanning \(roadmap\)](#).

24.1 Who owns what

Concern	Owner	Where
HTTP security headers (CSP, X-Frame, COOP, ...)	Bridge middleware	workspace/api/openfdd_bridge/security_headers.py
TLS termination + HSTS	Caddy	

Concern	Owner	Where
OT LAN TLS certificates	Site integrator / maintainer	workspace/deploy/caddy/, Ansible Caddyfile.j2 ./scripts/setup_caddy_certs.sh → workspace/deploy/caddy/certs/; Ansible /etc/openfdd/caddy/ on edge
Pentest credentials	Generated per stack	workspace/auth.pentest.local (never commit)
LAN ZAP + Nmap scans	Developer workstation on test LAN	scripts/security/

See [TLS and certificate ownership](#) for cert lifecycle.

24.2 Per-revision workflow

Typical cycle on a **control machine** (local dev) through a **lab edge host** (live bench):

flowchart LR

```

A[Local dev + pytest] --> B[Pentest stack verify]
B --> C[LAN ZAP + Nmap]
C --> D[PR + GitHub CI]
D --> E[CodeRabbit AI review]
E --> F[Merge + tag open-fdd-vX.Y.Z]
F --> G[GHCR + PyPI publish]
G --> H[Acme upgrade + post_deploy_check --full]
H --> I[Bump next patch locally]

```

24.2.1 1. Local functional tests

```

./scripts/build_and_test.sh
PYTHONPATH=workspace/api pytest tests/workspace_bridge/
test_security.py -q
infra/ansible/scripts/bench_5007_arrow_battery.sh    # when FDD/
Arrow paths change

```

24.2.2 2. Pentest production stack (on Open-FDD host)

```
./scripts/pentest_production_stack.sh start
./scripts/pentest_production_stack.sh verify
```

Confirms Caddy :80 fronts the bridge, auth is enabled, BACnet writes disabled, and security headers are singular (no Caddy duplicates). Details: [Health checks — pentest](#).

24.2.3 3. Host + exposure checks (on edge VM)

After IT Nessus scans or before Acme deploy sign-off:

```
./scripts/security/check_host_security.sh
./scripts/security/check_openfdd_exposure.sh --edge-ip <lan-ip>
```

Host remediation (dry-run first): `./scripts/security/remediate_ubuntu_host.sh`

24.2.4 4. LAN security scan (from laptop)

Use the packaged scripts — do not hand-roll one-off docker/nmap commands unless debugging:

Platform	Command
Windows	<code>.\scripts\security\Run-OpenFddSecurityScan.ps1</code>
macOS / Linux	<code>./scripts/security/run_openfdd_security_scan.sh</code>

Setup (Docker Desktop, Nmap): [scripts/security/README.md](#).

Target URL is usually `http://<lan-ip>/` (Caddy), not `:8765`.

Review `openfdd-security-report/90-quick-findings-summary.txt` and compare against [ZAP baseline expectations](#).

24.2.5 5. Pull request + automated review

- Open PR against master
- **GitHub Actions** (`ci.yml`): pytest, bridge security audit, dashboard build, Jekyll docs
- **CodeRabbit** (or similar AI PR review): style, security footguns, missing tests/docs

Do not merge with failing CI or unresolved duplicate-header regressions.

24.2.6 6. Release

```
git tag open-fdd-v3.0.4 # example
git push origin open-fdd-v3.0.4
```

Tag triggers GHCR :latest and PyPI publish workflows.

24.2.7 7. Lab edge validation

After pulling new images on a live bench VM:

```
OPENFDD_IMAGE_TAG=latest ./scripts/upgrade_edge_full.sh --limit
<inventory_host>
    infra/ansible/scripts/post_deploy_check.sh --limit
<inventory_host> --full
```

Re-run LAN ZAP + Nmap against the edge LAN IP if Caddy or auth behavior changed. Site-specific example: [GL36 lab note](#).

24.2.8 8. Next cycle

Bump patch version locally (pyproject.toml, open_fdd/__init__.py) on master for the next development round — do not tag until the next release is ready.

24.3 What each scan layer covers

Layer	Tool	Depth
Unit / integration	pytest, test_security.py	Header middleware, auth gates
Stack smoke	pentest_production_stack.sh verify	Live headers, health, auth file
Passive web	ZAP baseline	Public routes, CSP, cookies, CORS
Port exposure	Nmap scoped	One host, known service ports
Authenticated API/dashboard	ZAP full + login	Roadmap — <u>Authenticated scanning</u>

Baseline ZAP does **not** exercise integrator login or protected /api/* routes deeply.

24.4 Open-source contributors

The `scripts/security/` folder is part of the public repo so any fork can run the same LAN smoke against their bench. PRs that change Caddy, auth, or `security_headers.py` should update [ZAP baseline](#) and re-run the scan scripts.

25 Python recipes (full Arrow library)

26 Python recipes — full Arrow library

Copy-paste `apply_faults_arrow` modules for Rule Lab. Replaces the legacy pandas/YAML Expression Rule Cookbook.

- **No pandas** on the edge — PyArrow + `pyarrow.compute` only
- Set `fault_code` in Rule Lab metadata (letter codes or Grade-A AHU-ECON-001)
- Map historian column names in the data model / rule bindings (ma-t, oa-t, stat_zn-t, ...)

Quick patterns: [Arrow recipes](#) · Index: [Expression cookbook \(Arrow-native\)](#)

26.1 Shared helpers (paste once per rule file)

```
import pyarrow as pa
import pyarrow.compute as pc

from open_fdd.arrow_runtime.cookbook import mixing_envelope_mask
from open_fdd.arrow_runtime.windows import (
    arrow_rolling_min,
    arrow_rolling_max,
    arrow_abs_diff,
)

def _norm_cmd(col: pa.ChunkedArray) -> pa.ChunkedArray:
    c = pc.cast(col, pa.float64())
    return pc.if_else(pc.greater(c, 1.0), pc.divide(c, 100.0), c)
```

```
def _col(table, name: str) -> pa.ChunkedArray:
    return pc.cast(table[name], pa.float64())
```

```
def _hypot_tol(a: float, b: float) -> float:
    return (a * a + b * b) ** 0.5
```

26.2 GL36 AHU Rules A-M

26.2.1 Rule A — duct static low at full fan speed (AHU-A)

```
FAULT_CODE = "AHU-A"
SP, SP_SP, FAN = "duct-static-pressure", "duct-static-pressure-sp", "supply-fan-speed-command"
SP_MARGIN, DRV_HI, DRV_NEAR = 0.12, 0.93, 0.06
```

```
def apply_faults_arrow(table, cfg, context=None):
    sp, sp_sp = _col(table, SP), _col(table, SP_SP)
    fan = _norm_cmd(table[FAN])
    return pc.and_(
        pc.less(sp, pc.subtract(sp_sp, SP_MARGIN)),
        pc.greater_equal(fan, DRV_HI - DRV_NEAR),
    )
```

26.2.2 Rules B & C — blended air outside OAT/RAT band (AHU-D)

Uses `mixing_envelope_mask` (covers below-band and above-band).

```
FAULT_CODE = "AHU-D"
```

```
def apply_faults_arrow(table, cfg, context=None):
    return mixing_envelope_mask(
        table,
        {**cfg, "mixing_tol": 1.15, "normalize_cmd_percent": 1},
        mat_col="ma-t",
        oat_col="oa-t",
        rat_col="ra-t",
        fan_col="supply-fan-speed-command",
    )
```

26.2.3 Rule D — discharge cold when heating commanded (AHU-B)

```
FAULT_CODE = "AHU-B"
BLEND_TOL, SAT_TOL, FAN_DT = 1.15, 1.15, 0.55

def apply_faults_arrow(table, cfg, context=None):
    mat, sat = _col(table, "ma-t"), _col(table, "sa-t")
    valve = _norm_cmd(table["heating-valve-command"])
    fan = _norm_cmd(table["supply-fan-speed-command"])
    rhs = pc.add(pc.subtract(mat, BLEND_TOL), FAN_DT)
    cold = pc.less_equal(pc.add(sat, SAT_TOL), rhs)
    return pc.and_(cold, pc.greater(valve, 0.01), pc.greater(fan,
0.01))
```

26.2.4 Rule E — SAT too low with full heating (AHU-C)

```
FAULT_CODE = "AHU-C"
SUPPLY_ERR = 1.0

def apply_faults_arrow(table, cfg, context=None):
    sat = _col(table, "sa-t")
    sp = _col(table, "sa-t-sp") if "sa-t-sp" in
table.column_names else _col(table, "supply-air-temperature-
setpoint")
    valve = _norm_cmd(table["heating-valve-command"])
    fan = _norm_cmd(table["supply-fan-speed-command"])
    return pc.and_(
        pc.less(sat, pc.subtract(sp, SUPPLY_ERR)),
        pc.greater(valve, 0.9),
        pc.greater(fan, 0.01),
    )
```

26.2.5 Rule F — SAT/MAT mismatch in economizer mode (AHU-E)

```
FAULT_CODE = "AHU-E"
FAN_DT, BLEND_TOL, SAT_TOL, ECON_MIN = 0.55, 1.15, 1.15, 0.12
TOL = _hypot_tol(SAT_TOL, BLEND_TOL)

def apply_faults_arrow(table, cfg, context=None):
    mat, sat = _col(table, "ma-t"), _col(table, "sa-t")
    damper = _norm_cmd(table["oa-damper-command"])
    cool = _norm_cmd(table["cooling-valve-command"])
    mismatch = pc.greater(pc.abs(pc.subtract(pc.subtract(sat,
FAN_DT), mat)), TOL)
```

```

        econ = pc.and_(pc.greater(damper, ECON_MIN), pc.less(cool,
0.1))
    return pc.and_(mismatch, econ)

```

26.2.6 Rule G — ambient too warm for free cooling (AHU-E)

```

FAULT_CODE = "AHU-E"
OAT_TOL, FAN_DT, SAT_TOL, ECON_MIN = 1.15, 0.55, 1.15, 0.12

def apply_faults_arrow(table, cfg, context=None):
    oat = _col(table, "oa-t")
    sp = _col(table, "sa-t-sp")
    damper = _norm_cmd(table["oa-damper-command"])
    cool = _norm_cmd(table["cooling-valve-command"])
    warm = pc.greater(oat, pc.add(pc.subtract(sp, FAN_DT),
SAT_TOL - OAT_TOL))
    econ = pc.and_(pc.greater(damper, ECON_MIN), pc.less(cool,
0.1))
    return pc.and_(warm, econ)

```

26.2.7 Rule H — OAT/MAT mismatch (econ + mech cooling) (AHU-E)

```

FAULT_CODE = "AHU-E"
OAT_TOL, BLEND_TOL = 1.15, 1.15
TOL = _hypot_tol(OAT_TOL, BLEND_TOL)

def apply_faults_arrow(table, cfg, context=None):
    oat, mat = _col(table, "oa-t"), _col(table, "ma-t")
    cool = _norm_cmd(table["cooling-valve-command"])
    damper = _norm_cmd(table["oa-damper-command"])
    return pc.and_(
        pc.greater(pc.abs(pc.subtract(mat, oat)), TOL),
        pc.greater(cool, 0.01),
        pc.greater(damper, 0.9),
    )

```

26.2.8 Rule I — OAT/MAT mismatch (economizer only) (AHU-E)

```

FAULT_CODE = "AHU-E"
TOL = _hypot_tol(1.15, 1.15)

def apply_faults_arrow(table, cfg, context=None):
    oat, mat = _col(table, "oa-t"), _col(table, "ma-t")

```

```

        damper = _norm_cmd(table["oa-damper-command"])
    return
pc.and_(pc.greater(pc.abs(pc.subtract(mat, oat)), TOL),
pc.greater(damper, 0.9))

```

26.2.9 Rule J — discharge above blended in cooling (AHU-B)

```

FAULT_CODE = "AHU-B"
FAN_DT, BLEND_TOL, SAT_TOL, ECON_MIN = 0.55, 1.15, 1.15, 0.12
TOL = _hypot_tol(SAT_TOL, BLEND_TOL)

def apply_faults_arrow(table, cfg, context=None):
    sat, mat = _col(table, "sa-t"), _col(table, "ma-t")
    cool = _norm_cmd(table["cooling-valve-command"])
    damper = _norm_cmd(table["oa-damper-command"])
    hot = pc.greater(sat, pc.add(mat, pc.add(TOL, FAN_DT)))
    mode_a = pc.and_(pc.greater(damper, 0.9), pc.greater(cool,
0.0))
    mode_b = pc.and_(pc.less_equal(damper, ECON_MIN),
pc.greater(cool, 0.9))
    return pc.and_(hot, pc.or_(mode_a, mode_b))

```

26.2.10 Rule K — discharge above setpoint in full cooling (AHU-C)

```

FAULT_CODE = "AHU-C"
SAT_TOL, ECON_MIN = 1.15, 0.12

def apply_faults_arrow(table, cfg, context=None):
    sat = _col(table, "sa-t")
    sp = _col(table, "sa-t-sp")
    cool = _norm_cmd(table["cooling-valve-command"])
    damper = _norm_cmd(table["oa-damper-command"])
    high = pc.greater(sat, pc.add(sp, SAT_TOL))
    full = pc.or_(
        pc.and_(pc.greater(damper, 0.9), pc.greater(cool, 0.9)),
        pc.and_(pc.less_equal(damper, ECON_MIN), pc.greater(cool,
0.9)),
    )
    return pc.and_(high, full)

```

26.2.11 Rule L — cooling coil ΔT when inactive (CH-c)

```

FAULT_CODE = "CH-C"
ENTER_TOL, LEAVE_TOL, ECON_MIN = 1.15, 1.15, 0.12

```

```
TOL = _hypot_tol(ENTER_TOL, LEAVE_TOL)
```

```
def apply_faults_arrow(table, cfg, context=None):
    ent = _col(table, "cooling-coil-entering-air-temperature")
    leave = _col(table, "cooling-coil-leaving-air-temperature")
    heat = _norm_cmd(table["heating-valve-command"])
    cool = _norm_cmd(table["cooling-valve-command"])
    damper = _norm_cmd(table["oa-damper-command"])
    drop = pc.greater(pc.subtract(ent, leave), TOL)
    mode_a = pc.and_(pc.greater(heat, 0), pc.equal(cool, 0),
pc.less_equal(damper, ECON_MIN))
    mode_b = pc.and_(pc.equal(heat, 0), pc.equal(cool, 0),
pc.greater(damper, ECON_MIN))
    return pc.and_(drop, pc.or_(mode_a, mode_b))
```

26.2.12 Rule M — heating coil ΔT when inactive (AHU-B)

```
FAULT_CODE = "AHU-B"
ENTER_TOL, LEAVE_TOL, FAN_DT, ECON_MIN = 1.15, 1.15, 0.55, 0.12
TOL = _hypot_tol(ENTER_TOL, LEAVE_TOL) + FAN_DT
```

```
def apply_faults_arrow(table, cfg, context=None):
    ent = _col(table, "heating-coil-entering-air-temperature")
    leave = _col(table, "heating-coil-leaving-air-temperature")
    heat = _norm_cmd(table["heating-valve-command"])
    cool = _norm_cmd(table["cooling-valve-command"])
    damper = _norm_cmd(table["oa-damper-command"])
    rise = pc.greater(pc.subtract(leave, ent), TOL)
    m1 = pc.and_(pc.equal(heat, 0), pc.equal(cool, 0),
pc.greater(damper, ECON_MIN))
    m2 = pc.and_(pc.equal(heat, 0), pc.greater(cool, 0),
pc.greater(damper, 0.9))
    m3 = pc.and_(pc.equal(heat, 0), pc.greater(cool, 0),
pc.less_equal(damper, ECON_MIN))
    return pc.and_(rise, pc.or_(m1, m2, m3))
```

26.3 Starter pack (VAV / AHU baseline)

26.3.1 01_vav_zone_temp_bounds_occupied (VAV-C)

```
from open_fdd.arrow_runtime.cookbook import sensor_bounds_mask,
_unoccupied_mask
```

```
FAULT_CODE = "VAV-C"
```

```
def apply_faults_arrow(table, cfg, context=None):
    occupied = pc.invert(_unoccupied_mask(table, cfg))
    oob = sensor_bounds_mask(table, "zone_temp", cfg)
    return pc.and_(oob, occupied)
```

26.3.2 02_vav_zone_temp_flatline_occupied (VAV-C)

```
from open_fdd.arrow_runtime.cookbook import sensor_flatline_mask,
_unoccupied_mask
```

```
FAULT_CODE = "VAV-C"
```

```
def apply_faults_arrow(table, cfg, context=None):
    occupied = pc.invert(_unoccupied_mask(table, cfg))
    flat = sensor_flatline_mask(table, "zone_temp", cfg)
    return pc.and_(flat, occupied)
```

26.3.3 03_vav_damper_command_extreme_flatline (VAV-D)

```
from open_fdd.arrow_runtime.cookbook import flatline_1h_mask
```

```
FAULT_CODE = "VAV-D"
DAMPER = "damper-position-command"
```

```
def apply_faults_arrow(table, cfg, context=None):
    stuck = flatline_1h_mask(table, {**cfg, "column": DAMPER})
    wide = pc.greater(_norm_cmd(table[DAMPER]), 0.975)
    return pc.and_(stuck, wide)
```

26.3.4 04_ahu_runtime_outside_schedule (BLD-C)

Use run-hours analytics script + schedule compare; boolean mask:

```
from open_fdd.arrow_runtime.cookbook import _fan_on_mask,
_unoccupied_mask
```

```
FAULT_CODE = "BLD-C"
```

```
def apply_faults_arrow(table, cfg, context=None):
    return pc.and_( _fan_on_mask(table, cfg),
_unoccupied_mask(table, cfg))
```

26.3.5 05 / 06 / 07 — see Rules A, sensor_bounds_mask on sa-t / sensor_flatline_mask on sa-t

26.4 Economizer starters

26.4.1 ahu_econ_100oa_temp_tracking_fault (AHU-E)

```
FAULT_CODE = "AHU-E"
DAMPER_FULL, FAN_ON, TRACK_TOL, FAN_HEAT = 0.95, 0.5, 2.0, 0.5

def apply_faults_arrow(table, cfg, context=None):
    oat, mat, sat = _col(table, "oa-t"), _col(table, "ma-t"),
    _col(table, "sa-t")
    damper = _norm_cmd(table["oa-damper-command"])
    fan = _col(table, "supply-fan-status")
    econ = pc.and_(pc.greater(fan, FAN_ON),
pc.greater_equal(damper, DAMPER_FULL))
    mat_bad = pc.greater(pc.abs(pc.subtract(mat, oat)), TRACK_TOL)
    sat_bad = pc.greater(pc.abs(pc.subtract(pc.subtract(sat,
FAN_HEAT), mat))), TRACK_TOL)
    return pc.and_(econ, pc.or_(mat_bad, sat_bad))
```

26.4.2 ahu_mech_cooling_when_free_cooling_available (AHU-E)

```
FAULT_CODE = "AHU-E"
MARGIN, DAMPER_LOW, COOL_ON = 2.0, 0.50, 0.10

def apply_faults_arrow(table, cfg, context=None):
    oat = _col(table, "oa-t")
    sp = _col(table, "sa-t-sp")
    damper = _norm_cmd(table["oa-damper-command"])
    cool = _norm_cmd(table["cooling-valve-command"])
    free = pc.less_equal(oat, pc.subtract(sp, MARGIN))
    return pc.and_(free, pc.less(damper, DAMPER_LOW),
pc.greater_equal(cool, COOL_ON))
```

26.4.3 ahu_oa_damper_excess_open_extreme_ambient (AHU-E)

```
FAULT_CODE = "AHU-E"
DAMPER_MIN, HI_OAT, LO_OAT = 0.50, 70.0, 40.0

def apply_faults_arrow(table, cfg, context=None):
    oat = _col(table, "oa-t")
```



```

damper = _norm_cmd(table["oa-damper-command"])
extreme = pc.or_(pc.greater_equal(oat, HI_OAT),
pc.less_equal(oat, LO_OAT))
return pc.and_(pc.greater_equal(damper, DAMPER_MIN), extreme)

```

26.5 VAV zones

26.5.1 zone_reheat_warm_ambient (VAV-A)

```

FAULT_CODE = "VAV-A"
T_CUTOFF, REHEAT_MIN = 78.0, 0.52

def apply_faults_arrow(table, cfg, context=None):
    oat = _col(table, "oa-t")
    reheat = _norm_cmd(table["reheat-valve-command"])
    return pc.and_(pc.greater(oat, T_CUTOFF), pc.greater(reheat,
REHEAT_MIN))

```

26.5.2 zone_damper_valve_full_open (VAV-D)

```

FAULT_CODE = "VAV-D"
FULL_OPEN, ROLL = 97.5, 105

def apply_faults_arrow(table, cfg, context=None):
    d = _norm_cmd(table["damper-position-command"])
    hi = pc.greater(d, FULL_OPEN)
    rmin = arrow_rolling_min(d, ROLL)
    return pc.and_(hi, pc.greater(rmin, FULL_OPEN))

```

26.5.3 Zone / IAQ bounds

Use `sensor_bounds_mask(table, "zone_temp", cfg)` or `sensor_bounds_mask` with CO₂ profile — see [sensor validation](#).

26.6 Central plant

26.6.1 dp_below_sp_pump_max (CH-E)

```

FAULT_CODE = "CH-E"
DP_MARGIN, PMP_HI, PMP_NEAR = 2.2, 0.93, 0.06

def apply_faults_arrow(table, cfg, context=None):

```

```

dp = _col(table, "differential-pressure")
sp = _col(table, "differential-pressure-sp")
pump = _norm_cmd(table["pump-speed-command"])
return pc.and_(
    pc.less(dp, pc.subtract(sp, DP_MARGIN)),
    pc.greater_equal(pump, PMP_HI - PMP_NEAR),
)

```

26.6.2 flow_high_pump_max (CH-B)

```

FAULT_CODE = "CH-B"
FLOW_HI, PMP_HI, PMP_NEAR = 1100.0, 0.93, 0.06

```

```

def apply_faults_arrow(table, cfg, context=None):
    flow = _col(table, "water-flow")
    pump = _norm_cmd(table["pump-speed-command"])
    return pc.and_(pc.greater(flow, FLOW_HI),
pc.greater_equal(pump, PMP_HI - PMP_NEAR))

```

26.6.3 plant_supply_temp_deadband (CH-D)

```

FAULT_CODE = "CH-D"
SP_BAND = 2.2

```

```

def apply_faults_arrow(table, cfg, context=None):
    temp = _col(table, "chilled-water-supply-temperature")
    sp = _col(table, "chilled-water-supply-temperature-sp")
    pump = _norm_cmd(table["pump-speed-command"])
    low = pc.less(temp, pc.subtract(sp, SP_BAND))
    high = pc.greater(temp, pc.add(sp, SP_BAND))
    return pc.and_(pc.greater(pump, 0.01), pc.or_(low, high))

```

26.6.4 chiller_excessive_runtime (CH-F)

```

from open_fdd.arrow_runtime.cookbook import arrow_rolling_mean

```

```

FAULT_CODE = "CH-F"
WINDOW, MAX_ON = 276, 264 # 5-min data ≈ 23 h window, 22 h max on

```

```

def apply_faults_arrow(table, cfg, context=None):
    on = pc.greater(_col(table, "chiller-status"),
0.5).cast(pa.float64())
    roll = arrow_rolling_mean(on, WINDOW)
    count = pc.multiply(roll, float(WINDOW))
    return pc.greater(count, float(MAX_ON))

```

26.7 Heat pumps

26.7.1 hp_discharge_cold_when_heating (HP-D)

```
FAULT_CODE = "HP-D"  
MIN_SAT, ZONE_COLD, FAN_ON = 85.0, 69.0, 0.01
```

```
def apply_faults_arrow(table, cfg, context=None):  
    sat = _col(table, "sa-t")  
    zone = _col(table, "stat_zn-t")  
    fan = _col(table, "supply-fan-status")  
    return pc.and_(  
        pc.greater(fan, FAN_ON),  
        pc.less(zone, ZONE_COLD),  
        pc.less(sat, MIN_SAT),  
    )
```

26.8 Opportunistic / ventilation

26.8.1 econ_active_warm_ambient (AHU-E)

```
FAULT_CODE = "AHU-E"  
T_CUTOFF, DPR_MIN = 63.0, 0.42
```

```
def apply_faults_arrow(table, cfg, context=None):  
    oat = _col(table, "oa-t")  
    damper = _norm_cmd(table["oa-damper-command"])  
    return pc.and_(pc.greater(oat, T_CUTOFF), pc.greater(damper,  
DPR_MIN))
```

26.8.2 mech_cool_when_econ_available (AHU-E)

```
FAULT_CODE = "AHU-E"  
T_CUTOFF, DPR_MAX = 63.0, 0.32
```

```
def apply_faults_arrow(table, cfg, context=None):  
    oat = _col(table, "oa-t")  
    damper = _norm_cmd(table["oa-damper-command"])  
    cool = _norm_cmd(table["cooling-valve-command"])  
    return pc.and_(pc.less(oat, T_CUTOFF), pc.less(damper,  
DPR_MAX), pc.greater(cool, 0.01))
```

26.8.3 low_oa_fraction_estimated (VAV-B)

```
FAULT_CODE = "VAV-B"  
OA_MIN, GAP = 21.0, 2.2
```

```
def apply_faults_arrow(table, cfg, context=None):  
    mat, rat, oat = _col(table, "ma-t"), _col(table, "ra-t"),  
    _col(table, "oa-t")  
    fan = _norm_cmd(table["supply-fan-speed-command"])  
    denom = pc.subtract(oat, rat)  
    safe = pc.greater(pc.abs(denom), GAP)  
    num = pc.subtract(mat, rat)  
    oa_pct = pc.multiply(pc.divide(num, denom), 100.0)  
    low = pc.less(oa_pct, OA_MIN)  
    return pc.and_(pc.greater(fan, 0.01), safe, low)
```

26.8.4 preheat_excess_temp (AHU-B)

```
FAULT_CODE = "AHU-B"  
EXCESS = 2.2
```

```
def apply_faults_arrow(table, cfg, context=None):  
    pre = _col(table, "preheat-coil-leaving-air-temperature")  
    sp = _col(table, "sa-t-sp")  
    oat = _col(table, "oa-t")  
    valve = _norm_cmd(table["preheat-valve-command"])  
    hot_oa = pc.and_(  
        pc.greater(oat, sp),  
        pc.greater(pc.subtract(pre, oat), EXCESS),  
    )  
    cold_oa = pc.and_(  
        pc.less(oat, sp),  
        pc.greater(pc.subtract(pre, sp), EXCESS),  
    )  
    return pc.and_(pc.greater(valve, 0.01), pc.or_(hot_oa,  
cold_oa))
```

26.9 Weather station

26.9.1 weather_temp_spike (BLD-B)

```
from open_fdd.arrow_runtime.cookbook import rate_of_change_mask  
  
FAULT_CODE = "BLD-B"
```

```

def apply_faults_arrow(table, cfg, context=None):
    return rate_of_change_mask(
        table,
        {**cfg, "max_per_sample": 16.0, "column": "oa-t"},
        col="oa-t",
    )

```

26.9.2 weather_gust_lt_wind (BLD-B)

```

FAULT_CODE = "BLD-B"

```

```

def apply_faults_arrow(table, cfg, context=None):
    gust = _col(table, "wind-gust-speed")
    wind = _col(table, "wind-speed")
    return pc.and_(
        pc.is_valid(gust),
        pc.is_valid(wind),
        pc.less(gust, wind),
    )

```

26.9.3 RH bounds

```

from open_fdd.arrow_runtime.cookbook import sensor_bounds_mask

```

```

FAULT_CODE = "DC-C"

```

```

def apply_faults_arrow(table, cfg, context=None):
    merged = {**cfg, "value_kind": "rh"}
    return sensor_bounds_mask(table, "relative_humidity", merged)

```

26.10 Schedule + weather gating (occupied hours)

```

from open_fdd.arrow_runtime.cookbook import _unoccupied_mask,
_fan_on_mask

```

```

FAULT_CODE = "BLD-C"

```

```

def apply_faults_arrow(table, cfg, context=None):
    """Fan running when schedule says unoccupied."""
    return pc.and_( _fan_on_mask(table, cfg),
        _unoccupied_mask(table, cfg))

```

Pass occupied_start_hour, occupied_end_hour, tz_offset_hours in Rule Lab cfg.

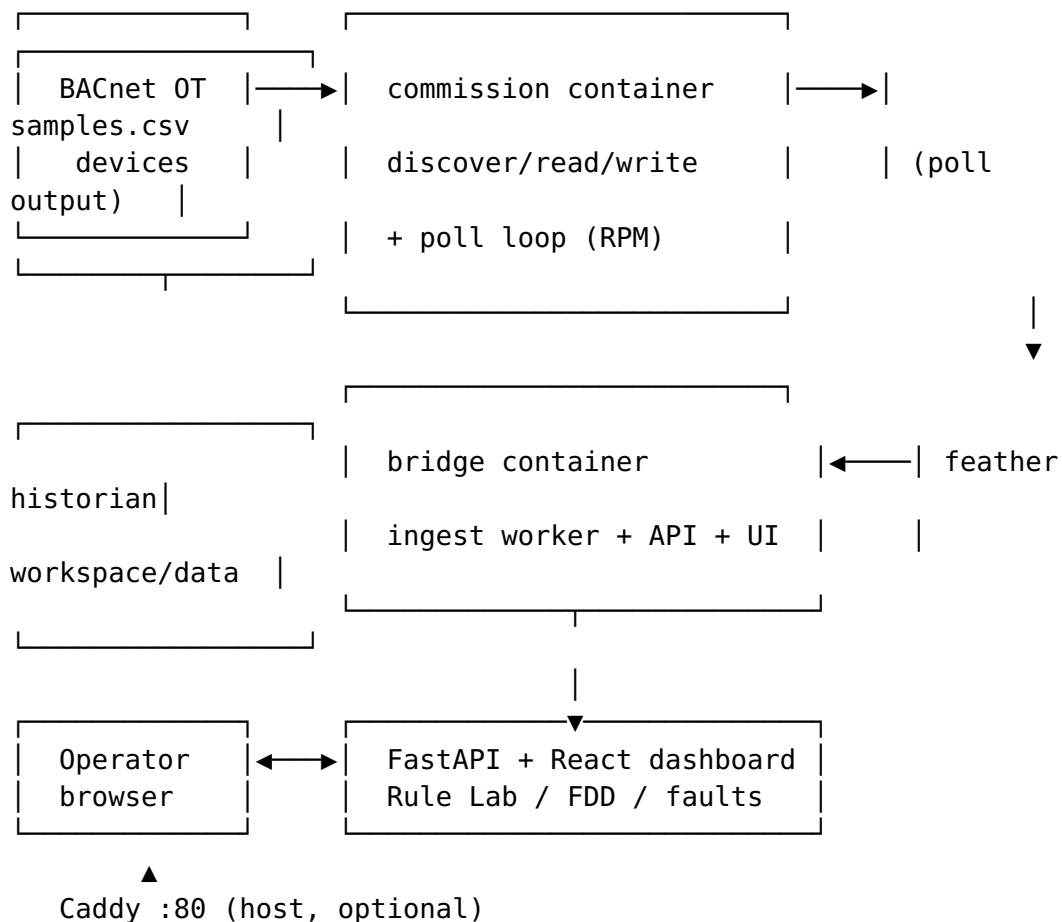
26.11 Next steps

1. Copy a module into workspace/data/rules_py/<site>_<rule>.py
2. Bind feather columns on the Rule Lab row
3. Set fault_code metadata
4. Quick-test on 3-24 h feather window
5. Ship via setup_gl36_fdd.py or Rule Lab save

See also: [Expression cookbook](#) · [Fault codes](#) · [Rule Lab](#)

27 System overview

28 System overview



28.1 Docker edge stack (production)

Three GHCR images — see [Containers](#):

Container	BACnet OT	Operator LAN
bridge	No (HTTP only)	Yes, via Caddy :80 or loopback :8765
commission	Yes (host network, :47808)	No (internal :8767)
mcp-rag	No	No (internal :8090)

BACnet **polling** is not a separate container — it runs inside **commission**. The bridge only **ingests** poll CSV into feather.

28.2 Components

Layer	Responsibility
Operator Bridge	Auth, REST/WebSocket API, compiled dashboard, Rule Lab, historian ingest
BACnet commission	Who-Is, read/write (gated), scheduled poll loop → samples.csv
Historian	Feather files per point; local-first retention
FDD runner	workspace/data/rules_py/*.py on timer or POST /api/rules/batch
Model	Brick TTL + model.json bindings (fdd_input, equipment tree)
MCP RAG	Optional doc retrieval for agent tools
Caddy	Host reverse proxy :80 → bridge (typical edge)
Ollama	Optional host LLM for building insight (not required for FDD)

28.3 Deployment

IT operators: [Quick Start — Docker](#). Modes and Caddy: [Deployment modes](#).

28.4 Optional PyPI-only path

pip install open-fdd ships Arrow runtime + playground lint helpers **without** the Operator Bridge UI. Use for offline rule tests or CI. See [Appendix — Python package](#).

29 Expression cookbook (Arrow-native)

30 Expression cookbook (Arrow-native)

Reference for **Open-FDD 3.x Rule Lab**: every rule is a Python module with `apply_faults_arrow(table, cfg, context)` using `pyarrow.compute` — **no pandas, no YAML expression files, no NumPy DataFrames on the IoT edge**.

This is the **only** expression cookbook for Open-FDD 3.x. The old pandas/YAML page is retired. Legacy GL36-style recipes map to **fixed fault codes** and Arrow patterns below.

Topic	Page
Full copy-paste library (GL36 A-M, VAV, plant)	<u>Python recipes (full Arrow library)</u>
Quick templates	<u>Arrow recipes</u>
Shared imports	<u>Python recipes</u>
Console window stats	<u>Lookback window</u>
Fault code catalog	<u>Fault codes</u>
Programmatic defaults	<code>open_fdd.arrow_runtime.sensor_catalog</code>

30.1 Legacy → modern translation

Legacy (pandas / YAML)	Arrow-native (3.x)
<code>type: expression + expression: string</code>	<code>apply_faults_arrow()</code> in <code>rules_py/*.py</code>
<code>RuleRunner.run(df, column_map=...)</code>	Rule Lab batch on feather PyArrow table
<code>np.maximum(a, b)</code>	<code>pc.max(a, b)</code>
<code>series.rolling(n).min()</code>	<code>arrow_rolling_min(vals, n)</code> from <code>open_fdd.arrow_runtime.windows</code>
<code>series.diff().abs()</code>	<code>arrow_abs_diff(vals, 1)</code>
<code>normalize_cmd(x)</code>	<code>pc.if_else(pc.greater(x, 1), pc.divide(x, 100), x)</code>
<code>params.max_temp</code> in YAML	

Legacy (pandas / YAML)	Arrow-native (3.x)
	Module constant or cfg["bounds_high"]
flag: rule_a_flag	Rule metadata fault_code → AHU-A ... VAV-C
Numeric codes VAV-03	Letter codes VAV-C (see LEGACY_CODE_MAP in bridge)

Edge constraint: Docker / BACnet poll path never imports pandas. Central portfolio Dash may use pandas for CSV analytics only.

30.2 How Arrow rules are structured

1. **Historian columns** — Feather column names from BACnet poll (oa-t, sa-t, stat_zn-t, ...) or Brick labels via model export.
2. **Module constants** — Thresholds at top of file (bench style) or read from cfg for site tuning.
3. **apply_faults_arrow** — Returns a **boolean PyArrow array** (True = fault sample).
4. **fault_code** — Set in Rule Lab metadata; must exist in GET /api/faults/catalog.

```
import pyarrow.compute as pc
from open_fdd.arrow_runtime.cookbook import sensor_bounds_mask

FAULT_CODE = "VAV-C" # metadata in Rule Lab UI
```

```
def apply_faults_arrow(table, cfg, context=None):
    return sensor_bounds_mask(table, "zone_temp", cfg)
```

30.2.1 Command scaling (0-1 vs 0-100 %)

BACnet often exposes **0-100** for damper/VFD commands. Scale explicitly:

```
def _norm_cmd(col):
    return pc.if_else(pc.greater(col, 1.0), pc.divide(col,
100.0), col)
```

30.2.2 Occupied hours (schedule gating)

Use open_fdd.arrow_runtime.cookbook._unoccupied_mask or compare local hour from timestamp column. Example: **fan on when unoccupied** → **BLD-C** / schedule_compare.

30.3 Sensor validation (bounds, flatline, rate of change)

Use `open_fdd.arrow_runtime.sensor_catalog.SENSOR_PROFILES` for defaults, or copy constants into `rules_py`. Tune per site in Rule Lab `cfg` or building-agent tuning brief.

30.3.1 Bounds (out of range) — `oob_rolling`

Sensor kind	Min	Max	Flatline tol	Max Δ / hour	Max Δ / 15 min	Typical fault code
Zone temp (°F)	55	90	0.10	4.0	2.0	VAV-C
Supply air temp	50	110	0.15	8.0	3.0	AHU-C, RTU-C
Return air temp	55	95	0.10	3.0	1.5	AHU-D (also mixing)
Mixed air temp	40	110	0.15	6.0	2.5	AHU-D
Outdoor air temp	−40	130	0.10	12.0	6.0	BLD-B
Duct static (inH ₂ O)	−0.5	3.0	0.02	0.5	0.25	AHU-A
Relative humidity (%)	0	100	1.0	15.0	8.0	DC-C
Chilled water (°F)	40	90	0.10	4.0	2.0	CH-D
Hot water (°F)	70	200	0.15	6.0	3.0	CH-D
Condenser water (°F)	50	110	0.15	5.0	2.5	CH-A
CO ₂ (ppm, occupied)	400	1000	5.0	200	80	VAV-B (ventilation)
Discharge air temp	45	120	0.15	10.0	4.0	VAV-E, HP-D

Return air uses a **narrow band** vs zone/OAT because RAT should track building return conditions, not outdoor extremes. When OAT/RAT/MAT are all present, prefer **mixing envelope** (below) over RAT bounds alone.

CO₂ upper bound is an occupied ventilation target; unoccupied rules may use 400–2000 ppm per policy.

30.3.2 Flatline (stuck sensor) — flatline_1h

Default **12 samples ≈ 1 hour** at 5-minute poll. True when rolling $\max - \min \leq \text{tolerance}$.

```
from open_fdd.arrow_runtime.cookbook import sensor_flatline_mask

def apply_faults_arrow(table, cfg, context=None):
    return sensor_flatline_mask(table, "outdoor_air_temp", cfg)
# BLD-B
```

Bench references: workspace/data/rules_py/bench_oa-t_flatline_1h.py, bench_stat_zn-t_flatline_1h.py.

30.3.3 Rate of change (spike) — rate_of_change

Flags physically impossible step changes (comms glitch, substituted value).

```
from open_fdd.arrow_runtime.cookbook import rate_of_change_mask
from open_fdd.arrow_runtime.sensor_catalog import cfg_from_profile

def apply_faults_arrow(table, cfg, context=None):
    merged = cfg_from_profile("outdoor_air_temp", cfg)
    merged["samples_per_hour"] = 12 # 5-min poll
    return rate_of_change_mask(table, merged, col="oa-t")
```

Legacy **weather_temp_spike** (spike_limit: 16 °F per step) → **BLD-B** with max_per_15min: 6 or stricter per-step limit.

30.3.4 Mixing envelope (MAT vs OAT/RAT) — mixing_envelope

Legacy GL36 **Rule B / C** and **AHU-D**. MAT should sit between OAT and RAT ($\pm$ tolerance) while fan runs.

```
from open_fdd.arrow_runtime.cookbook import mixing_envelope_mask

def apply_faults_arrow(table, cfg, context=None):
    return mixing_envelope_mask(
        table,
        {**cfg, "mixing_tol": 1.15},
        mat_col="ma-t",
        oat_col="oa-t",
        rat_col="ra-t",
        fan_col="supply-fan-speed-command",
    )
```

30.4 GL36-inspired AHU rules (legacy expression → fault code)

ASHRAE Guideline 36-style rules from the old YAML cookbook, translated to Arrow. Assign **fault_code** per row in Rule Lab.

Legacy rule	Summary	Code	Full Arrow module
Rule A — duct static low @ full fan	SP below SP setpoint at high VFD	AHU-A	Rule A
Rule B — blend below band	MAT below OAT/RAT envelope	AHU-D	Rules B & C
Rule C — blend above band	MAT above OAT/RAT envelope	AHU-D	Rules B & C
Rule D — discharge cold when heating	SAT low vs MAT, heat valve open	AHU-B	Rule D
Rule E — SAT low, full heating	SAT below SP, valve > 90%	AHU-C	Rule E
Rule F — SAT/MAT mismatch econ	Econ mode, SAT ≠ MAT	AHU-E	Rule F
Rule G — ambient warm free cool	OAT > SAT SP, econ open, cool off	AHU-E	Rule G
Rule H — OAT/MAT mismatch econ+mech	Mech + econ, MAT ≠ OAT	AHU-E	Rule H
Rule I — OAT/MAT mismatch econ-only	Econ only, MAT ≠ OAT	AHU-E	Rule I
Rule J — discharge above blend cooling	SAT > MAT in cooling	AHU-B	Rule J
Rule K — discharge above SP full cool	SAT > SP, full cooling	AHU-C	Rule K

Legacy rule	Summary	Code	Full Arrow module
Rule L — cooling coil ΔT when off	CHW drop when valves closed	CH-C	Rule L
Rule M — heating coil ΔT when off	HW rise when valves closed	AHU-B	Rule M

All rules A-M have **full apply_faults_arrow modules** in [Python recipes \(full Arrow library\)](#).

30.5 Starter pack (VAV / AHU baseline)

Maps to legacy YAML starter filenames; use as first deploy bundle.

Legacy YAML starter	Arrow approach	Fault code
01_vav_zone_temp_bounds_occupied	sensor_bounds_mask(..., "zone_temp") + occupied mask	VAV C
02_vav_zone_temp_flatline_occupied	sensor_flatline_mask(..., "zone_temp") + occupied	VAV C
03_vav_damper_command_extreme_flatline	flatline_1h_mask on damper cmd column	VAV D
04_ahu_runtime_outside_schedule	after_hours_fan_satisfied_mask / run-hours script	BL C
05_ahu_duct_static_pressure_not_maintained	Rule A	AH A
06_ahu_internal_temp_sensor_bounds	sensor_bounds_mask on SAT/MAT	AH C
07_ahu_internal_temp_sensor_flatline	sensor_flatline_mask on SAT	AH C

30.5.1 Economizer starters

Full modules: [economizer section](#)
(ahu_econ_100oa_temp_tracking_fault,
ahu_mech_cooling_when_free_cooling_available,
ahu_oa_damper_excess_open_extreme_ambient).

30.6 VAV, central plant, heat pump

Full modules: [VAV](#) · [Central plant](#) · [Heat pumps](#).

Rolling persistence: `arrow_rolling_min + pc.and_` (replaces `pandas.rolling(n).min()`).

30.7 Opportunistic / ventilation

Full modules: [Opportunistic section](#) · [Weather](#).

30.8 Data quality vs equipment faults

Symptom	Pattern	Code	Do not confuse with
Flatline 1h	flatline_1h	VAV-C, AHU-C, BLD-B	Equipment off / damper closed
OOB sample	oob_rolling	sensor-specific	True process excursion
Spike between polls	rate_of_change	BLD-B	Fast legitimate weather front
No new samples	stale_points	BLD-D	Equipment fault
MAT ∉ [OAT, RAT]	mixing_envelope	AHU-D	Stratification during startup

Always gate sensor faults with **fan on, valve open, or occupied** where appropriate — see [Sensor & data quality](#).

30.9 Metric (°C) equivalents

Set `cfg["temp_unit"] = "metric"` in Rule Lab or scale constants:

Imperial	Metric (approx)
55 °F	12.8 °C
90 °F	32.2 °C
50 °F	10.0 °C
110 °F	43.3 °C

Imperial	Metric (approx)
2.0 °F tol	1.1 °C
0.15 inH ₂ O	~37 Pa

open_fdd.playground.temp_units documents UI field keys for °F/°C toggle.

30.10 Binding rules to the fault catalog

1. Pick a **letter code** from [Fault codes](#) — never invent suffixes.
2. In Rule Lab, set **fault_code** on the rule row.
3. Batch FDD aggregates into GET /api/faults/status and portfolio rollup.
4. Legacy numeric codes (AHU-03) migrate via LEGACY_CODE_MAP in the bridge.

```
curl -s http://127.0.0.1:8765/api/faults/catalog | jq
'.families[].codes[] | select(.code=="VAV-C")'
```

30.11 Pandas YAML → Arrow checklist (commissioning)

☐

Replace each type: expression YAML with a rules_py module

☐

Map inputs Brick labels → feather column names in model/poll CSV

☐

Set fault_code per rule (letter suffix)

☐

Run Rule Lab kit on 7-day feather window; confirm flag rate sane

☐

Apply sensor_catalog defaults; tune bounds for site climate

☐

Enable building-agent check-in; verify faults in portfolio rollup

Next: [Python recipes \(full Arrow library\)](#) · [Arrow recipes](#) · [Fault codes](#) · [Rule Lab](#)

31 Deployment modes

32 Deployment modes

How Open-FDD is typically run: local development, lab LAN trials, and production edge on a trusted OT/IT network. Open-FDD is designed for **building LAN or OT edge** deployment — not direct exposure to the public internet.

32.1 Mode summary

Mode	Stack	BACnet	Front door	Auth
Local dev	Compose on control machine	Optional bench simulator	127.0.0.1:8765	OFDD_AUTH_DISABLED=1 optional on loopback
Lab LAN	Three GHCR containers	Real or simulator VLAN	Caddy :80 or direct :8765	Required unless explicit insecure lab flags
Edge production	Three GHCR containers + host Caddy	OT NIC via commission	Caddy :80 or :443 → loopback bridge	Required — workspace/auth.env.local

32.2 v3 container stack (all edge modes)

Image	Role
openfdd-bridge	Operator Bridge API, dashboard, feather historian ingest
openfdd-commission	BACnet discover/read/write + poll loop
openfdd-mcp-rag	Doc-search sidecar for agent tools

openfdd-bacnet-poll is retired. BACnet polling runs inside **commission** only. Do not run the legacy poll container or openfdd-bacnet-poll systemd unit alongside Docker commission — that double-polls the OT network.

See [Containers](#) for ports, networks, and persistence.

32.3 Caddy HTTP (typical edge)

- Caddy listens on building LAN :80
- Reverse-proxies to bridge at 127.0.0.1:8765
- Bridge stays loopback-only; operators bookmark `http://<edge-ip>/`

Bootstrap and compose: [Quick Start — Docker](#).

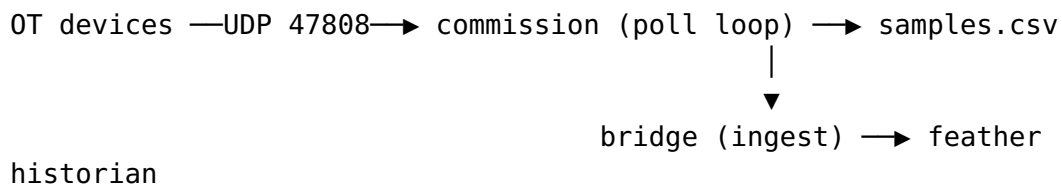
32.4 Caddy TLS (OT LAN)

For HTTPS on the edge LAN (self-signed or site CA):

1. Generate certs: `./scripts/setup_caddy_certs.sh` (control machine) or site PKI
2. Start with `OFDD_CADDY_MODE=tls`
3. Trust the CA on operator PCs

Details: [TLS and certificates](#). Security headers are set by the **Operator Bridge**; Caddy adds **HSTS** only in TLS mode.

32.5 BACnet data path



The bridge thread named `bacnet_poll_worker` is **ingest only** — it does not bind BACnet.

32.6 What not to do on production edges

- Port-forward :8765 to the public internet without TLS and strong auth
- Run `docker compose down -v` or delete workspace/ on a live site
- Deploy the retired `openfdd-bacnet-poll` image alongside commission

32.7 Related

Topic	Page
First deploy	Quick Start — Docker
Upgrade	Updating the stack
LAN security	LAN hardening
Lab example site	Examples — GL36 lab note

33 Arrow recipes

34 Arrow recipes (default)

Open-FDD 3.0 Rule Lab rules use `apply_faults_arrow(table, cfg, context)`, `pyarrow.compute`, and **module constants** (no `config.json`).

For **full GL36 A-M and plant recipes**, see [Python recipes \(full Arrow library\)](#). Sensor tables: [Expression cookbook](#).

34.1 Simple threshold

```
import pyarrow.compute as pc

VALUE_COLUMN = "zone_temp"
MAX_ZONE_TEMP = 78.0

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table[VALUE_COLUMN], MAX_ZONE_TEMP)
```

34.2 Supply air temp high

```
import pyarrow.compute as pc

VALUE_COLUMN = "sa-t"
HIGH_F = 75.0

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(pc.cast(table[VALUE_COLUMN], "float64"),
HIGH_F)
```

34.3 Fan commanded but no airflow

```
import pyarrow.compute as pc

FAN_CMD = "fan-cmd"
AIRFLOW = "supply-cfm"
FAN_ON = 0.5
MIN_CFM = 500.0

def apply_faults_arrow(table, cfg, context=None):
    fan_on = pc.greater(table[FAN_CMD], FAN_ON)
    low_airflow = pc.less(table[AIRFLOW], MIN_CFM)
    return pc.and_(fan_on, low_airflow)
```

34.4 Flatline (1 h rolling window)

```
import pyarrow.compute as pc
from open_fdd.arrow_runtime.windows import arrow_rolling_max,
arrow_rolling_min

VALUE_COLUMN = "oa-t"
WINDOW_SAMPLES = 12
FLATLINE_TOLERANCE = 0.1

def apply_faults_arrow(table, cfg, context=None):
    vals = pc.cast(table[VALUE_COLUMN], "float64")
    spread = pc.subtract(arrow_rolling_max(vals, WINDOW_SAMPLES),
arrow_rolling_min(vals, WINDOW_SAMPLES))
    return pc.less_equal(pc.abs(spread), FLATLINE_TOLERANCE)
```

34.5 Sensor out of range (catalog defaults)

```
from open_fdd.arrow_runtime.cookbook import sensor_bounds_mask

def apply_faults_arrow(table, cfg, context=None):
    return sensor_bounds_mask(table, "outdoor_air_temp", cfg) #
fault_code: BLD-B
```

34.6 Sensor flatline + spike (catalog)

```
from open_fdd.arrow_runtime.cookbook import sensor_flatline_mask,
rate_of_change_mask
from open_fdd.arrow_runtime.sensor_catalog import cfg_from_profile
```

```
def apply_faults_arrow(table, cfg, context=None):
    merged = cfg_from_profile("zone_temp", cfg)
    merged["samples_per_hour"] = 12
    flat = sensor_flatline_mask(table, "zone_temp", cfg)
    spike = rate_of_change_mask(table, merged, col="stat_zn-t")
    return pc.or_(flat, spike) # import pyarrow.compute as pc
```

Bounds table: [Expression cookbook — sensor validation](#).

More templates ship in `open_fdd.playground.arrow_templates` and via
GET `/api/playground/arrow-templates`.

Bench examples: `workspace/data/rules_py/bench_*.py`.

Dev kit zip layout: [Rule Lab — dev kit zip](#).

35 Python recipes (Arrow)

36 Python recipes (Arrow)

All Rule Lab Python rules use `apply_faults_arrow(table, cfg, context)` on PyArrow tables with `pyarrow.compute` and **module constants**.

See [Arrow recipes](#) for the canonical cookbook (flatline, spread, OOB, schedule faults).

Shared helpers live in `open_fdd.arrow_runtime.cookbook` and `open_fdd.arrow_runtime.windows`. For console validation, copy `_kit_lookback_stats` from [Lookback window helper](#).

```
from open_fdd.arrow_runtime.windows import arrow_rolling_min,
arrow_rolling_max
import pyarrow.compute as pc

VALUE_COLUMN = "oa-t"
WINDOW_SAMPLES = 12
FLATLINE_TOLERANCE = 0.1

def apply_faults_arrow(table, cfg, context=None):
    vals = pc.cast(table[VALUE_COLUMN], "float64")
    spread = pc.subtract(arrow_rolling_max(vals, WINDOW_SAMPLES),
arrow_rolling_min(vals, WINDOW_SAMPLES))
    return pc.less_equal(pc.abs(spread), FLATLINE_TOLERANCE)
```

37 Containers

38 Containers

Open-FDD v3 edge sites run **three** GHCR containers plus optional **host** services (Caddy, Ollama). There is no separate BACnet poll container — **field BACnet polling lives in commission**.

Quick answer: Who binds BACnet UDP 47808? **openfdd-commission**. Bridge only ingests samples.csv into the feather historian.

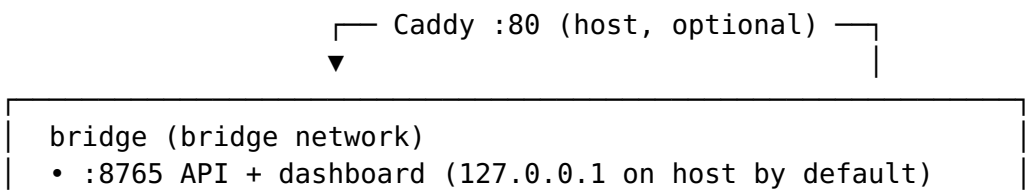
38.1 GHCR images

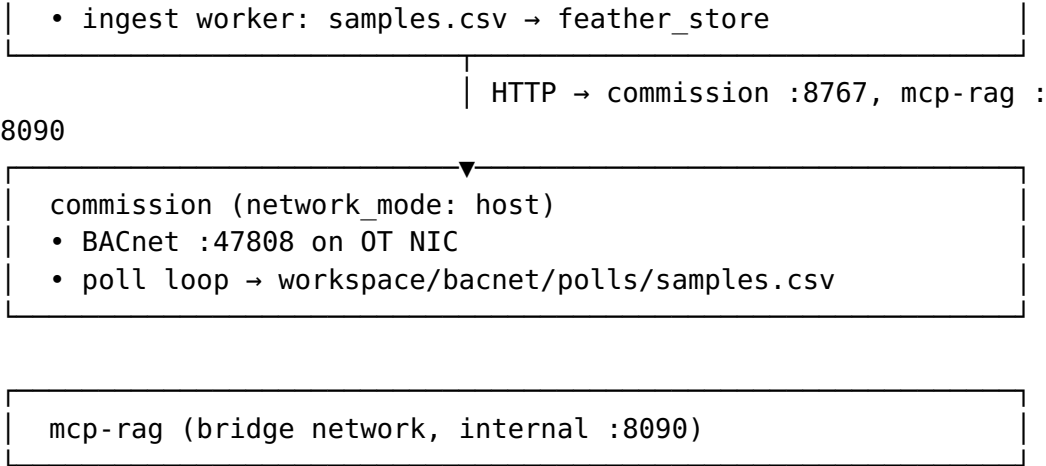
Image	Compose service	Role
ghcr.io/bbartling/openfdd-bridge	bridge	Operator API, React dashboard, feather ingest
ghcr.io/bbartling/openfdd-commission	commission	BACnet discover / read / write + poll loop
ghcr.io/bbartling/openfdd-mcp-rag	mcp-rag	Doc-search sidecar for agent tools

Tag: **latest** on [GitHub Packages](#). Retired: openfdd-bacnet-poll (poll runs inside **commission**). GHCR retains only the **three newest** versions per image.

Template: docker/compose.edge.yml → copy to ~/open-fdd/docker-compose.yml.

38.2 Stack diagram





38.3 Responsibilities

Concern	Container
BACnet Who-Is / read / write	commission
BACnet scheduled RPM reads	commission (bacnet_poll_loop)
CSV → feather historian	bridge (feather ingest worker — bacnet_poll_worker module name is historical)
Operator login, Rule Lab, faults UI	bridge
Agent doc search	mcp-rag

The bridge thread named bacnet_poll_worker is **ingest only** — it does not bind BACnet.

38.4 Network and ports (typical edge)

Endpoint	Reachable from	Notes
Caddy :80	Building LAN	Reverse proxy to bridge
Bridge :8765	Loopback (default)	Bind 127.0.0.1 in compose
Commission : 8767	Bridge / loopback	HTTP API for BACnet ops
Commission : 47808	OT BACnet LAN	UDP; requires

Endpoint	Reachable from	Notes
MCP :8090	Bridge container network	network_mode: host Not exposed to LAN by default

38.5 Long-lived deployment

All compose services set restart: unless-stopped. After host reboot:

```
sudo systemctl enable docker
cd ~/open-fdd && docker compose ps
```

If containers are down: docker compose up -d (idempotent).

See [Run with Docker images](#).

38.6 Persistent data

State lives on the **host** under workspace/ (bind mount). Containers are replaceable; **backup workspace/** before image upgrades.

Path	Contents
workspace/data/feather_store/	Historian
workspace/data/*.json	Model, rules, FDD results
workspace/bacnet/commissioning/	commission.env, points.csv
workspace/bacnet/polls/samples.csv	Poll output
workspace/auth.env.local	Login secrets
workspace/api/static/app/	Dashboard bundle (if updated separately)

38.7 Optional host services

Service	Role
Caddy	TLS or plain HTTP :80 → 127.0.0.1:8765
Ollama	

Service	Role
	Optional LLM on :11434 (not in the three-image stack)
FDD loop timer	Optional openfdd-fdd-loop.timer on host (docker exec batch)

39 Lookback window helper

40 Lookback window helper

Rules that depend on **time history** (flatline, spread, streaks, trim/respond validation) should print a clear window summary in the Rule Lab console or local `run_test.py` output.

40.1 Recommended pattern (copy into `rule.py`)

Use module constants for the lookback you intend; the bridge passes the full feather window into `apply_faults_arrow` based on batch `lookback_hours` (default **1 h** on the scheduled loop; **24 h** when using **Update all records** with chunks).

```
import pyarrow.compute as pc

LOOKBACK_HOURS = 3 # tune: 1, 3, 6, 12, 24

def _kit_lookback_stats(table, *, hours=None):
    """Dev helper – prints row count, timestamp span, and
    configured lookback."""
    h = hours if hours is not None else LOOKBACK_HOURS
    if "timestamp" not in table.column_names:
        print(f"lookback={h}h rows={table.num_rows} (no timestamp
column)")
        return
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin = pc.min(ts).as_py()
    tmax = pc.max(ts).as_py()
    if tmin is None or tmax is None:
        print(f"lookback={h}h rows={table.num_rows} start=None
stop=None span=0.00h")
        return
```



```

span_h = (tmax - tmin).total_seconds() / 3600.0
print(
    f"lookback={h}h rows={table.num_rows} "
    f"start={tmin.isoformat()} stop={tmax.isoformat()}"
    span={span_h:.2f}h"
)

def _kit_value_stats(table, column):
    vals = pc.cast(table[column], "float64")
    print(
        f"column={column} min={pc.min(vals).as_py():.2f} "
        f"max={pc.max(vals).as_py():.2f}"
        mean={pc.mean(vals).as_py():.2f}"
    )

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    _kit_value_stats(table, "oa-t")
    # ... fault logic ...

```

40.2 Why not hide this behind a library call yet?

A future `open_fdd.arrow_runtime.kit.lookback_stats(table, hours=3)` helper is reasonable once the Algorithms tab shares the same kit export path. For now, keeping the helper **inline in rule.py** matches the constants-first Rule Lab style and prints the same fields in:

- Local `run_test.py`
- Rule Lab **Quick test** console
- Batch run logs

40.3 Lookback vs rolling samples

Concept	Controlled by	Typical use
Historian lookback	Batch <code>lookback_hours</code> , FDD loop env	How many rows are in table
Rolling window	<code>WINDOW_SAMPLES</code> constant ($\sim 12 \approx 1 \text{ h}$ @ 5 min poll)	Flatline, spread, consecutive-true

Always call `_kit_lookback_stats(table)` first so operators can confirm the table span matches expectations before interpreting rolling-window faults.

40.4 Batch intervals

Trigger	Default lookback
openfdd-fdd-loop / OFDD_FDD_INTERVAL_MINUTES	1 h
Rule Lab Update all records	24 h (chunk_hours: 6, use_chunks: true)
Rule Lab Quick test	3 h (UI default)

See [Windowing & debugging](#) for `arrow_rolling_min / arrow_consecutive_true`.

41 Data flow

42 Data flow

42.1 Telemetry ingest

Open-FDD supports three commissioning drivers. Each appends long-format samples to workspace CSVs and writes **feather shards** tagged by source (bacnet, modbus, json_api).

42.1.1 BACnet

1. **Commission poll loop** (inside the commission container) reads enabled BACnet points from `points.csv` on a schedule (host network, BACnet :47808).
2. RPM results append to `workspace/bacnet/polls/samples.csv`.
3. **Bridge ingest worker** loads new rows into `feather_store/bacnet/`.

BACnet devices → commission (poll loop) → `samples.csv` → bridge (ingest) → `feather_store/bacnet`

42.1.2 Modbus TCP

1. Operator configures registers on the **Modbus** tab (or `read_and_store` API).
2. Poll worker reads holding/input registers via Modbus TCP.
3. Samples append to `workspace/modbus/polls/samples.csv` → `feather_store/modbus/`.

42.1.3 JSON API (HTTP/HTTPS)

1. Operator configures REST endpoints on the **JSON API** tab — GET/POST, Bearer or Basic auth, `${ENV:VAR}` placeholders from `json_api.env.local`, optional TLS verify off for self-signed OT gateways.
2. Poll worker issues HTTP requests (one GET can fan out to multiple JSON paths, e.g. OpenWeather `web-oat-t / web-rh / web-weather-desc`).
3. Samples append to `workspace/json_api/polls/samples.csv` → `feather_store/json_api/`.
4. FDD batch **merges** `json_api` columns with BACnet/Modbus on the same site (nearest timestamp, 30 min tolerance) for cross-source rules.

OT REST / weather API → bridge JSON API worker → `samples.csv` → `feather_store/json_api`

↘ merged

into FDD frame with bacnet/modbus

Showcase: [OpenWeatherMap bundle](#). Details: [Driver framework](#), [BACnet polling](#), [JSON API driver](#), [Containers](#).

42.2 Rule evaluation

1. Rules are Python files in `workspace/data/rules_py/` with metadata in `rules_store.json`.
2. **Bindings** map rules to points via the Brick model (`fdd_input` tags).
3. Batch runner (`POST /api/rules/batch` or host timer) produces `fdd_results.json`.
4. Dashboard **faults** view and check-engine status read aggregated results.

42.3 Model sync

- Commissioning imports `update model.json` / TTL graph.
- `POST /api/model/sync-ttl` refreshes SPARQL-backed tree used by the UI.

42.4 Retention

Feather files grow on disk; retention is configurable (`workspace/data.env.local`). Image upgrades must **preserve** `workspace/data/` — backup before `docker compose up -d --force-recreate`. See [Updating the stack](#).

43 Architecture

44 Architecture

Open-FDD v3 edge: **three Docker containers** (bridge, commission, mcp-rag), BACnet polling in **commission**, feather ingest in **bridge**, optional host Caddy and Ollama.

Page	Topic
System overview	Components and diagram
Deployment modes	Local dev, lab LAN, edge production, Caddy HTTP/TLS
Data flow	BACnet / Modbus / JSON API → feather → FDD → dashboard
Containers	GHCR images, ports, persistence, retired poll image
Driver framework	Shared commissioning pattern for OT sources

Operators: [Quick Start](#) — [Docker](#) — no git clone on the edge host.

BACnet polling: [Polling](#) — commission container only.

45 Rule Cookbook

46 Rule Cookbook

Practical patterns for **Arrow-native Rule Lab** (apply_faults_arrow) on feather historian PyArrow tables — **no pandas on the IoT edge**.

Page	Use when
Python recipes (full Arrow library)	Copy-paste GL36 A-M, VAV, plant, economizer, weather — replaces legacy YAML cookbook
Expression cookbook (Arrow-native)	Sensor bounds, legacy→Arrow map, commissioning checklist
Arrow recipes	

Page	Use when
	Short templates — threshold, flatline, OOB, fan/schedule
Python recipes	Shared <code>open_fdd.arrow_runtime.cookbook</code> imports
Lookback window helper	<code>_kit_lookback_stats</code> — print start/stop timestamps and span
Windowing & debugging	Rolling windows, batch runtime, Rule Lab console
GL36 algorithm stubs	Doc-only supervisory patterns (trim & respond, plant resets)

Programmatic sensor defaults:

`open_fdd.arrow_runtime.sensor_catalog.SENSOR_PROFILES.`

```
import pyarrow.compute as pc
from open_fdd.arrow_runtime.cookbook import sensor_bounds_mask

def apply_faults_arrow(table, cfg, context=None):
    return sensor_bounds_mask(table, "zone_temp", cfg) #
fault_code: VAV-C
```

Future graph ML (sklearn / PyG) runs in a separate training service — see [GitHub issue #211](#).

47 Windowing & debugging

48 Windowing & debugging

48.1 Lookback vs rolling window

	Historian lookback	Rolling window
Set by	Batch <code>lookback_hours</code> , FDD loop, Quick test UI	<code>WINDOW_SAMPLES</code> constant in <code>rule.py</code>
Typical	1 h scheduled; 24 h Update all	12 samples $\approx$ 1 h @ 5 min poll
Validate with	<code>_kit_lookback_stats(table)</code> — see Lookback window helper	Rule logic + console flagged count

48.2 Rolling windows (Arrow)

Use `open_fdd.arrow_runtime.windows` for sample-based rolling min/max and consecutive-true streaks:

```
from open_fdd.arrow_runtime.windows import arrow_rolling_min,
arrow_rolling_max, arrow_consecutive_true
```

Cookbook masks (`flatline_1h_mask`, `spread_1h_mask`, `oob_mask`) default to ~12 samples (~1 h at 5 min poll).

Bench rules use constants: `WINDOW_SAMPLES`, `FLATLINE_TOLERANCE`, `OAT_LOW` / `OAT_HIGH`, etc. Legacy `cfg` keys still work for uploaded rules that read `cfg.get(...)`.

48.3 Rule Lab console

Quick-test and batch responses include backend: `arrow`, `ms`, and flagged row counts. Arrow summary events appear in the event console on fault hits.

49 GL36 algorithm stubs

50 GL36 algorithm stubs (doc-only)

Draft **supervisory** PyArrow patterns for the future Algorithms tab. These mirror Niagara GL36 blocks documented in:

README_TRIM_RESPOND.md

FDD rules return **fault masks**. Algorithms return **setpoints or request integers** — the stubs below use `apply_algorithm_arrow` as the planned entryptpoint. For early testing in Rule Lab, you can adapt the logic into advisory FDD rules (boolean “sequence unhealthy” flags).

50.1 Shared lookback helper

```
import pyarrow.compute as pc
```

```
LOOKBACK_HOURS = 6
```

```

def _kit_lookback_stats(table, *, hours=None):
    h = hours if hours is not None else LOOKBACK_HOURS
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin = pc.min(ts).as_py()
    tmax = pc.max(ts).as_py()
    span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
    print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")

```

50.2 VAV zone cooling requests (0-3)

Simplified from GL36 VAV box request generator — zone temp vs cooling setpoint bands.

```

"""GL36 VAV cooling requests – doc stub."""

import pyarrow.compute as pc

ZONE_TEMP = "zn-t"
ZONE_SP = "zn-t-sp"
ZONE_DEMAND = "zn-cool-loop"
HIGH_DIFF_F = 5.0
MED_DIFF_F = 3.0
LOOP_ON = 95.0
LOOP_OFF = 85.0

def apply_algorithm_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    tz = pc.cast(table[ZONE_TEMP], "float64")
    sp = pc.cast(table[ZONE_SP], "float64")
    diff = pc.subtract(tz, sp)
    demand = pc.cast(table[ZONE_DEMAND], "float64")
    # Last row request level for kit demo (full port needs timer
state)

    last_diff = diff[-1].as_py() if table.num_rows else 0.0
    last_demand = demand[-1].as_py() if table.num_rows else 0.0
    if last_diff >= HIGH_DIFF_F:
        req = 3
    elif last_diff >= MED_DIFF_F:
        req = 2
    elif last_demand > LOOP_ON:
        req = 1
    else:
        req = 0
    print(f"cooling_requests={req} dT={last_diff:.1f}F
loop={last_demand:.0f}%")
    return {"cooling_requests": req}

```

50.3 AHU duct static trim & respond (advisory fault)

Detect **duct static stuck high** while VAV pressure requests are low — trim & respond may not be trimming.

```
"""AHU duct static trim advisory – doc stub."""

import pyarrow.compute as pc

DUCT_SP = "duct-static"
DUCT_SP_MAX = 1.50
DUCT_SP_TRIM_TARGET = 0.80
VAV_PRESS_REQ_SUM = "vav-pressure-req-sum"
LOOKBACK_HOURS = 12

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table, hours=LOOKBACK_HOURS)
    sp = pc.cast(table[DUCT_SP], "float64")
    reqs = pc.cast(table[VAV_PRESS_REQ_SUM], "float64")
    stuck_high = pc.and_(pc.greater(sp, DUCT_SP_TRIM_TARGET),
pc.less(reqs, 1.0))
    return pc.and_(stuck_high, pc.greater(sp, DUCT_SP_MAX * 0.9))
```

50.4 Chiller plant enable (valve request counting)

From GL36 §5.18.15.2 — count AHUs with CHW valve above threshold.

```
"""Chiller plant enable advisory – doc stub."""

import pyarrow.compute as pc

AHU_VALVE_COLS = ["ahu1-clg-vlv", "ahu2-clg-vlv", "ahu3-clg-vlv"]
REQ_THRESH = 95.0
DIS_THRESH = 10.0
MIN_AHUS_REQ = 2

def apply_algorithm_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    requesting = 0
    for col in AHU_VALVE_COLS:
        if col not in table.column_names:
            continue
        v = pc.cast(table[col], "float64")[-1].as_py()
        if v >= REQ_THRESH:
```



```

        requesting += 1
        enable = requesting >= MIN_AHUS_REQ
        print(f"chiller_enable={enable} requesting_ahus={requesting}/
{MIN_AHUS_REQ}")
        return {"chiller_enable": enable, "requesting_ahus":
requesting}

```

50.5 Hot water plant HWST trim & respond (advisory)

Flags **HWST trimmed too low** while heating requests remain high.

```

"""HWST trim advisory – doc stub."""

import pyarrow.compute as pc

HWST = "hw-supply-t"
HWST_SP = "hw-supply-t-sp"
HEAT_REQ_SUM = "hw-reset-req-sum"
LOW_HWST_F = 120.0
LOOKBACK_HOURS = 6

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table, hours=LOOKBACK_HOURS)
    temp = pc.cast(table[HWST], "float64")
    reqs = pc.cast(table[HEAT_REQ_SUM], "float64")
    return pc.and_(pc.less(temp, LOW_HWST_F), pc.greater(reqs,
2.0))

```

50.6 Chilled water plant (DP + CHWST reset advisory)

Flags **CHWST at design cold** with low cooling requests — possible stuck 100% plant loop.

```

"""CHW plant reset advisory – doc stub."""

import pyarrow.compute as pc

CHWST = "chw-supply-t"
CHWST_DESIGN_COLD_F = 44.0
COOL_REQ_SUM = "chw-reset-req-sum"
LOOKBACK_HOURS = 6

def apply_faults_arrow(table, cfg, context=None):

```

```

    _kit_lookback_stats(table, hours=LOOKBACK_HOURS)
    temp = pc.cast(table[CHWST], "float64")
    reqs = pc.cast(table[COOL_REQ_SUM], "float64")
    return pc.and_(pc.less(temp, CHWST_DESIGN_COLD_F + 1.0),
pc.less(reqs, 1.0))

```

50.7 Testing later

1. Bind historian columns in **Model & assignments** (fdd_input / brick types).
2. Copy a stub into Rule Lab as an FDD advisory rule, or wait for the **Algorithms** tab kit export.
3. Run python run_test.py from a downloaded kit and confirm _kit_lookback_stats prints expected start / stop / span.

Reference implementation (Niagara Java):
[README_TRIM_RESPOND.md](#).

51 Central plants (CHW / CTW / BLR)

52 Central plant Arrow recipes

Chiller plant, condenser water, cooling tower, boiler, and hot-water distribution faults use the same Arrow-native primitives as air-side rules.

52.1 Starter fault codes

Code	Primitive	Recipe
CHW-DT-001	low_delta_t_mask	Low chilled-water ΔT syndrome
CHW-DP-001	reset_stuck_mask	CHW differential pressure reset stuck high

52.2 Required point roles (CHW)

- chw_supply_temp, chw_return_temp — ΔT rules
- chw_dp, chw_dp_setpoint — reset stuck rules
- pump_command, pump_feedback — command/feedback mismatch

52.3 Synthetic test pattern

```
import pyarrow as pa
from open_fdd.arrow_runtime.primitives import low_delta_t_mask

table = pa.table({
    "chw_supply_temp": [44.0] * 10,
    "chw_return_temp": [44.5] * 10,
})
mask = low_delta_t_mask(
    table,
    {"min_delta_t": 4.0},
    supply_col="chw_supply_temp",
    return_col="chw_return_temp",
)
assert mask.to_pylist()[-1] is True
```

See [Fault codes — chiller plant](#) (expanded in 3.0.19).

53 Dashboard overview

54 Dashboard overview

54.1 Primary areas

Area	Purpose
Home / Building insight	Check-engine status (green/yellow/red), active fault summary
Trends	Point history from feather store
Faults	Rule hits grouped by equipment family
Rule Lab	Edit, lint, and test Python FDD rules (PyArrow kit zip)
Algorithms	Supervisory GL36 trim & respond (coming soon)

Area	Purpose
Model & assignments	Equipment tree, FDD rule pins, commissioning JSON
BACnet	Discovery, reads, commission workflows
Settings / health	Stack status, auth mode

54.2 Live updates

Dashboard tiles use REST polling and WS `/ws/dashboard` for selected live views (ticket auth via POST `/api/auth/ws-ticket`).

54.3 Roles

Read-only operators see trends and faults; integrators access Rule Lab and model import. See [Auth and roles](#).

55 Auth and roles

56 Auth and roles

56.1 Production (LAN / edge)

- Set `OFDD_AUTH_SECRET` and role passwords in `workspace/auth.env.local`.
- Bridge on non-loopback addresses **requires** auth unless explicit insecure dev flags are set (lab only).

Role	Typical access
viewer	Read trends, faults, model tree
operator	Run batch FDD, acknowledge views
integrator	Rule Lab, bindings, model import
commission	BACnet writes (plus write guard)
agent	Agent tools / app-edit gates
admin	Full configuration

Login: POST `/api/auth/login` → Bearer JWT on API calls.

56.2 Localhost dev

OFDD_BRIDGE_HOST=127.0.0.1 with OFDD_AUTH_DISABLED=1 is acceptable on a single machine. **Never** use auth-disabled mode on a building LAN.

Full hardening: [Security](#).

57 Rule Lab

58 Rule Lab

Rule Lab authors **Arrow-native** Python FDD rules against the feather historian. Bench rules use **module constants** at the top of rule.py — no browser config panel, no config.json in the dev kit zip.

```
"""Bench OA-T out of bounds (Arrow)."""

import pyarrow.compute as pc

VALUE_COLUMN = "oa-t"
OAT_LOW = 68.0
OAT_HIGH = 88.0
LOOKBACK_HOURS = 1

def _kit_lookback_stats(table, *, hours=None):
    h = hours if hours is not None else LOOKBACK_HOURS
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin = pc.min(ts).as_py()
    tmax = pc.max(ts).as_py()
    span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
    print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")

def _kit_value_stats(table):
    vals = pc.cast(table[VALUE_COLUMN], "float64")
    print(
        f"column={VALUE_COLUMN} min={pc.min(vals).as_py():.2f} "
        f"max={pc.max(vals).as_py():.2f}
mean={pc.mean(vals).as_py():.2f}"
    )
```

```
def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    _kit_value_stats(table)
    vals = pc.cast(table[VALUE_COLUMN], "float64")
    return pc.or_(pc.less(vals, OAT_LOW), pc.greater(vals,
OAT_HIGH))
```

58.1 Workflow

1. **Download kit** — PyArrow zip (see below)
2. **Edit constants** locally (VALUE_COLUMN, limits, LOOKBACK_HOURS, WINDOW_SAMPLES)
3. **Run** `pip install -r requirements.txt` then `python run_test.py`
4. **Upload** rule.py on Rule Lab (integrator)

Upload validation (Phase B): AST parse, forbidden imports, `apply_faults_arrow(table, cfg, context=None)` signature.

The bridge runs rules on **PyArrow tables** via `open_fdd.arrow_runtime`, and persists .py sources under `workspace/data/rules_py/`.

- **Quick test** — `POST /api/playground/test-rule` (default 3 h lookback)
- **Batch** — `POST /api/rules/batch / openfdd-fdd-loop timer (1 h default lookback)`
- **Update all records** — 24 h lookback, 6 h chunks (`use_chunks: true`)
- **Templates** — `GET /api/playground/arrow-templates`

Pin points to rules via **Model & assignments** (`/model`) commissioning JSON or BACnet tree right-click. Export shows Rule Lab **names** on each point (`fdd_rules_linked`); import uses rule **ids** (`fdd_rule_ids`).

Bench seed: `python3 scripts/setup_bench_afdd.py` — imports `bench_import_model.json` and bench cookbook rules with matching id/name pairs.

58.2 Dev kit zip (download)

Download kit on Rule Lab exports `openfdd-rule-kit-<rule_id>.zip` with:

File	Purpose
rule.py	Arrow rule + constants at top; optional <code>_kit_lookback_stats / _kit_value_stats</code> helpers

File	Purpose
data.py	SITE_ID, LOOKBACK_HOURS, load_table() reading sample.feather
sample.feather	Full historian window for the kit lookback (all rows, not a CSV preview)
run_test.py	Runs rule.apply_faults_arrow(table, {}, context=...) and prints flagged=N
requirements.txt	open-fdd>=3.0.1 and pyarrow
README.md	Install and run instructions
column_map.json	BRICK logical keys → feather column names
kit_meta.json	Export metadata (site, columns, row count, lookback hours)

Not included: config.json (retired — tune thresholds as Python constants).

Local test:

```
unzip openfdd-rule-kit-*.zip
cd openfdd-rule-kit-*
pip install -r requirements.txt
python run_test.py
```

Expected console output:

```
site=demo lookback_h=3 rows=...
lookback=3h rows=... start=... stop=... span=2.98h
column=oa-t min=... max=... mean=...
flagged=...
```

58.3 Lookback windows

Rules that need multi-hour history should call `_kit_lookback_stats(table)` (see [Lookback window helper](#)). Pass `hours=1, 3, 6, 12, or 24` to validate timestamp span in the console.

Rolling-window rules (flatline, spread) use `WINDOW_SAMPLES` (~12 samples $\approx$ 1 h at 5 min poll) — see [Windowing & debugging](#).

58.4 Algorithms (supervisory — coming soon)

GL36 trim & respond and plant reset sequences will live on the **Algorithms** tab with the same zip kit shape but `apply_algorithm_arrow` outputs. Reference: [README_TRIM_RESPOND](#).

58.5 Related

- [Model workflow](#)
- [Arrow recipes](#)
- [GL36 algorithm stubs](#) (doc-only)

59 JSON API driver

60 JSON API driver

Poll **HTTP or HTTPS** REST endpoints on the OT LAN (or public APIs such as [OpenWeatherMap](#)) and store extracted JSON values in the feather historian (`source=json_api`).

Use this driver for gateways, micro-PLCs, IoT hubs, vendor APIs, and **web weather** when you want to cross-check shaky local outdoor-air sensors against a reference.

60.1 Supported features

Feature	Support
Methods	GET, POST
Transport	http:// and https://
TLS certificate verify	On by default; disable for self-signed OT gateways
Authentication	None, Bearer token , HTTP Basic , or query-string API keys via env
Env placeholders	<code>\${ENV:VAR}</code> or <code>\${VAR}</code> in URL, headers, body, bearer token
Custom headers	Optional headers map (merged with auth headers)

Feature	Support
Value extraction	Dot-path into JSON body (title, main.temp, weather. 0.description)
Multi-extract poll	One HTTP request per URL group; fan-out to multiple json_path rows
Polling	1 / 5 / 15 / 30 min or 1 hour (shared with BACnet/Modbus)
Historian	workspace/json_api/polls/ samples.csv → feather_store/ json_api/
FDD merge	Scheduled FDD merges bacnet + modbus + json_api columns by nearest timestamp

60.2 Secrets: json_api.env.local

Copy workspace/json_api.env.example → workspace/
json_api.env.local (gitignored). The bridge loads this file at
startup and when run_local.sh starts the stack.

Use placeholders in commissioning URLs so API keys never land
in git:

```
https://api.openweathermap.org/data/2.5/weather?q=$
{ENV:OPENWEATHER_CITY}&appid=${ENV:OPENWEATHER_API_KEY}&units=$
{ENV:OPENWEATHER_UNITS}
```

Check which vars are configured: GET /api/json-api/env/status.

60.3 OpenWeatherMap showcase (recommended demo)

This is the flagship example of what the generic JSON API driver
can do — **one URL, three historian columns**, env-based API
key, browser tree like BACnet/Modbus.

60.3.1 1. Configure env

```
cp workspace/json_api.env.example workspace/json_api.env.local
# Edit: OPENWEATHER_API_KEY, OPENWEATHER_CITY, OPENWEATHER_UNITS
(imperial = °F)
```

Restart the bridge (or `./scripts/run_local.sh restart`) so env vars load.

60.3.2 2. Register from the UI

1. Sign in as **integrator** → **JSON API** tab.
2. Open the **OpenWeatherMap showcase** panel.
3. Use the **OpenWeatherMap** preset → **Register** (default poll 30 min; choose interval in the form).

Three endpoints appear under `api.openweathermap.org` in the commissioning tree:

Label	JSON path	Historian column	Units
web-oat-t	main.temp	web-oat-t	degF / degC
web-rh	main.humidity	web-rh	%
web-weather-desc	weather.0.description	web-weather-desc	text

Poll worker deduplicates: **one GET** serves all three extractions per cycle.

60.3.3 3. Headless / script

```
# Standalone fetch (like the original playground tester)
python3 scripts/openweather_json_api_example.py

# Register via REST (integrator login)
python3 scripts/openweather_json_api_example.py --register --base
http://127.0.0.1:8000
```

60.3.4 4. FDD: local OA-T vs web weather

With BACnet `oa-t` and JSON API `web-oat-t` in the same site, scheduled FDD merges both sources. Example rule:

```
workspace/data/rules_py/oat_vs_web_spread_1h.py — flags when |oa-t - web-oat-t| > 8 °F (stuck or drifting outdoor-air sensor).
```

Enable it in Rule Lab, bind to your OA-T BACnet point, and run **Update all records**.

60.4 Commissioning (UI)

The **JSON API** tab follows `Home Assistant sensor.rest`:

1. Sign in → **RESTful sensor (JSON API)** tab.
2. Pick a **preset** (JSONPlaceholder, DummyJSON, httpbin, Open-Meteo, OpenAQ, OpenWeather) or enter a **resource URL**.
3. Add one or more **sensors** — each row is a `value_json_path` + historian **label** (multi-value from one GET, like HA `json_attributes`).
4. Click **Test resource** — probes without saving; shows extracted values and raw JSON.
5. Click **Register & poll** — writes `endpoints.csv`, enables polling at the selected interval.
6. Poll choices match BACnet/Modbus: **1 / 5 / 15 / 30 min** or **1 hour**.

60.4.1 API for agents

Endpoint	Purpose
GET /api/json-api/presets	Full preset catalog + poll intervals (human + AI readable)
POST /api/json-api/test	Probe resource + multi-sensor extraction
POST /api/json-api/register-bundle	Register one resource with many sensors
POST /api/json-api/presets/{id}/register	One-click preset registration

60.4.2 Bench preset

Use **JSONPlaceholder** — **todo title** → **Test resource** to verify outbound HTTPS without field hardware.

60.5 Authentication examples

60.5.1 Query-string API key via env (OpenWeather)

```
{
  "url": "https://api.openweathermap.org/data/2.5/weather?q=${ENV:OPENWEATHER_CITY}&appid=${ENV:OPENWEATHER_API_KEY}&units=${ENV:OPENWEATHER_UNITS}",
  "method": "GET",
  "json_path": "main.temp",
  "label": "web-oat-t",
```

```

    "auth_type": "none"
  }

```

60.5.2 Bearer token (API key)

```

{
  "url": "https://192.168.10.50/api/v1/status",
  "method": "GET",
  "json_path": "sensors.zone_temp",
  "label": "zone-temp",
  "auth_type": "bearer",
  "bearer_token": "${ENV:GATEWAY_API_KEY}",
  "verify_tls": false
}

```

60.5.3 HTTP Basic

```

{
  "url": "http://192.168.10.50/data.json",
  "method": "GET",
  "json_path": "value",
  "label": "sensor-value",
  "auth_type": "basic",
  "basic_user": "operator",
  "basic_password": "changeme"
}

```

60.5.4 POST with JSON body

```

{
  "url": "https://gateway.local/api/read",
  "method": "POST",
  "json_path": "result.pv",
  "label": "pv",
  "body": { "point": "AHU-1_SAT", "command": "read" },
  "auth_type": "bearer",
  "bearer_token": "..."
}

```

Credentials in endpoints.csv are redacted in API responses (***).
Prefer \${ENV:...} for production keys.

60.6 REST API

Method	Path	Role	Description
GET	/api/json-api/ env/status	operator+	Which env vars are configured (no secret values)

Method	Path	Role	Description
POST	/api/json-api/presets/openweather	integrator+	Register 3-point OpenWeather bundle + optional poll
GET	/api/json-api/driver/tree	operator+	Commissioning tree grouped by host
GET	/api/json-api/poll/status	operator+	Poll worker status
POST	/api/json-api/poll/once	integrator+	Run one poll cycle
POST	/api/json-api/request	operator+	One-shot HTTP request (no store)
POST	/api/json-api/read_and_store	integrator+	Request + CSV + feather ingest
POST	/api/json-api/refresh	operator+	Re-fetch one endpoint
PATCH	/api/json-api/endpoint/poll	integrator+	Enable/disable poll interval
DELETE	/api/json-api/endpoint/{point_id}	integrator+	Remove endpoint

Full route list: [API routes](#).

60.7 Typical OT URLs

Device type	Example URL	Notes
Web weather	https://api.openweathermap.org/data/2.5/weather?...	Env-based appid; 3 historian columns
Gateway status	http://192.168.1.50/api/status	Often plain HTTP on LAN
HTTPS BMS API	https://10.0.0.12/rest/points/zone1	May need verify_tls: false
Local mock	https://jsonplaceholder.typicode.com/todos/1	Bench / CI only

60.8 Limitations

- Response body must be **JSON** (not XML or plain text).
- Numeric paths (e.g. `main.temp`) are stored as strings in poll CSV; FDD rules cast with `pc.cast(..., "float64")`.
- Outbound requests originate from the **bridge** container/host — ensure routing and firewall rules allow the edge box to reach the target (OT subnet or public internet for weather).

60.9 Disable polling (save API quota)

Stop scheduled calls without deleting endpoints: JSON API tree → select OpenWeather points → **Stop polling**, or integrator API:

```
curl -X PATCH http://127.0.0.1:8765/api/json-api/endpoint/poll \
-H "Authorization: Bearer $TOKEN" \
-d '{"point_id":"ja-api-openweathermap-org-get-web-oat-
t","enabled":false,"poll_interval_s":0}'
```

API keys live in `json_api.env.local` (gitignored) — never committed; audit logs do not record expanded URLs with `appid=`. See [Logging and audit](#).

60.10 Smoke test

```
python3 scripts/smoke_multi_driver_stack.py
```

Validates JSON API ingest alongside BACnet and Modbus, feather isolation, BRICK scope, and FDD batch.

61 Algorithms

62 Algorithms (coming soon)

The **Algorithms** dashboard tab (`/algorithms`) is a placeholder for **supervisory Python sequences** — the mirror image of Rule Lab FDD:

	FDD (Rule Lab)	Algorithms (planned)
Purpose	Detect faults	Compute setpoints, request levels, plant enables
Entrypoint	<code>apply_faults_arrow(table, cfg, context)</code>	<code>apply_algorithm_arrow(table, cfg, context)</code> (planned)

	FDD (Rule Lab)	Algorithms (planned)
Output	Boolean fault mask	Numeric outputs / structured dict
Historian	Same feather_store PyArrow tables	Same

Until the tab ships, draft patterns live in this doc and in [GL36 algorithm stubs](#).

62.1 GL36 Trim & Respond reference

Open-FDD algorithms will follow the same ASHRAE Guideline 36 trim & respond methodology already implemented for Niagara in:

README_TRIM_RESPOND.md

That guide covers:

- VAV zone request generators (cooling + static pressure)
- AHU duct static pressure reset
- AHU supply air temperature reset
- Central plant chiller / boiler enable and HWST / CHWST trim & respond

Porting those blocks to PyArrow is in progress; the dashboard tab shows **COMING SOON** until upload, batch, and BACnet write paths are wired.

62.2 Planned workflow (same kit shape as Rule Lab)

1. Download **algorithm kit** zip from the Algorithms tab (future)
2. Edit **constants** at the top of algorithm.py (no config.json)
3. Run `python run_test.py` locally against `sample.feather`
4. Upload algorithm.py when satisfied

Kit files will match the Rule Lab zip layout (see [Rule Lab — dev kit zip](#)).

62.3 Lookback windows

Rules and algorithms that need multi-hour history should use the shared **lookback stats helper** documented in [Lookback window helper](#). Pass `hours=1, 3, 6, 12, or 24` to print row count, timestamp start/stop, and span for console validation.

Scheduled batch defaults: **1 h** lookback on the FDD loop; **Update all records** in Rule Lab uses **24 h** with 6 h chunks.

62.4 AHU supervisory checks (doc-only stubs)

These are **not** uploaded rules yet — copy-paste references for future algorithm or FDD work.

62.4.1 Duct static pressure too high

Flags sustained high duct static (possible stuck setpoint, blocked filter, or trim/respond not trimming).

```
"""AHU duct static high – doc stub (Arrow constants)."""

import pyarrow.compute as pc

VALUE_COLUMN = "duct-static"
HIGH_INWC = 1.20
LOOKBACK_HOURS = 3
MIN_SAMPLES_ABOVE = 6

def _kit_lookback_stats(table, *, hours=None):
    h = hours if hours is not None else LOOKBACK_HOURS
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin = pc.min(ts).as_py()
    tmax = pc.max(ts).as_py()
    span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
    print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    vals = pc.cast(table[VALUE_COLUMN], "float64")
    high = pc.greater(vals, HIGH_INWC)
    # Require several consecutive highs (~30 min at 5 min poll)
    from open_fdd.arrow_runtime.windows import
arrow_consecutive_true

    streak = arrow_consecutive_true(high, MIN_SAMPLES_ABOVE)
    return streak
```


62.4.2 Supply air temperature too cold (cooling coil over-delivering)

```
"""AHU SAT too cold vs setpoint – doc stub."""

import pyarrow.compute as pc

SAT_COLUMN = "sa-t"
SP_COLUMN = "sa-t-sp"
COLD_DELTA_F = 5.0
LOOKBACK_HOURS = 1

def _kit_lookback_stats(table, *, hours=None):
    h = hours if hours is not None else LOOKBACK_HOURS
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin = pc.min(ts).as_py()
    tmax = pc.max(ts).as_py()
    span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
    print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    sat = pc.cast(table[SAT_COLUMN], "float64")
    sp = pc.cast(table[SP_COLUMN], "float64")
    return pc.less(pc.subtract(sat, sp), -COLD_DELTA_F)
```

62.5 Plant supervisory checks (doc-only)

See [GL36 algorithm stubs](#) for chiller plant enable, HWST trim & respond, and CHW DP/CHWST reset patterns aligned with the Niagara README.

62.6 Related

- [Rule Lab](#) — live FDD authoring
- [Model workflow](#) — point bindings
- [Windowing & debugging](#) — rolling windows

63 Model workflow

64 Model workflow

64.1 Brick model

- **Equipment tree** — SPARQL over synced TTL (workspace/data/data_model.ttl)
- **Point registry** — BACnet inventory → series_id for historian bindings
- **Health score** — orphan points, missing fdd_input, stale telemetry

64.2 Typical integrator flow

1. Import or sync BACnet point registry.
2. Map equipment classes (AHU, VAV, ...) in the model UI — set brick_type on points and feeds on equipment (AHU→VAV).
3. Tag points with fdd_input roles (zone temp, SAT, damper cmd, ...).
4. Bind Rule Lab rules to points, equipment, or BRICK classes (bindings in rules_store.json).
5. Run POST /api/model/sync-ttl after bulk imports.
6. Validate on **Trend plot** with FDD overlays (?fdd=1) — faults render on a right-hand 0/1 axis per device scope.

AI Agent and commissioning JSON import can bulk-apply steps 2-4; integrator should review model health before production FDD. See [FDD and assignments](#) and [Ollama and analytics](#).

Export: GET /api/model/export (TTL/JSON). Commissioning bundle: GET /api/model/commissioning-export — includes per-point fdd_rules_linked (Rule Lab names) plus fdd_rules[] catalog. Use **Import / export** tab preview table or **Point → FDD rule pins** before editing raw JSON. API detail in [Appendix](#).

65 Driver framework

66 Driver framework

Open-FDD edge commissioning uses a **shared driver pattern** for OT data sources. Each driver polls field devices on a schedule, stores long-format samples in workspace CSVs, and ingests into the **feather historian** under a distinct source tag for FDD and plots.

66.1 Drivers

Driver	Protocol	Feather source	Commissioning tab
<u>BACnet</u>	MS/TP / IP (: 47808)	bacnet	BACnet
Modbus	Modbus TCP	modbus	Modbus
<u>JSON API</u>	HTTP/HTTPS REST (+ <u>OpenWeather showcase</u>)	json_api	JSON API

All three drivers share the same operator workflow:

1. **Discover or configure** — add devices/endpoints to the driver registry.
2. **Request once** — on-demand read to verify connectivity and JSON path / register map.
3. **Request & store** — append to poll CSV and write a feather shard.
4. **Enable polling** — 1 / 5 / 10 / 15 minute intervals via tree right-click or bulk toolbar.
5. **Poll all now** — force one worker cycle without waiting for the interval.

66.2 Data layout

```
workspace/
  bacnet/commissioning/  points.csv, discovered inventory
  bacnet/polls/         samples.csv
  modbus/commissioning/ registers.csv
  modbus/polls/         samples.csv
  json_api/commissioning/ endpoints.csv
  json_api/polls/      samples.csv
```

```
data/feather_store/  
  bacnet/{site_id}/      shard-*.feather  
  modbus/{site_id}/  
  json_api/{site_id}/
```

66.3 Historian and FDD

- **Plot tab** — numeric columns from bacnet and modbus sources (GET /api/timeseries/plot?source=...).
- **String JSON fields** — stored in feather under json_api; use the JSON API tree or feather directly (plot is numeric-only).
- **FDD batch** — merges bacnet + modbus + json_api historian columns by nearest timestamp; rules use BRICK-bound external_id names (e.g. compare BACnet oa-t vs JSON API web-oat-t — see [JSON API](#)).
- **BRICK model** — bind points in **Model & assignments** commissioning JSON; sync TTL for SPARQL scope.

66.4 Rule Lab (Arrow upload/download)

1. **Download kit** — zip with rule.py (constants at top), data.py, sample.feather (~3h), run_test.py, requirements.txt.
2. Local test: pip install -r requirements.txt then edit constants in rule.py and python run_test.py.
3. **Upload rule.py** — Arrow-only; must define apply_faults_arrow(table, cfg, context=None).
4. Browser shows **read-only** source; use Quick test + Update all records.

API: GET /api/rules/export-kit, POST /api/rules/upload.

66.5 Smoke validation

From repo root on a running stack (<http://127.0.0.1:8765>):

```
python3 scripts/smoke_multi_driver_stack.py  
python3 scripts/validate_modbus_temp_e2e.py --no-server
```

66.6 Related docs

- [Data flow](#) — poll → CSV → feather → FDD
- [API routes](#) — REST reference
- [BACnet polling](#) — commission container loop

67 Operator Bridge

68 Operator Bridge

The Operator Bridge is the FastAPI service and React dashboard operators use daily.

Page	Topic
Dashboard overview	Main UI areas
Auth and roles	Login and permissions
Rule Lab	Python FDD rule authoring (PyArrow kit zip)
Algorithms	Supervisory GL36 sequences (coming soon)
Model workflow	Brick bindings and imports
FDD and assignments	Rule bindings, BRICK classes, commissioning JSON
Ollama and analytics	Home briefing, zone levers, model RAM tiers

REST details: [Appendix — API routes](#). Env files: [Workspace env files](#).

69 Ollama and analytics

70 Ollama and analytics

70.1 What runs without login

The **Building status** home page (/) and GET /openfdd-agent/building-insight are **public** (same policy as fault status). Anyone on the LAN can read the briefing sentence and underlying numeric levers; only the **AI Agent** chat and stack revision endpoints require sign-in.

70.2 Ollama setup

1. Copy workspace/ollama.env.example → workspace/ollama.env.local.
2. Pick a model for available RAM (comments in the example file):

RAM tier	Suggested model
4 GB Pi	qwen3:0.6b
8 GB	qwen3:1.7b
16 GB	qwen3:4b
32 GB	qwen3:8b
64 GB	qwen3:14b

1. Set OFDD_OLLAMA_BASE_URL (default http://127.0.0.1:11434).
2. Optional: OFDD_OLLAMA_GPU_MODE (cpu / auto / gpu), OFDD_OLLAMA_TIMEOUT_S for slow CPUs.
3. Bootstrap helper: ./scripts/bootstrap_ollama.sh.

Bridge reads env via run_local.sh / compose. Model resolution: workspace/api/openfdd_bridge/ollama_profiles.py.

70.3 Automatic analytics (deterministic, always on)

These pandas calculations run on a refresh interval and feed the home dashboard **whether or not Ollama is up**:

Lever	Module	Refresh env	What it computes
Zone temperature snapshot	zone_temp_analytics.py	OFDD_ZONE_TEMP_INTERVAL_S (default 3600s)	Site-wide and per AHU zone temperature averages, spread out-of-band cooling using BRICK feedback (AHU→VAV) which is modeled; falls back to all Zone_Air_Temperature points
Device poll health	device_poll_health.py	stack health cycle	BACnet device liveness seen / stale poll detection
Operational brief	building_insight.py → get_operational_brief	same as insight	Compact numerical status for integrators
	root_cause_hints.py	on insight refresh	

Lever	Module	Refresh env	What it computes
Root-cause hints			Multi-zone fault chains using brick:feeds from

Shared lookback window: `OFDD_ANALYTICS_LOOKBACK_DAYS` (default 14) in `operational_analytics.py`.

API routes (auth optional unless noted):

- GET `/openfdd-agent/building-insight` — sentence + zone snapshot + fault sentences
- GET `/openfdd-agent/zone-temp-analytics`
- GET `/openfdd-agent/device-poll-health`
- GET `/openfdd-agent/ollama/health` — integrator diagnostics

70.4 When Ollama is used

`building_insight.get_building_insight()`:

1. Refreshes zone + device snapshots (pandas).
2. Calls `ollama_client.health()`.
3. If Ollama is reachable and `should_use_ollama_for_insight()` passes, sends a **compact context** (status, zone levers, feeds chains) to the configured model for a one-sentence briefing (source: "ollama").
4. On failure, uses `_fallback_sentence()` (source: "deterministic").

Context assembly: `building_insight._compact_context()`. Prompt template references BRICK feeds and `root_cause_hints`.

70.5 AI Agent (authenticated)

`/agent` page uses `POST /openfdd-agent/chat` with tool access (`agent_tools.py`): BRICK graph, scope bundles, rule binding patches, model health, etc. This is separate from the public home briefing.

70.6 Code map

```
workspace/api/openfdd_bridge/
  building_insight.py      # cached home briefing + Ollama
sentence
  zone_temp_analytics.py  # zone temp levers (BRICK feeds)
  device_poll_health.py   # BACnet poll staleness
  operational_analytics.py # lookback window + methodology dict
  root_cause_hints.py     # multi-fault BRICK chain hints
```

```
ollama_client.py      # HTTP client to Ollama
ollama_profiles.py    # RAM tiers / model names
```

Dashboard consumers: BuildingStatusPage.tsx,
StackStatusStrip.tsx (WebSocket /ws/dashboard).

71 FDD and assignments

72 FDD and assignments

72.1 What the product actually does

Open-FDD separates **data modeling** (BRICK equipment, points, feeds relationships) from **rule assignment** (which FDD rules apply to which points, equipment, or BRICK classes). Both are editable in **Model & assignments** and via commissioning JSON import.

72.1.1 BRICK modeling (integrator + AI-assisted)

Concern	Where it lives	Notes
Equipment tree	ModelService + TTL sync	AHU, VAV, sensors; equipment_type and brick_type on points
brick:feeds	model_feeds.py, ttl_service.py	AHU→VAV→zone relationships; used by zone analytics and root-cause hints
Point registry	BACnet inventory → model points	fdd_input roles (oa-t, stat_zn-t, ...), series_id / historian column
Commissioning import	POST /api/model/commissioning-import	Bulk JSON: sites, equipment, points, optional fdd_rules

AI Agent (`agent_tools.py`) can read `slim_brick_graph`, `scope_bundle`, patch rule bindings, and upsert points — but it does **not** auto-generate a full site model without operator review. Expected workflow:

1. BACnet discover/poll populates point registry.
2. Integrator or agent tags `brick_type`, `fdd_input`, and feeds on equipment.
3. `POST /api/model/sync-ttl` writes `workspace/data/data_model.ttl`.
4. Model health (`model_health.py`) flags orphan points and missing `fdd_input`.

72.1.2 Rule assignment (three binding kinds)

Saved rules in `rules_store.json` carry bindings:

```
{
  "point_ids": ["5007-analog-input-1173"],
  "equipment_ids": ["bench-1"],
  "brick_types": ["Outside_Air_Temperature_Sensor"]
}
```

Binding kind	Effect
point_ids	Rule runs for that historian point / series
equipment_ids	Rule runs for all points on equipment (mass bind)
brick_types	Rule runs for any point with that BRICK class on site

Assign via:

- **Model & assignments** UI — equipment/point chips, rule pin menu on trend plot
- **Rule Lab** — bindings panel
- **Commissioning import** — `fdd_rules[].bindings` and per-point `fdd_rule_ids`
- **AI Agent** — `patch_rule_binding` tool

Runtime evaluation: `fdd_runner.py` (batch FDD) and `plot_readings.evaluate_fault_plots()` (trend overlays). Plot scope filters rules to those bound to **selected telemetry** (point, equipment, or BRICK class); unbound rules still evaluate site-wide.

Brick bindings (3.0.20+): when a rule binds `brick_types` (or `equipment_ids` / `point_ids`), the batch runner resolves **every** matching historian column via `historian_columns_for_rule`, runs the Arrow rule per column with `value_column` set, and OR-combines fault masks. One rule config therefore applies consistently to all zone temps, discharge temps, etc.

72.1.3 CLASS vs device

- **BRICK class** (`brick_type` on points, or `brick_types` on rule bindings) — template-style assignment (“all `Zone_Air_Temperature_Sensor`”).
- **Device / equipment** (`equipment_id`, BACnet device instance) — instance assignment (“this VAV-12”).
- **Point** (`point_ids`) — single-sensor assignment.

`applies_to` on legacy rules is deprecated; use bindings in 3.0+.

72.2 Commissioning JSON shape

Export: GET `/api/model/commissioning-export`. Import accepts `import_ready_json` wrappers (see `commissioningImport.ts`).

72.2.1 Rule names vs ids (Rule Lab alignment)

Rule Lab shows **human names** (Bench OA-T flatline 1h). Saved rules use stable **ids** (`bench-oa-t-flatline-1h` from setup scripts, or a UUID for uploaded custom rules). Commissioning export includes both so you never have to cross-reference sections by hand:

Field	Direction	Purpose
<code>points[].fdd_rule_ids</code>	import + export	Assignment surface — array of rule ids
<code>points[].fdd_rules_linked</code>	export only	<code>[{id, name}]</code> — Rule Lab names beside each point
<code>fdd_rules[].id</code>	import + export	Stable rule key (matches Rule Lab / <code>rules_store.json</code>)
<code>fdd_rules[].name</code>	import + export	Same label as Rule Lab dropdown
<code>fdd_rules[].source_file</code>	export	Basename of deployed .py

Field	Direction	Purpose
		when present (e.g. bench_oa- t_flatline_1h.py)

On import, `fdd_rules_linked` is stripped (like `fdd_rule_ids` — not stored in `model.json`). Use the **Point → FDD rule pins** table on Model & assignments to read names without parsing JSON.

Minimum fields integrators/AI should produce:

- `sites, equipment[]` with `id, equipment_type, optional feeds[]`
- `points[]` with `site_id, equipment_id, brick_type, fdd_input, BACnet IDs, optional fdd_rule_ids`
- `fdd_rules[]` with `id, name, bindings, enabled (optional source_file on export)`

After import, verify in **Trend plot** with `?fdd=1` — fault lanes appear on the right-hand 0/1 axis for bound rules.

72.3 Related

- [Model workflow](#)
- [Rule Lab](#)
- [Trend plot / fault overlays](#)

73 Network setup

74 Network setup

74.1 Bind address

Commission and poll containers need the **OT interface** IP and UDP **47808**:

```
# workspace/bacnet/commissioning/commission.env
BACNET_BIND=192.168.1.50/24:47808
BACNET_INSTANCE=599999
```

- Use the NIC that can reach controllers (not only the management VLAN unless routed).
- Poll driver uses `network_mode: host` on Linux edges.

74.2 Discovery range

```
DISCOVER_LOW=1
DISCOVER_HIGH=4194302
DISCOVER_TIMEOUT=30
```

Narrow the range in large sites to avoid broadcast storms.

74.3 Verify

From commission container logs or API: Who-Is → I-Am responses. If zero devices, check firewall, bind IP, and VLAN routing before tuning rules.

75 Discover and read

76 Discover and read

The **commission** container exposes HTTP APIs for discovery and read-property workflows used by the dashboard BACnet tools.

Capability	Notes
Who-Is / I-Am	Device discovery on OT subnet
Read property	Present value, object name, units
RPM	Bulk reads for inventory export
Write property	Gated — see Write safety

Exported inventory feeds `points_discovered.csv` and the Brick point registry.

76.1 Operator workflow

1. Confirm [network setup](#).
2. Run discover from dashboard or API.
3. Review discovered objects; enable points for poll profiles.
4. Sync model bindings for FDD inputs.

77 Polling

78 Polling

BACnet field reads run in the openfdd-commission container.
The Operator Bridge does not bind UDP 47808.

Retired: openfdd-bacnet-poll image and openfdd-bacnet-poll systemd unit. Poll loop lives in **commission** only — do not run both.

BACnet polling has two stages:

- 1. **Field reads** — commission poll loop → workspace/bacnet/polls/samples.csv
- 2. **Historian ingest** — bridge background worker → workspace/data/feather_store/

78.1 Commission poll loop (default)

The **commission** container runs the poll loop at startup. It reads enabled rows from points.csv and appends BACnet RPM results to samples.csv.

Setting	Location
BACnet bind	workspace/bacnet/commissioning/commission.env
Enabled points	workspace/bacnet/commissioning/points.csv
Poll output	workspace/bacnet/polls/samples.csv
Historian	workspace/data/feather_store/

Check activity:

```
tail -1 workspace/bacnet/polls/samples.csv
docker compose logs --since 5m commission | grep -i poll
```

78.2 Bridge ingest worker

Inside **bridge**, a background thread watches samples.csv and ingests new rows into feather. Disable only for debugging:
OFDD_DISABLE_POLL_WORKER=1.

78.3 Health

Dashboard stack health (GET /health/stack) reports bacnet_poll status from the commission agent — not a separate container.

Stale points trigger data-quality fault patterns — see [Sensor quality faults](#).

79 Write safety

80 Write safety

BACnet writes can change live plant behavior. Open-FDD disables writes by default. Enable only with operator sign-off, a tested allowlist, and audit logging.

80.1 Defaults

Control	Default
Supervisory writes	Off
Write allowlist	Empty — must enumerate device/object
Audit	Commands logged when writes enabled

80.2 Enable writes (commission role)

1. Set env flag documented in [Security](#) (OFDD_BACNET_WRITE_ENABLED or site equivalent).
2. Populate write allowlist CSV / config with explicit object IDs.
3. Use **commission** role JWT; every write should include reason/ note where API supports it.
4. Test on bench hardware before production OT.

80.3 Operator rules

- Never write setpoints or overrides on life-safety or fire/smoke interlocks through experimental rules.
- Prefer read-only polling for FDD until rules are validated in Rule Lab.
- Revert overrides through BACnet priority relinquish where supported.

81 BACnet

82 BACnet

Open-FDD integrates with field controllers via BACnet/IP (and bench MSTP overlays in dev compose).

Page	Topic
Network setup	Bind address and NIC
Discover and read	Commission agent
Polling	Historian driver
Write safety	Defaults and allowlists

83 Convention

84 Fault code convention

84.1 Format (Grade-A, 3.0.18+)

Short stable code: FAMILY-SUBSYSTEM-NNN — e.g. AHU-ECON-001, CHW-DT-001, DATA-STAL-001.

Semantic ID: dotted canonical_id — e.g. ahu.economizer.not_using_free_cooling.

Legacy letter codes (AHU-E, VAV-C) remain **aliases** in open_fdd/faults/catalog/*.yaml until bridge sync completes.

Field	Example
code	AHU-ECON-001
canonical_id	ahu.economizer.not_using_free_cooling
family	AHU, VAV, CHW, DATA, CRS, ...
rule_doc_path	Link to rule cookbook recipe

Full schema: open_fdd/faults/schema.py. Catalog loader: open_fdd/faults/catalog.py.

84.2 Categories (Grade-A)

Category	Meaning
energy	Waste, reset stuck, plant inefficiency
comfort	Setpoint not met, poor zone performance
reliability	Short cycling, staging, equipment stress
maintenance	Leaking valves, filters, fouling proxies
indoor_air_quality	Ventilation shortfall, humidity
healthcare_risk	Pressure, isolation, critical-space excursions
data_quality	Stale, flatline, OOB, missing historian
controls_integrity	Overrides, chatter, schedule violations

84.3 Severity

Level	Operator response
info	Log; trend for maintenance window
low	Review in routine rounds
medium	Investigate within days
high	Prompt attention; comfort/energy impact likely
critical	Immediate operational risk (especially healthcare_risk)

84.4 Linking rules

In Rule Lab, set `fault_code` on the rule metadata. Batch FDD aggregates hits into `GET /api/faults/status` by family.

84.5 Cookbook mapping

Each catalog entry lists `cookbook_patterns` (e.g. `flatline_1h`, `mixing_envelope`, `rate_of_change`) — see [Expression cookbook \(Arrow-native\)](#).

Legacy pandas type: expression YAML rules are **not** used on the edge in 3.x; translate to `rules_py` Arrow modules and bind **`fault_code`** in Rule Lab.

85 Live site update

86 Live site update

Upgrade GHCR images on a **live edge host** that already has ~/open-fdd/ — no git pull on the host.

IT operators: prefer Quick Start — Updating the stack and openfdd_site_backup.sh / openfdd_site_update.sh on the edge host. This page adds control-machine UI deploy and Ansible paths.

86.1 Layout on the edge host

```
~/open-fdd/  
  docker-compose.yml  
  workspace/          # site state – backup first
```

workspace/ holds BACnet commissioning, poll CSV, feather historian, BRICK model, FDD rules, and auth.env.local. Image upgrades must **preserve** it.

See Backup and restore before any upgrade.

86.2 Concepts

Term	Meaning
Container	Running instance (bridge, commission, mcp-rag) — recreated on upgrade
Image	GHCR package (tag latest or dated tag)
workspace/	Open-FDD site state on disk — do not delete

86.3 1. Verify new images on GHCR

Three images must exist for every tag:

```
export NEW_TAG="${NEW_TAG:-latest}"  
  
docker manifest inspect ghcr.io/bbartling/openfdd-bridge:${NEW_TAG} >/dev/null &&  
docker manifest inspect ghcr.io/bbartling/openfdd-commission:$
```

```
{NEW_TAG} >/dev/null &&
    docker manifest inspect ghcr.io/bbartling/openfdd-mcp-rag:$
{NEW_TAG} >/dev/null &&
    echo "All three images exist for ${NEW_TAG}"
```

86.4 2. Update dashboard UI (when release changed React)

Image pull alone does not update the browser UI. The Operator Bridge serves workspace/api/static/app/ from the bind mount **before** image-baked assets.

From control machine (full repo):

```
cd /path/to/open-fdd
./scripts/build_operator_dashboard.sh prod
cd infra/ansible
RUN_POST_CHECK=0 ./deploy.sh ui --limit <inventory_host>
```

Confirm hash on edge:

```
curl -sf http://<edge-ip>/ | grep -o 'index-[^"]*\.\js' | head -1
```

One command (UI + images + insurance check):

```
OPENFDD_IMAGE_TAG=latest ./scripts/upgrade_edge_full.sh --limit
<inventory_host>
```

86.5 3. Pull and recreate (edge host)

```
cd ~/open-fdd
./scripts/openfdd_site_backup.sh
./scripts/openfdd_site_update.sh
```

Manual equivalent:

```
cd ~/open-fdd
docker compose pull
docker compose up -d --force-recreate
docker compose ps
curl -sf http://127.0.0.1:8765/health
```

86.6 4. Ansible image-only (control machine)

```
OPENFDD_IMAGE_TAG=latest RUN_POST_CHECK=1 ./scripts/
upgrade_edge_ghcr.sh --limit <inventory_host>
```

86.7 5. Validate

→ [Deployment validation](#)

86.8 Rollback

Restore docker-compose.yml from backup, pull previous tag, docker compose up -d --force-recreate. workspace/ is unchanged. Details: [Backup and restore](#).

86.9 Related

- [Updating the stack](#) — edge-host helper scripts
- [Containers](#) — three-image architecture

87 AHU faults

88 AHU faults

Air handling unit codes (sample from catalog).

Code	Title	Severity	Detection summary	Likely causes	Operator checks
AHU-A	Supply fan performance degradation	warning	Spread/runtime vs baseline	Belt slip, dirty filters, VFD limit	Trend speed vs airflow; filter dP
AHU-B	Simultaneous heating and cooling	critical	Heat + cool outputs active together	Sequencing bug, leaking valve	Trend coil commands; valve feedback
AHU-C	SAT sensor fault	warning	Flatline / OOB SAT	Failed sensor, bad calibration	SAT trend; compare to coil discharge
AHU-D	MAT inconsistent with OAT/RAT	warning	MAT outside OAT-RAT envelope	MAT/OAT/RAT sensor errors	Verify damper positions; cross-check OAT

Code	Title	Severity	Detection summary	Likely causes	Operator checks
AHU-E	Economizer not economizing	warning	Free cool available, OA minimum	Stuck damper, lockout	OA damper vs OAT trend
AHU-F	Damper command vs feedback mismatch	warning	Cmd $\neq$ feedback	Stuck actuator, linkage	Compare cmd/feedback trends

Example rule: flatline_1h on SAT → tag **AHU-C**. Recipe: [Python recipes](#).

89 Standards crosswalk

90 Standards crosswalk

Grade-A fault definitions carry a standards_crosswalk block in open_fdd/faults/catalog/*.yaml.

Field	Source	Use in Open-FDD
ornl_lbnl_taxonomy	ORNL/ LBNL unified HVAC fault taxonomy	Canonical ontology backbone
ashrae_g36_reference_type	ASHRAE Guideline 36	Trim-and-respond, plant requests, supervisory sequences
ashrae_207_reference_type	ASHRAE Standard 207	Economizer FDD test categories
brick_classes	<u>Brick</u> <u>Schema</u>	Equipment/point semantic roles
haystack_tags	Project Haystack (optional)	Interoperability tags

90.1 Example

```
standards_crosswalk:  
  ornl_lbnl_taxonomy: ahu/economizer/free_cooling  
  ashrae_g36_reference_type: supply_air_temp_reset  
  ashrae_207_reference_type: economizer_fault_detection  
  brick_classes: [brick:Air_Handler_Unit, brick:Economizer]  
  haystack_tags: [equip, ahu, econ]
```

```
Export full catalog: python3 -c "from open_fdd.faults.catalog  
import catalog_export; import json;  
print(json.dumps(catalog_export(), indent=2))"
```

91 Backup and restore

92 Backup and restore

workspace/ on the edge host is the **live site state** — feather historian, BACnet commissioning files, BRICK model, FDD rules, auth, and dashboard static bundle. Container images are replaceable; **backup workspace/ before every upgrade.**

92.1 Quick backup (edge host)

```
cd ~/open-fdd  
./scripts/openfdd_site_backup.sh
```

Archives land under ~/openfdd-backups/<timestamp>/.

Custom location:

```
BACKUP_ROOT=~/openfdd-backups/manual ./scripts/  
openfdd_site_backup.sh
```

92.2 Manual backup (SSH)

Container processes may write files as **root:root** with mode **0600**. Use **sudo** for a full archive:

```
cd ~/open-fdd  
  
export BACKUP_ROOT="$HOME/openfdd-backups/$(date +%Y%m%d-%H%M%S)"  
mkdir -p "$BACKUP_ROOT"
```

```
cp docker-compose.yml "$BACKUP_ROOT/docker-compose.yml.before"
docker compose ps > "$BACKUP_ROOT/docker-compose-ps-before.txt"
docker compose config --images > "$BACKUP_ROOT/docker-images-
before.txt"
```

```
sudo tar --xattrs --acls -czf "$BACKUP_ROOT/workspace-full.tgz"
workspace
sudo chown "$USER:$USER" "$BACKUP_ROOT/workspace-full.tgz"

du -h "$BACKUP_ROOT/workspace-full.tgz"
```

92.3 What to protect

Path	Contents
workspace/data/ feather_store/	Feather historian
workspace/bacnet/ commissioning/	points.csv, commission.env
workspace/bacnet/ polls/samples.csv	Poll output
workspace/data/*.json	Model, rules store, FDD results
workspace/ auth.env.local	Login secrets
workspace/api/static/ app/	Dashboard bundle (if deployed separately)

92.4 Restore after a bad upgrade

1. Stop containers: `docker compose stop` (do **not** use `down -v`)
2. Restore `docker-compose.yml` from backup
3. If data corruption: extract selective files from `workspace-full.tgz` — model, rules, feather shards
4. `docker compose pull && docker compose up -d --force-recreate`
5. Verify: [Deployment validation](#)

92.5 Safe cleanup

```
docker image prune -f
```

Never on live sites: `docker compose down -v`, `docker volume prune`, `docker system prune -a --volumes`, or deleting `workspace/`.

92.6 Related

- [Updating the stack](#)
- [Live site update](#)

93 VAV faults

94 VAV faults

Variable air volume terminal codes (sample).

Code	Title	Severity	Detection summary	Likely causes	Operator checks
VAV-A	Zone heating/cooling overlap	warning	Heat and cool at terminal	Reheat valve stuck, SAT too low	Zone temp vs SAT; valve cmd
VAV-B	Poor zone temperature control	warning	Persistent deviation from setpoint	Oversized box, bad tuning	PI loop trends; occupancy
VAV-C	Zone temperature sensor fault	warning	Flatline / OOB zone temp	Failed zone sensor	Flatline test; compare to neighbor zones
VAV-D	Minimum airflow not met	warning	Flow below minimum cmd	Damper stuck, pressure issue	Airflow vs damper cmd
VAV-E	Discharge temp deviation from SAT	warning	DAT far from SAT setpoint	Coil issue, sensor drift	DAT vs SAT SP

Example rule: oob_rolling on zone temp → **VAV-C** or comfort bounds per site policy.

95 RTU faults

96 RTU faults

Packaged rooftop unit codes (sample).

Code	Title	Severity	Detection summary	Likely causes	Operator checks
RTU-A	Compressor short cycling	warning	Rapid on/off cycles	Low charge, oversized unit	Run-time histogram
RTU-B	Economizer / OA damper fault	warning	OA behavior vs conditions	Actuator, sensor	OA damper vs enthalpy/OAT
RTU-C	Supply air temperature fault	warning	SAT flatline or OOB	Sensor failure	SAT trend vs outdoor conditions
RTU-D	Simultaneous heat and cool	critical	Heat + cool stages together	Control board, relay stuck	Stage status trends

Bind rules to packaged unit fdd_input points (SAT, OAT, compressor cmd, ...).

97 Deployment validation

98 Deployment validation

Non-destructive checks after deploy or image upgrade. Run from the **edge host** (smoke) or from a **control machine** with the full repo (insurance suite).

98.1 Edge host smoke (5 minutes)

```
cd ~/open-fdd

docker compose ps
curl -sf http://127.0.0.1:8765/health | jq . || curl -sf http://
127.0.0.1:8765/health

tail -1 workspace/bacnet/polls/samples.csv
du -sh workspace/data/feather_store/

docker compose logs --since 20m | grep -Ei "error|exception|
traceback|critical" | tail -20 || true
```

Via Caddy (building LAN):

```
curl -sf http://<edge-ip>/health
curl -sf http://<edge-ip>/ | grep -o 'index-[^"]*\.' | head -1
```

98.2 Browser checklist

1. Open `http://<edge-ip>/` (Caddy :80, not only loopback :8765)
2. Integrator login (`workspace/auth.env.local` on the edge host)
3. **Host stats** — container image tag / git SHA
4. **BACnet** — driver tree shows devices; last poll timestamp recent
5. **Model & assignments** — commissioning export returns JSON
6. **Rule Lab** — quick-test a saved rule (backend: arrow)

98.3 Control-machine insurance check

From a machine with the full open-fdd repo and Ansible inventory:

```
cd infra/ansible/scripts
./post_deploy_check.sh --limit <inventory_host> --full
```

Or by IP:

```
./post_deploy_check.sh --host <edge-ip> --ssh-user <deploy-user>
--full
```

Checks include: Caddy → React dashboard, `/health/stack`, MCP RAG, BACnet tree, BRICK/SPARQL, Docker log scan.

Do not run on live commissioned sites without approval: `acme_operational_verify.sh` with BACnet rediscover — use `--skip-discover` for smoke only. See [Examples — GL36 lab](#).

98.4 LAN security smoke (optional)

From a workstation on the same LAN as the edge:

- Windows: scripts/security/Run-OpenFddSecurityScan.ps1
- macOS/Linux: scripts/security/run_openfdd_security_scan.sh

Expected findings: [ZAP baseline](#).

98.5 Arrow / FDD rules (3.0+)

On control machine:

```
./scripts/validate_fdd_backends.sh --docker
```

On edge after upgrade:

```
grep -R "apply_faults_arrow" workspace/data/rules_py | wc -l
```

98.6 Related

- [Health check](#)
- [Security testing cycle](#) (maintainers)

99 Sensor and data quality

100 Sensor and data quality

Sensor faults use cookbook patterns `flatline_1h`, `oob_rolling`, `rate_of_change`, and `mixing_envelope`. Full thresholds: [Expression cookbook — sensor validation](#).

100.1 Catalog codes

Code	Title	Severity	Pattern	Detection summary
BLD-B	Outdoor air sensor fault	warning	flatline, oob, rate_of_change	OAT flatline, OOB, or unrealistic spike
		critical	stale_points	No new samples

Code	Title	Severity	Pattern	Detection summary
BLD-D	Stale telemetry			within threshold
AHU-C	SAT sensor fault	warning	flatline_1h, oob_rolling	Supply air temp stuck or out of band
AHU-D	MAT inconsistent with OAT/RAT	warning	mixing_envelope	Mixed air outside OAT-RAT envelope
VAV-C	Zone temperature sensor fault	warning	flatline_1h, oob_rolling	Zone temp stuck or 55–90 °F band breach
RTU-C	Supply air temperature fault	warning	flatline_1h, oob_rolling	Packaged unit SAT fault
CH-D	CHW supply temperature sensor fault	warning	flatline_1h, oob_rolling	Plant CHW sensor fault
DC-C	Datacom sensor anomaly	warning	flatline_1h	CRAH/room sensor flatline or RH OOB
HP-D	Discharge/suction temperature sensor fault	warning	flatline_1h, oob_rolling	Heat pump refrigerant/air temp sensor

100.2 Default bounds (imperial, occupied)

Sensor	Min	Max	Flatline tol	Max Δ/hr
Zone temp	55 °F	90 °F	0.10 °F	4 °F
Supply air temp	50 °F	110 °F	0.15 °F	8 °F
Return air temp	55 °F	95 °F	0.10 °F	3 °F
Duct static	–0.5	3.0 inH ₂ O	0.02	0.5
RH	0 %	100 %	1.0 %	15 %
CHW	40 °F	90 °F	0.10 °F	4 °F
HW	70 °F	200 °F	0.15 °F	6 °F
	50 °F	110 °F	0.15 °F	5 °F

Sensor	Min	Max	Flatline tol	Max Δ /hr
Condenser water				
CO ₂ (occupied)	400 ppm	1000 ppm	5 ppm	200 ppm

Return-air checks use **narrow bands**; when MAT/OAT/RAT are available, prefer **mixing envelope** over RAT bounds alone.

100.3 Communication quality

- Monitor poll cycle timestamps in dashboard.
- Use stale_points for dropout detection (**BLD-D**).
- Do not confuse **comm loss** with **equipment fault** — verify BACnet device online before dispatching mechanical.

100.4 Operator workflow

1. Confirm **BLD-D** (stale) is resolved before trusting other sensor codes.
2. Cross-check **BLD-B** OAT against weather service or JSON API driver.
3. For **VAV-C** / **AHU-C**, compare neighbor zones / coil behavior before replacing sensor.

101 Logging and audit

102 Logging and audit

Open-FDD uses a **two-tier logging model** familiar to IT and security teams: structured **audit/error JSONL** for forensics and compliance, plus **service stdout** for engineering troubleshooting. Defaults include **age pruning**, **size rotation in MB**, and **Docker log caps** — no operator cron required.

102.1 Log types

Tier	Location	Format	Who reads it
Audit	workspace/logs/ audit.jsonl	JSON Lines	Security, MSI, integrator API

Tier	Location	Format	Who reads it
Errors	workspace/logs/error.jsonl	JSON Lines	Support, integrator API
Service stdout	Docker: docker compose logs · Dev: workspace/.local-run/*.log	Plain text (uvicorn, workers)	Engineering
Historian	workspace/data/feather_store/	Feather	FDD, plots — separate retention (data.env.local)

Structured audit logs are **not** mixed into uvicorn access logs. That separation matches common practice: SIEM-friendly JSONL for auth and OT commands; unstructured stdout for stack traces and poll worker noise.

102.2 Audit record schema

Each line in audit.jsonl / error.jsonl is one JSON object:

Field	Example	Notes
@timestamp	2026-06-07T19:57:21+00:00	UTC ISO-8601
event_id	UUID	Unique per event
event_type	auth.login.failure	Stable taxonomy (see below)
severity	warning	debug ... critical
outcome	failure	success, failure, denied, ...
service	openfdd-bridge	
host	hostname	
action	login	Human-readable verb
actor	{username, role}	After successful auth
client	{ip, user_agent}	Respects OFDD_TRUST_X_FORWARDED_FOR behind Caddy
http	method, path, status	When HTTP-triggered
resource	BACnet object, model id, ...	When applicable
detail	Sanitized map	Passwords, tokens, secrets stripped by key name
request_id	UUID	On audited HTTP paths

102.2.1 Auth events (industry-standard login trail)

event_type	When
auth.login.success	Valid credentials
auth.login.failure	Bad password (generic client message)
auth.login.lockout	Rate limit exceeded (default 5 failures / 5 min)

Login rate limits and lockouts are **audited**; responses never reveal whether the username exists.

102.2.2 OT / commissioning events

event_type	When
bacnet.command	Supervisory write (allowed, denied, dry-run)
bacnet.discover	Who-Is / point discovery
model.write	BRICK model mutations
agent.tool	Agent tool calls (prompts hashed/truncated)
api.access	Sensitive POST/PUT/PATCH/DELETE (BACnet, ingest)
error.unhandled	Unhandled exception (also in error.jsonl)

Not audited: routine GETs (dashboard reads, plot data, health). This reduces noise while keeping **auth and field commands** on the trail.

102.2.3 Secret redaction

Never logged: passwords, bearer tokens, 0FDD_AUTH_SECRET, private keys. Detail objects drop keys containing password, token, secret, authorization. Agent tool arguments use additional hashing for prompts and rule source.

Opt-in full prompt logging (lab only): 0FDD_AUDIT_LOG_PROMPTS=1.

102.3 Integrator API (read logs in the UI stack)

Method	Path	Role
GET	/api/audit/events?limit=100	integrator
GET	/api/audit/errors?limit=100	integrator

Method	Path	Role
GET	/api/audit/summary	integrator

Use these for post-incident review without shell access. Forward workspace/logs/*.jsonl to SIEM (rsyslog, Filebeat, Wazuh) on production edges.

102.4 Retention and rotation (defaults)

Configured in workspace/data.env.local (copy from data.env.example):

Variable	Default	Behavior
OFDD_LOG_RETENTION_DAYS	90	Drop audit/error JSONL lines older than N days; delete aged .local-run and archived *.log / audit.*.jsonl
OFDD_LOG_MAX_MB	50	When audit.jsonl or error.jsonl exceeds N MB, archive to audit.{UTC}.jsonl and start fresh
OFDD_LOCAL_RUN_LOG_MAX_MB	25	Rotate workspace/.local-run/*.log (bridge, Caddy, FDD loop, ...)
OFDD_LOG_ROTATE_INTERVAL_HOURS	6	Background retention while bridge is running (not only at restart)

Rotation runs at **bridge startup** and on the **scheduled interval**. No logrotate.d entry is required for audit JSONL.

Optional path overrides:

```
OFDD_AUDIT_LOG_PATH=/var/log/openfdd/audit.jsonl
OFDD_ERROR_LOG_PATH=/var/log/openfdd/error.jsonl
```

102.5 Docker edge: json-file caps (not journald for app audit)

Production stacks use **Docker Compose** (docker/compose.edge.yml). Container stdout uses the **json-file driver with size limits**:

```
logging:
  driver: json-file
  options:
    max-size: "25m"
    max-file: "5"
```

That caps each service at roughly **125 MB** of rotated container logs on disk — the same pattern many teams use instead of unbounded docker logs.

Why audit JSONL stays on disk (not journald)?

- Append-only files under workspace/logs/ survive container recreate and bind-mount to the host.
- JSONL lines import cleanly into SIEM without journal field parsing.
- OT incident response often needs **months** of auth/BACnet write history on the edge box.

When journald still applies: native systemd units (Caddy, Ollama on bare metal) — use journalctl -u <unit> per your Linux runbook. Bridge audit remains in workspace/logs/ even when uvicorn stdout goes to Docker or .local-run/.

102.6 Local dev (run_local.sh)

File	Service
workspace/.local-run/bridge.log	Bridge / uvicorn
workspace/.local-run/commission.log	BACnet commission agent
workspace/.local-run/caddy.log	Caddy
workspace/.local-run/fdd_loop.log	Scheduled FDD
workspace/.local-run/mcp_rag.log	MCP RAG
workspace/.local-run/ollama.log	Ollama

These are **engineering logs**. Structured audit still lands in workspace/logs/audit.jsonl when auth is enabled.

102.7 Quick checks

```
# Last auth failures
grep 'auth.login' workspace/logs/audit.jsonl | tail -5

# Integrator API (after login)
curl -s -H "Authorization: Bearer $TOKEN" http://127.0.0.1:8765/
api/audit/summary | jq .

# Docker edge
docker compose logs bridge --tail 50
```

102.8 Comparison to typical web apps

Practice	Open-FDD
Structured auth audit	Yes — JSONL with actor, IP, outcome
Login rate limit + logout audit	Yes
Secret redaction in logs	Yes — key-based + agent sanitization
Request correlation ID	Yes — on audited mutation paths
Centralized stdout JSON	No — stdout is plain text; audit is JSONL
Full HTTP access log	No — mutations and auth only (OT noise reduction)
Log rotation / size caps	Yes — defaults above + Docker max-size
SIEM-ready export	Yes — file tail / Filebeat on workspace/logs/

For BACnet write safety and credential generation, see [SECURITY.md](#) and [Secrets and auth](#).

102.9 Related

- [JSON API driver](#) — OpenWeatherMap showcase, env-based API keys (json_api.env.local)
- [Live site update](#) — docker compose logs after upgrades
- [Data flow](#) — historian retention (OFDD_FEATHER_*) is separate from audit logs

103 Fault Codes

104 Fault Codes

Open-FDD uses a **check-engine** model: one building-level status (green / yellow / red) with a catalog of fixed fault codes per equipment family.

Page	Content
Convention	Naming, severity, categories
AHU faults	Air handling units
VAV faults	Terminal units
RTU faults	Packaged rooftops
Sensor & data quality	Stale, flatline, comms

Live catalog API: GET [/api/faults/catalog](#) on the Operator Bridge.

105 Operations

106 Operations

Runbooks for live edge hosts after the initial [Quick Start](#) deploy.

Page	When to use
Live site update	Pull new GHCR tags, preserve workspace/
Backup and restore	Archive workspace/ before upgrades
Logging and audit	Auth audit trail, rotation, Docker log caps
Deployment validation	Post-upgrade smoke and insurance checks

Site-specific lab notes (example BACnet scope, GL36 rules):
[Examples & lab notes](#).

For first-time deploy: [Quick Start](#) — [Docker](#).

107 LAN hardening

108 LAN hardening

Open-FDD targets **trusted building LAN / OT edge** deployment. The Operator Bridge is not intended for direct public internet exposure.

Full mode matrix: [Deployment modes](#).

108.1 Auth and bind

Mode	Bridge bind	Auth
Local dev	127.0.0.1	Optional OFDD_AUTH_DISABLED=1 on loopback
Lab LAN	0.0.0.0	Required unless explicit insecure lab flags
Edge production	Loopback + Caddy on LAN	Required — OFDD_AUTH_SECRET + role passwords

Startup **fails** on non-loopback bind without credentials (unless insecure lab flags are set).

108.2 Network exposure

- Put **Caddy** on :80 / :443; keep bridge :8765 on loopback when possible.
- Do not port-forward the Operator Bridge to the public internet without TLS and strong auth.
- Keep BACnet on OT VLANs; management UI on IT VLAN with firewall rules.
- BACnet **writes** are gated by default — [BACnet write guard](#).

108.3 Rule Lab

Rule Lab runs **operator-authored Python** in a sandbox. Restrict integrator accounts; review rules before enable on production sites.

108.4 TLS and LAN security scans

Topic	Page
Certificates (self-signed / site CA)	TLS and certificates
ZAP + Nmap bench smoke	ZAP baseline · scripts/security/
Ubuntu host + Tenable/Nessus	Linux host hardening · Tenable remediation
Release test cycle (maintainers)	Security testing

108.5 Ollama on the edge

Ollama has **no built-in auth**. Bind `OLLAMA_HOST=127.0.0.1:11434` on the host; never expose `:11434` on the OT LAN. Open-FDD bridge reaches Ollama server-side only. Validation: [scripts/security/check_openfdd_exposure.sh](#).

109 ZAP baseline (local bench)

110 ZAP baseline (local bench)

Passive [OWASP ZAP baseline](#) against the **pentest production stack** on benserver is the standard LAN security smoke before Acme edge deploys.

Full per-revision workflow (pytest → PR → GHCR → Acme):
[Developer — security testing cycle](#).

Packaged scan scripts (Windows + Mac/Linux): [scripts/security/README.md](#).

110.1 Run

On the Open-FDD host:

```
./scripts/pentest_production_stack.sh start
./scripts/pentest_production_stack.sh verify
```

From a LAN workstation (after verify passes):

```
# Windows
.\scripts\security\Run-OpenFddSecurityScan.ps1

# macOS / Linux
./scripts/security/run_openfdd_security_scan.sh --url http://
192.168.204.18
```

ZAP target (example): `http://192.168.204.18/` — Caddy on `:80` only;
bridge `:8765` stays loopback. TLS bench: see [TLS and certificates](#).

110.2 Expected results (HTTP bench mode)

Finding	Severity	Status
CSP style-src 'unsafe-inline'	Medium	Accepted — Vite/React inline styles; TODO to nonce/hash
Missing COEP	Low	Accepted — require-corp not enabled (breaks some assets)
Port 80 open (Caddy)	Info	Expected — intentional LAN entry
Ports 8765, 5173, 8000, 8080 closed	—	Good
Port 443 filtered	—	Expected in HTTP mode; use <code>OFDD_CADDY_MODE=tls</code> for HTTPS bench
Duplicate Referrer-Policy / X-Frame-Options	Low	Fixed — bridge owns headers; Caddy does not duplicate

110.3 Header ownership

Layer	Sets
Bridge (security_headers.py)	CSP, X-Frame-Options: DENY, frame-ancestors 'none', COOP, CORP, Permissions-

Layer	Sets
	Policy, Referrer-Policy, X-Content-Type-Options
Caddy HTTP	Reverse proxy only; strips Server banners
Caddy TLS	Adds Strict-Transport-Security only

110.4 Authenticated scan (later)

Baseline scan hits ~4 public endpoints (/health, public agent insight routes). Deep API/dashboard coverage requires ZAP with integrator login — see [Authenticated scanning \(roadmap\)](#).

110.5 After fixes

Re-run `pentest_production_stack.sh` verify, then the packaged LAN scan from `scripts/security/`. Confirm response headers show **one** X-Frame-Options: DENY and no duplicate Referrer-Policy.

111 TLS and certificates

112 TLS and certificates

Who generates, stores, and renews TLS material for Open-FDD on OT LANs — and how that ties into ZAP/Nmap bench testing.

Developer release cycle: [Security testing cycle](#).

112.1 Ownership

Environment	Who owns certs	Storage
Local / benserver bench	Developer / maintainer	workspace/ deploy/caddy/ certs/ (cert.pem, key.pem)
Ansible edge (Acme, customer sites)	Site integrator	/etc/openfdd/ caddy/ on the VM

Environment	Who owns certs	Storage
Public internet	Not in scope	(playbook-managed or manual) Open-FDD targets OT LAN; use site PKI or self-signed

Open-FDD does **not** ship Let's Encrypt or public CA automation. OT benches use **self-signed** or customer-provided certs.

112.2 Generate bench certs (local)

```
./scripts/setup_caddy_certs.sh
# or: ./scripts/setup_caddy_certs.sh --cn openfdd.local --out
workspace/deploy/caddy/certs
```

Then start with TLS mode:

```
OFDD_CADDY_MODE=tls ./scripts/run_local.sh
# or pentest stack with OFDD_CADDY_MODE=tls
```

Caddy serves :443 and redirects :80 → HTTPS. Caddy adds **Strict-Transport-Security** only; all other security headers remain on the **bridge** (ZAP baseline).

112.3 ZAP / Nmap with TLS bench

When OFDD_CADDY_MODE=tls:

- Nmap: expect **443 open**, 80 may redirect
- ZAP: use https://<lan-ip>/ (add -k trust or import cert.pem on the scan workstation)
- PowerShell: .\Run-OpenFddSecurityScan.ps1 -TargetUrl "https://192.168.204.18"
- Bash: ./run_openfdd_security_scan.sh --url https://192.168.204.18

Self-signed certs produce ZAP TLS warnings — expected on OT LAN.

112.4 Renewal

Self-signed certs from `setup_caddy_certs.sh` default to **365 days**. Re-run the script before expiry or bake renewal into site runbooks.

Edge Ansible deploys should document whether the integrator rotates certs manually or via site PKI.

112.5 Enterprise CA on Caddy (edge)

When IT requires **clean Nessus SSL plugins** (no self-signed / untrusted chain), install a site-provided certificate on the edge VM instead of `setup_caddy_certs.sh` output.

Typical layout (Ansible / manual):

```
/etc/openfdd/caddy/cert.pem # full chain (leaf + intermediates)
/etc/openfdd/caddy/key.pem  # private key (0600, root or caddy
user)
```

Ansible TLS mode (`caddy_mode: tls`) already points Caddy at those paths — see `infra/ansible/templates/Caddyfile.j2`.

Internal enterprise CA workflow:

1. Generate CSR on the edge host or request cert from site PKI for the operator hostname (e.g. `openfdd.acme.local`).
2. Install `cert.pem` + `key.pem` under `/etc/openfdd/caddy/`.
3. Distribute the **CA root** to admin laptops (trust store) so browsers and ZAP scans validate the chain.
4. Monitor expiry:

```
openssl x509 -in /etc/openfdd/caddy/cert.pem -noout -dates
```

Caddyfile snippet (TLS, enterprise cert):

```
:443 {
    tls /etc/openfdd/caddy/cert.pem /etc/openfdd/caddy/key.pem
    header {
        Strict-Transport-Security "max-age=31536000;
includeSubDomains"
    }
    reverse_proxy 127.0.0.1:8765
}
```

Lab/self-signed mode still produces expected **Medium** Tenable findings — document as accepted risk or switch to enterprise CA. See [Tenable remediation](#).

112.6 Caddy internal CA mode (lab)

`./scripts/setup_caddy_certs.sh` generates a **self-signed** pair for bench TLS. Trust `cert.pem` on scan workstations (`-k curl`, ZAP import) or accept browser warnings. Not suitable when IT mandates trusted chains.

112.7 Secrets

- **Never commit** `key.pem` or production cert bundles
- `workspace/deploy/caddy/certs/` is for local bench only unless your `.gitignore` already excludes generated keys (verify before push)

113 Authenticated scanning (roadmap)

114 Authenticated scanning (roadmap)

ZAP baseline is **unauthenticated** — it checks public pages, response headers, and a handful of routes reachable without login. That is the right default for every docker image / patch revision smoke on a test LAN.

For **deep** coverage of integrator dashboards, `/api/*` routes, WebSocket sessions, and role-gated writes, plan a separate **authenticated** scan — only on fake/bench stacks with explicit approval.

Release context: [Security testing cycle](#).

114.1 Current state (3.0.x)

Capability	Status
ZAP baseline via <code>scripts/security/</code>	Shipped
Pentest creds in <code>workspace/auth.pentest.local</code>	Shipped (host-local, not in git)
ZAP full / active scan automation	Not shipped — manual only

Capability	Status
ZAP authenticated context (login script + session)	Roadmap
CI running ZAP against PR previews	Not planned (LAN bench only)

114.2 Why baseline is not enough

After login, the bridge exposes model editing, BACnet/JSON API configuration, Rule Lab, commissioning export, and write-gated commands. Baseline ZAP never exercises:

- Session cookies / JWT persistence
- CSRF or auth bypass on mutating endpoints
- IDOR on site-scoped resources
- Rate limits and audit logging on failed auth

114.3 Planned approach (contributors welcome)

1. **Bench-only target** — same pentest stack; never point active scans at live OT controllers
2. **Login hook** — ZAP zap-full-scan.py or Automation Framework with:
 - POST /api/auth/login using integrator creds from env (not committed)
 - Regex or JSON extractor for session token / cookie
3. **Scope file** — include /api/model/*, /api/commissioning/*, dashboard static assets; exclude BACnet UDP and unrelated LAN hosts
4. **Report parity** — extend scripts/security/ with opt-in - Authenticated / --zap-full flags and separate report prefix (22-zap-authenticated-*)
5. **Gate** — document accepted findings; fail release only on High/Critical regressions

114.4 Manual authenticated ZAP today

1. Start pentest stack; note creds in workspace/auth.pentest.local
2. Open ZAP desktop or docker with API port exposed
3. Configure **Context** → **Authentication** → script or form-based login to /api/auth/login
4. Run **Spider** then **Active Scan** on http://<lan-ip>/ only
5. Compare with baseline report in openfdd-security-report/

Do **not** run active scans against production Acme without a maintenance window and BACnet write lock.

114.5 Related

- [scripts/security/README.md](#) — baseline setup
- [Secrets and auth](#) — env file layout
- [LAN hardening](#) — bind and auth modes

115 Secrets and auth

116 Secrets and auth

116.1 Files (gitignored)

File	Contents
workspace/ auth.env.local	JWT secret, role passwords
workspace/ json_api.env.local	REST API keys for JSON API <code>\${ENV:VAR}</code> URLs (e.g. OpenWeather <code>OPENWEATHER_API_KEY</code>)
workspace/ data.env.local	Historian + audit log retention (<code>OFDD_LOG_*</code> , <code>OFDD_FEATHER_*</code>)
workspace/ ollama.env.local	Ollama URL, model, RAM tier, GPU mode
workspace/ mcp.env.local	MCP RAG sidecar
workspace/ caddy.env.local	Reverse proxy TLS/HTTP mode
workspace/ pentest.env.local	LAN security scan stack
infra/ansible/secrets/ <host>.env.local	SSH deploy secrets (maintainers)

Copy from *.example templates only. Full inventory: [Workspace env files](#).

116.2 Required variables (production)

Variable	Purpose
OFDD_AUTH_SECRET	HMAC signing (32+ chars)
OFDD_INTEGRATOR_USER / PASSWORD	Rule Lab, bindings
OFDD_OPERATOR_USER / PASSWORD	Operations

116.3 GHCR pulls

Use read-only registry credentials on edge if images are private; rotate tokens periodically.

116.4 Backup

Back up workspace/data/ and auth env in secure operator vault — not in git.

Audit JSONL under workspace/logs/ may contain client IPs and usernames — treat like security logs. See [Logging and audit](#).

117 BACnet write allowlists

118 BACnet write allowlists

Supervisory writes require:

1. **Feature flag** enabled in environment (off by default).
2. **Allowlist** file at workspace/bacnet/write_allowlist.json (copy from [write_allowlist.example.json](#)). If the flag is on but the allowlist is missing, writes are **denied**.
3. **Integrator** role on API calls.
4. **Audit** log review after commissioning sessions.

Allowlist entries support device_instance, object_identifier, property_identifier, optional priority_min/priority_max, value_min/value_max, and allowed_values for binary/multistate targets.

Lab-only override: OFDD_BACNET_WRITE_ALLOW_ANY=1 (audited; not for production edges).

See operator guide: [BACnet write safety](#).

Never enable blanket write access for agent or integrator automation without human approval per site.

119 Linux host hardening

120 Linux host hardening

Generic **Ubuntu edge VM** guidance for Open-FDD deployments scanned by IT tools (Tenable/Nessus, Qualys, etc.). This complements application-level [LAN hardening](#) and [TLS](#).

Nessus-specific finding tables: [Tenable remediation](#).

120.1 Scope

Layer	Owner	Repo support
Ubuntu kernel, OpenSSH, apt packages	Site IT / integrator	scripts/security/remediate_ubuntu_host.sh
Docker engine, compose, Caddy	Integrator	Ansible + check_openfdd_exposure.sh
Open-FDD containers (pip/pyOpenSSL)	Open-FDD maintainers	docker/python-security-constraints.txt, GHCR images
Self-signed TLS on OT LAN	Expected lab finding	Enterprise CA path in TLS

120.2 Quick operator workflow

On the edge VM (SSH):

```
cd ~/open-fdd
./scripts/security/check_host_security.sh
./scripts/security/check_openfdd_exposure.sh --edge-ip "$
(hostname -I | awk '{print $1}')
```

Remediate host OS (maintenance window):

```
./scripts/security/remediate_ubuntu_host.sh # dry-run
sudo ./scripts/security/remediate_ubuntu_host.sh --apply
# if /var/run/reboot-required exists:
sudo reboot
./scripts/security/check_host_security.sh
```

Pull new Open-FDD images after a release that pins container pip/pyOpenSSL:

```
OPENFDD_IMAGE_TAG=latest ./scripts/upgrade_edge_ghcr.sh --limit
<inventory_host>
```

120.3 Ubuntu version and EOL

Supported targets: **Ubuntu 22.04 LTS** or **24.04 LTS** (64-bit).
Rebuild hosts still on **16.04** or **18.04** — apt upgrade cannot clear an SEoL finding.

Validate a surprising 16.04 Nessus hit:

```
lsb_release -a
cat /etc/os-release
# Scan credential / stale DNS / container banner mismatch?
```

120.4 Kernel and package updates

```
sudo apt update
sudo apt full-upgrade -y
uname -r
test -f /var/run/reboot-required && echo "reboot required"
sudo reboot # maintenance window
```

Optional status tools:

```
ubuntu-security-status # or: pro status
apt list --upgradable | grep -i security
```

120.5 Host Python (pip, wheel, pyOpenSSL)

Tenable often reports **host** pip, wheel, and pyOpenSSL versions installed by Ubuntu packages or admin pip install. Prefer **apt** on the host:

```
python3 -m pip show pip setuptools wheel pyOpenSSL
sudo apt install --only-upgrade python3-pip python3-openssl
python3-wheel
```

Open-FDD **containers** pin toolchain versions at image build — see `docker/python-security-constraints.txt`. Rescan after `docker compose pull`.

120.6 Ollama — no LAN exposure

Ollama has **no built-in authentication** in typical deployments. Treat LAN exposure as **Critical**.

Rule	Implementation
Bind loopback only	OLLAMA_HOST=127.0.0.1:11434 in systemd (ollama.service) or dev bootstrap
Bridge calls server-side	OFDD_OLLAMA_BASE_URL=http://127.0.0.1:11434 or host.docker.internal:11434 from containers
Never publish :11434 on 0.0.0.0	Compose uses 127.0.0.1:11434:11434 when Docker Ollama is enabled

Validation:

```
ss -tulpn | grep 11434
curl -sf http://127.0.0.1:11434/api/tags | head
curl -sf --max-time 3 http://<edge-lan-ip>:11434/api/tags
# must FAIL from another host
```

Firewall (if management host needs GPU Ollama — rare):

```
# Example: block LAN → 11434; allow loopback only (adjust
interface names)
sudo ufw deny 11434/tcp
sudo ufw allow from 127.0.0.0/8 to any port 11434 proto tcp
```

120.7 OpenSSH Terrapin (CVE-2023-48795)

Apply Ubuntu security updates first (openssh-server, openssh-client). Verify:

```
ssh -V
sudo sshd -T | grep -Ei '^(ciphers|macs|kexalgorithms)'
```

Manual algorithm hardening is **optional** and can lock out clients — document rollback (`sshd_config.bak`) before changes. Do not automate cipher stripping in repo scripts.

120.8 ICMP timestamp (Low)

Nessus may report ICMP timestamp disclosure. Optional hardening at host firewall or upstream network — low priority for isolated OT VLANs.

Example (nftables — site-specific):

```
# Block ICMP timestamp / type 13 – verify with IT before applying
sudo nft add rule inet filter input icmp type timestamp-request
drop
```

120.9 TLS and certificate scans

Self-signed or internal CA certificates produce **Medium** Nessus SSL findings (untrusted, self-signed, expiry). That is **expected** for lab/OT encryption-in-transit. For clean SSL plugin results, install a **site enterprise CA** certificate on Caddy — [TLS and certificates](#).

120.10 Maintainer security audits (CI parity)

```
./scripts/security/run_maintainer_audits.sh
```

Matches GitHub Actions operator-bridge-security job where possible.

121 Tenable / Nessus remediation

122 Tenable / Nessus remediation

Remediation plan for **IT/Tenable.io Nessus** findings on an Open-FDD **OT/LAN edge** deployment (e.g. Acme GL36 lab VM). Open-FDD runs behind a building firewall on Ubuntu with Docker Compose, Caddy, and optional host Ollama.

Related: [Linux host hardening](#), [TLS and certificates](#), [LAN hardening](#), [Security testing cycle](#).

122.1 Prioritized action plan

Priority	Action	Owner
P0	Confirm Ubuntu is not 16.04/18.04; rebuild if EOL	IT / integrator
P0	Bind Ollama to 127.0.0.1 only; verify LAN cannot reach :11434	Integrator
P0	apt full-upgrade + reboot for kernel USNs (8254, 8278, 8373)	IT / integrator
P1	Upgrade host pip/wheel/pyOpenSSL via apt or documented pip path	IT / integrator
P1	Pull GHCR images built with docker/python-security-constraints.txt	Integrator
P2	OpenSSH packages current; review Terrapin after patch	IT
P2	TLS: accept self-signed findings or install enterprise CA cert	IT decision
P3	ICMP timestamp: optional firewall rule	Network / IT

122.2 Finding classification table

Nessus finding	Severity	Class	Remediation
Canonical Ubuntu Linux SEoL (16.04.x)	Critical	Host OS / false positive?	lsb_release -a — rebuild VM on 24.04 LTS if truly 16.04; if host is 22.04/24.04, check scan target IP, credentials, stale asset record
	Critical	Deployment config	

Nessus finding	Severity	Class	Remediation
Ollama Unauthenticated Access			OLLAMA_HOST=127.0.0.1:11434; no 0.0.0.0:11434; firewall deny; see Ollama
Ubuntu kernel USN-8254-1	Critical	Host OS	sudo apt update && sudo apt full-upgrade -y && sudo reboot
pyOpenSSL 22.0.x < 26.0.0 Buffer Overflow	Critical	Host pip and container	Host: apt upgrade python3-openssl or pip ≥26; Container: GHCR image with constraints file
Ubuntu pip USN-8344-1	High	Host OS + container build	apt full-upgrade; image build upgrades pip>=26
SSL Certificate Cannot Be Trusted	Medium	Accepted risk (lab) / deployment	Self-signed Caddy — use enterprise CA for clean scan
SSL Self-Signed Certificate	Medium	Accepted risk (lab)	Same as above
SSL Certificate Expiry	Medium	Deployment / monitoring	openssl x509 -dates; renew setup_caddy_certs.sh or site PKI
SSH Terrapin (CVE-2023-48795)	Medium	Host OS	Upgrade openssh-server; validate ssh -V
Ubuntu wheel USN-8221-1	Medium	Host OS	apt full-upgrade
Ubuntu kernel USN-8278-1, USN-8373-1	Medium	Host OS	apt full-upgrade + reboot
pyOpenSSL 0.14.x Security Bypass	Medium	Host + container	Same as pyOpenSSL Critical — upgrade to ≥26
ICMP Timestamp Request	Low	Network / optional host	Firewall drop; often accepted on isolated OT VLAN

Class legend: (1) repo-owned fix · (2) Docker base/image · (3) host OS · (4) deployment/config · (5) accepted risk · (6) validate false positive

122.3 Operator commands (edge VM)

122.3.1 Validate (read-only)

```
cd ~/open-fdd
./scripts/security/check_host_security.sh
./scripts/security/check_openfdd_exposure.sh --edge-ip "$
```

```
(hostname -I | awk '{print $1}')"
lsb_release -a
uname -r
test -f /var/run/reboot-required && cat /var/run/reboot-
required.pkgs
python3 -m pip show pip setuptools wheel pyOpenSSL 2>/dev/null |
grep -E '^(Name|Version):'
ss -tulpn | grep -E ':80|:443|:8765|:11434'
curl -sf http://127.0.0.1:11434/api/tags | head -3
```

From a **different LAN workstation** (must fail):

```
curl -sf --max-time 3 http://<edge-ip>:11434/api/tags
```

122.3.2 Remediate host (maintenance window)

```
./scripts/security/remediate_ubuntu_host.sh # dry-run
./scripts/security/remediate_ubuntu_host.sh --apply
sudo reboot # if reboot-required
./scripts/security/check_host_security.sh
```

122.3.3 Remediate Open-FDD containers

```
OPENFDD_IMAGE_TAG=latest ./scripts/upgrade_edge_ghcr.sh --limit
<inventory_host>
docker compose pull && docker compose up -d
```

122.3.4 Rescan

After reboot and image pull, request IT **Nessus rescan** of the edge IP. Compare plugin output to the table above.

122.4 Ollama unauthenticated access (Critical)

Not an Open-FDD code bug — Ollama's API is unauthenticated by design. Risk is **network exposure**.

Repo defaults (3.0.5+):

- Systemd: Environment=OLLAMA_HOST=127.0.0.1:11434 in infra/ansible/templates/ollama.service.j2
- Compose Ollama (optional profile): 127.0.0.1:11434:11434 only
- Acme-style edge: host Ollama + bridge via host.docker.internal:11434 (container → host loopback path)
- post_deploy_check.sh warns if :11434 listens on 0.0.0.0

Re-apply Ansible Ollama unit after upgrade:

```
ansible-playbook -i inventory.yml ollama_bootstrap.yml --limit
<host> -e enable_ollama=true
```

122.5 pyOpenSSL and pip in containers

Docker build installs `docker/python-security-constraints.txt` before application requirements:

- `pip>=26.0`, `setuptools>=75.0`, `wheel>=0.45`
- `pyOpenSSL>=26.0.0`, `cryptography>=44.0.0`

CI `pip-audit` runs against `bridge` + `bacnet` requirements with the same constraints.

122.6 TLS — expected vs production

Mode	Nessus SSL plugins	When to use
Caddy HTTP :80	No TLS plugins	Dev / HTTP-only VLAN
Caddy self- signed : 443	Medium: untrusted, self- signed, expiry	OT LAN encryption, lab
Enterprise CA cert on Caddy	Clean SSL plugins (if chain trusted)	IT mandates clean Nessus

See [TLS — enterprise CA](#).

122.7 Residual findings after remediation

Even with a patched host and loopback-only Ollama, IT may still see:

- **SSL** Medium on self-signed TLS (accepted for lab; document in risk register)
- **ICMP** Low (optional network block)
- **Container** plugins if Nessus scans Docker overlay networks — clarify scan scope with IT

122.8 Notes for IT / security team

1. Scan the **edge VM IP**, not individual container IPs, unless container CVE correlation is required.

- 2. Open-FDD **bridge :8765** should be **loopback-only**; LAN entry is **Caddy :80/:443**.
- 3. **BACnet UDP 47808** on commission is expected OT traffic — out of scope for web/plugin remediation.
- 4. Do not expose Ollama **11434** to the building LAN; agent chat is server-side only.
- 5. Kernel CVEs require **host reboot** — schedule with operations.
- 6. For authenticated web/API depth, see roadmap [Authenticated scanning](#).

122.9 Repo-owned fixes (this document’s release)

Item	Location
Container pip/ pyOpenSSL pins	docker/python-security-constraints.txt, docker/Dockerfile
Ollama loopback systemd	infra/ansible/templates/ ollama.service.j2
Host/exposure check scripts	scripts/security/check_*.sh, remediate_ubuntu_host.sh
Post-deploy Ollama bind warn	infra/ansible/scripts/ post_deploy_check.sh
CI pip-audit + constraints	.github/workflows/ci.yml

123 Security

124 Security

LAN deployment security for the Operator Bridge and BACnet write guard. Open-FDD is for **trusted edge / OT LAN** — not direct public internet exposure.

Page	Topic
LAN hardening	Auth and bind modes
ZAP baseline (local bench)	Pentest stack, expected warnings
TLS and certificates	Cert ownership, bench TLS mode
Linux host hardening	Ubuntu edge VM, SSH, kernel, Ollama bind
Tenable / Nessus remediation	IT scan findings, operator runbook

Page	Topic
Authenticated scanning (roadmap)	Deep ZAP / protected API
Secrets and auth	Env files
BACnet write allowlists	Supervisory write gates
Release & LAN scan cycle	Per-revision ZAP + CI workflow
Logging and audit	Auth audit trail, rotation, SIEM

125 API routes

126 API routes

REST API served by **openfdd-bridge** (default <http://127.0.0.1:8765>).
Production: Caddy on :80 proxies to the bridge.

OpenAPI: GET /docs and GET /redoc when the bridge runs.

Auth: JWT via POST /api/auth/login. Most routes require Authorization: Bearer <token>. Roles: viewer, operator, integrator, commission, agent, admin. BACnet writes require commission role + write safety. Dev: OFDD_AUTH_DISABLED on loopback only.

WebSocket: WS /ws/dashboard — POST /api/auth/ws-ticket, then pass the ticket via Sec-WebSocket-Protocol: ofdd.ws, <ticket>. Query ?ticket= is dev-only when OFDD_WS_ALLOW_QUERY_TICKET=1. Tickets are short-lived and not interchangeable with the Bearer token.

126.1 Health & audit

Method	Path	Role	Description
GET	/health	public	Minimal liveness (ok, service, version, auth_required)
GET	/health/stack	auth	Stack traffic-light (verbose URLs/bind with OFDD_DEBUG_DIAGNOSTICS)
POST	/api/auth/ws-ticket	auth	Short-lived WebSocket ticket
GET		auth	Audit counters

Method	Path	Role	Description
	/api/audit/summary		
GET	/api/audit/events	auth	Audit log (query params)
GET	/api/audit/errors	auth	Recent API errors

*Exact public set depends on OFDD_AUTH_DISABLED and middleware.

126.2 Auth

Method	Path	Description
POST	/api/auth/login	Username/password → JWT
GET	/api/auth/me	Current user
GET	/api/auth/status	Auth mode / logout hints

126.3 Rules & FDD

Method	Path	Role	Description
GET	/api/rules/saved	user	Rule index
POST	/api/rules/save	operator+	Create/update rule metadata
GET	/api/rules/saved/{id}/source	user	Python source
PUT	/api/rules/saved/{id}/source	operator+	Write .py file
GET	/api/rules/assignments	user	Point bindings
POST	/api/rules/bind	integrator+	Bind rule to point
POST	/api/rules/bindings	integrator+	Bulk bindings
POST	/api/rules/batch	operator+	Run all enabled rules → fdd_results.json

Method	Path	Role	Description
GET/POST	/api/rules/drafts	user	Draft rules

126.4 Rule Lab playground

Method	Path	Description
POST	/api/playground/lint	AST lint Python rule
POST	/api/playground/test-rule	Run apply_faults_arrow() on feather PyArrow window
POST	/api/playground/run-script	Execute script mode
GET	/api/playground/sample-frame	Sample DataFrame for editor

126.5 BRICK / data model

Method	Path	Description
GET	/api/model/sites	Site list
GET	/api/model/tree	Equipment tree (SPARQL)
GET	/api/model/graph	RDF graph fragment
GET	/api/model/scope	Scoped query
GET	/api/model/export	Export TTL/JSON
GET	/api/model/health	Model point/equipment counts
GET	/api/model/ttl	TTL metadata
POST	/api/model/sync-ttl	Regenerate TTL from JSON
POST	/api/model/import	Import model payload
POST	/api/model/sites	Create site
GET/POST	/api/model/bacnet-sync	BACnet ↔ model sync state

126.6 BACnet

Method	Path	Role	Description
GET	/config/ bacnet	read	Commission URL / config
GET	/api/ bacnet/ commission/ status	read	Commission agent health
GET	/api/ bacnet/ server/ points	read	Server point table
GET	/api/ bacnet/ inventory	read	Discovered inventory
POST	/api/ bacnet/ whois	commission	Who-Is
POST	/api/ bacnet/ discover	commission	Discover devices
POST	/api/ bacnet/read	commission	Read property
POST	/api/ bacnet/ read- multiple	commission	RPM read
POST	/api/ bacnet/ write	write	Supervisory write (gated)
POST	/api/ bacnet/ import-to- model	integrator	Inventory → BRICK
POST	/api/ bacnet/ driver/ sync- discovery	commission	Sync driver tree
POST	/api/ bacnet/ poll/once	commission	Trigger poll cycle
GET	/api/ bacnet/ poll/status	read	Poll worker status

Method	Path	Role	Description
POST	/ingest/ bacnet	read	Ingest samples into feather

Bind address: BACNET_BIND in commission env — see [BACnet network setup](#). Operator guide: [Discover and read](#).

PATCH | /api/bacnet/driver/point | commission | Enable poll + interval on point |
 PATCH | /api/bacnet/driver/device | commission | Enable poll for all points on device |
 PATCH | /api/bacnet/driver/device/remap | commission | Change instance and/or address |
 DELETE | /api/bacnet/driver/point/{point_id} | commission | Remove point from registry |
 DELETE | /api/bacnet/driver/device/{device_instance} | commission | Remove device |
 DELETE | /api/bacnet/driver/registry | commission | Clear CSVs + model sync |

126.7 Faults (check-engine)

Method	Path	Description
GET	/api/faults/catalog	Fixed fault codes
GET	/api/faults/status	GREEN/YELLOW/RED aggregate
GET	/api/faults/tree	Fault tree by equipment
GET	/api/faults/code/{code}	Code metadata

126.8 Timeseries & sites

Method	Path	Description
GET	/api/timeseries/sites	Feather sites
GET	/api/timeseries/series	Series metadata
GET	/api/timeseries/plot	Plot payload
GET	/api/timeseries/readings	Raw readings
GET	/api/sites	Site list
GET	/api/sites/{site_id}/frame	Wide frame for site

126.9 Building & host

Method	Path	Description
GET	/api/building/status	Building summary
GET	/api/building/alerts	Active alerts
GET	/api/host/stats	CPU/mem/disk

126.10 Local agent (Ollama)

Prefix `/openfdd-agent`. Role `agent` for tool execution.

Method	Path	Description
GET	/openfdd-agent/context	Composed prompt context
GET	/openfdd-agent/tools	Tool manifest
POST	/openfdd-agent/tool	Execute tool (e.g. <code>rules.save</code>)
GET	/openfdd-agent/ollama/health	Ollama reachability
GET	/openfdd-agent/building-insight	Check-engine narrative context
GET	/openfdd-agent/zone-temps	Zone temperature insight
POST	/openfdd-agent/chat	Chat completion (catalog-bound)

Optional host Ollama for building insight — not required for core FDD. Configure via compose profile or host service; see [Architecture overview](#).

126.11 Modbus

Method	Path	Role	Description
GET	/api/modbus/driver/tree	operator+	Commissioning tree

Method	Path	Role	Description
GET	/api/modbus/poll/status	operator+	Poll worker status
POST	/api/modbus/poll/once	integrator+	Force poll cycle
POST	/api/modbus/read_registers	operator+	One-shot register read
POST	/api/modbus/read_and_store	operator+	Read + CSV + feather (source=modbus)
POST	/api/modbus/refresh	operator+	Re-read one register
PATCH	/api/modbus/register/poll	integrator+	Enable poll interval

See [Driver framework](#).

126.12 JSON API (HTTP/HTTPS)

Method	Path	Role	Description
GET	/api/json-api/env/status	operator+	Env var configured flags (no secret values)
POST	/api/json-api/presets/openweather	integrator+	Register 3-point OpenWeather bundle + optional poll
GET	/api/json-api/driver/tree	operator+	Endpoints grouped by host
GET	/api/json-api/poll/status	operator+	Poll worker status
POST	/api/json-api/poll/once	integrator+	Force poll cycle
POST	/api/json-api/request	operator+	One-shot GET/POST (no store)
POST	/api/json-api/read_and_store	operator+	Request + CSV + feather (source=json_api)
POST	/api/json-api/refresh	operator+	Re-fetch endpoint
PATCH	/api/json-api/endpoint/poll	integrator+	Enable poll interval
DELETE		integrator+	Remove endpoint

Method	Path	Role	Description
	/api/json-api/ endpoint/ {point_id}		

Outbound auth: `auth_type = none | bearer | basic`; optional `verify_tls` (default `true`). See [JSON API driver](#).

126.13 PyPI package (separate)

Library-only HTTP is **not** bundled. See [Python package](#) for `open_fdd.arrow_runtime` (Arrow rules offline).

126.14 Errors

JSON error bodies; stack traces hidden unless `OFDD_DEBUG_TRACEBACKS=1`. See [LAN hardening](#).

127 Python package

128 Python package (open-fdd)

Install from PyPI:

```

pip install open-fdd
# Arrow runtime + playground (default 3.0.1+)
pip install "open-fdd[ml]"
# optional numpy/sklearn for offline graph ML experiments

```

128.1 Modules

Module	Purpose
<code>open_fdd.arrow_runtime</code>	Default — <code>apply_faults_arrow</code> on PyArrow Tables, cookbook masks, column maps
<code>open_fdd.playground</code>	Rule Lab lint/compile helpers for Arrow rules

Retired in 3.0.1+: open_fdd.engine (YAML/pandas RuleRunner) is **not** shipped on PyPI. Use Operator Bridge Rule Lab or import rules from workspace/data/rules_py/.

128.2 When to use package-only

Scenario	Use
Lint/test Arrow rules offline	open_fdd.arrow_runtime + open_fdd.playground
Graph ML experiments (Layer B)	open-fdd[ml] in experiments/ (see issue #211)
Full building operator stack	Docker Operator Bridge

128.3 Versioning

Publish: git tag open-fdd-vX.Y.Z → GitHub Actions **Publish open-fdd**.

```
import open_fdd
print(open_fdd.__version__)
```

128.4 Offline Arrow example

```
import pyarrow as pa
import pyarrow.compute as pc
from open_fdd.arrow_runtime import run_arrow_rule

code = '''
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["SAT"], float(cfg["high"]))
'''

table = pa.table({"SAT": [70.0, 90.0, 88.0]})
result = run_arrow_rule(code, table, {"high": 85})
print(result.flagged_count)
```

API surface: open_fdd/arrow_runtime docstrings and [Arrow recipes](#).

129 Configuration reference

130 Configuration reference

130.1 Bridge / auth

Variable	Default	Notes
OFDD_BRIDGE_HOST	127.0.0.1	0.0.0.0 for LAN
OFDD_AUTH_SECRET	—	Required for LAN
OFDD_AUTH_DISABLED	0	Localhost dev only
OFDD_INSECURE_LAN_DEV	0	Lab only with auth disabled

130.2 BACnet

Variable	Notes
BACNET_BIND	ip/mask:47808 in commission.env
OFDD_BACNET_WRITE_ENABLED	Default off — site-specific name may vary

130.3 Docker edge

Variable	Notes
OPENFDD_IMAGE_TAG	GHCR tag for Ansible deploy
openfdd_docker_pull_from_ghcr	Ansible host_var

Rule Lab Python rules use `rules_store.json + workspace/data/rules_py/*.py`. Legacy YAML schema (`docs/config_schema.json`) is archived — not used by Operator Bridge 3.0.1+.

131 Glossary

132 Glossary

Term	Definition
Operator Bridge	FastAPI service serving API + static dashboard
Rule Lab	UI and APIs for Arrow apply_faults_arrow() rules
Feather store	On-disk columnar historian under workspace/data/feather_store/
fdd_input	Brick tag linking a point to a rule binding role
Check-engine	Building-level GREEN/YELLOW/RED fault summary
Commission	BACnet discovery/read (and optional write) agent container
Poll driver	Scheduled BACnet RPM reads into historian
GHCR	GitHub Container Registry — ghcr.io/ bbartling/openfdd-*

133 Workspace env files

134 Workspace env files

All live under workspace/. Copy from *.example templates;
committed *.local files are gitignored.

File	Loaded by	Purpose
auth.env.local	run_local.sh, Docker compose, Ansible	JWT secret (OFDD_AUTH_SECRET), integrator/operator/ agent passwords, OFDD_AUTH_DISABLED for localhost dev only
data.env.local	run_local.sh, bridge, feather maintain	Feather historian caps (OFDD_FEATHER_*), audit/ error log rotation

File	Loaded by	Purpose
		(OFDD_LOG_*), BACnet discovery mutation toggle
json_api.env.local	bridge, Ansible edge sync	API keys for JSON API driver <code>\${ENV:VAR}</code> substitution (e.g. OPENWEATHER_API_KEY)
ollama.env.local	run_local.sh, bridge	Ollama base URL, model name, RAM tier, GPU mode, timeouts — see Ollama and analytics
mcp.env.local	run_local.sh, MCP sidecar	Enable MCP RAG REST (OFDD_MCP_*), index path
caddy.env.local	run_local.sh, Ansible edge	Reverse proxy mode (http / tls / off), TLS CN, port overrides
pentest.env.local	pentest_production_stack.sh	Production-like LAN scan stack: CORS, auth, passive BACnet defaults

134.1 Ansible / edge extras

Path	Purpose
infra/ansible/secrets/<host>.env.local	SSH deploy credentials, GHCR pull tokens, site-specific BACnet bind — maintainers only

134.2 Related docs

- [Secrets and auth](#) — required production variables
- [Logging and audit](#) — OFDD_LOG_* detail
- [JSON API driver](#) — `${ENV:VAR}` pattern
- [Configuration reference](#) — bridge runtime flags not in env files

135 Appendix

136 Appendix

Reference material for integrators and library users.

Page	Content
API routes	Operator Bridge REST/WebSocket
Python package	PyPI open-fdd
Configuration reference	Key env variables
Workspace env files	workspace/*.env.local inventory
Glossary	Terms

137 GL36 lab site (Acme)

138 GL36 lab site (Acme)

Lab note only. This documents a maintainer **example edge host** (site acme, building vm-bbartling) used for Guideline 36 supervisory FDD development. Replace inventory host names, IPs, and BACnet ranges for your site.

Reference for **ASHRAE Guideline 36** supervisory monitoring on a live lab edge. Rule code lives in workspace/data/rules_py/; commissioning uses **Arrow constants** (no config.json).

Trim & respond Niagara reference: [README_TRIM_RESPOND](#).

138.1 BACnet poll scope

Device filter (infra/ansible/scripts/acme_trim_devices.sh):

Range	Equipment
1-100	JCI VMA/VAV
1100	RTU-01 (AHU)
1002	Hot water plant
11000-13000	Trane UC210 VAV

Poll interval: **60 s**. Commission from control machine:

```
cd infra/ansible
POLL_INTERVAL=60 ./scripts/acme_commission_gl36.sh
```

Points land in edge_backup/local/acme/vm-bbartling/points.csv for Ansible sync.

138.2 Example FDD rules (Arrow)

138.2.1 Duct static pressure high (AHU)

```
"""AHU duct static high – GL36 trim advisory."""

import pyarrow.compute as pc

VALUE_COLUMN = "duct-static"
HIGH_INWC = 1.20
LOOKBACK_HOURS = 3

def _kit_lookback_stats(table, *, hours=None):
    h = hours if hours is not None else LOOKBACK_HOURS
    ts = pc.cast(table["timestamp"], "timestamp[us, UTC]")
    tmin, tmax = pc.min(ts).as_py(), pc.max(ts).as_py()
    span_h = (tmax - tmin).total_seconds() / 3600.0 if tmin and
tmax else 0.0
    print(f"lookback={h}h rows={table.num_rows} start={tmin}
stop={tmax} span={span_h:.2f}h")

def apply_faults_arrow(table, cfg, context=None):
    _kit_lookback_stats(table)
    return pc.greater(pc.cast(table[VALUE_COLUMN], "float64"),
HIGH_INWC)
```

138.2.2 SAT too cold vs setpoint

```
"""AHU SAT too cold vs setpoint."""

import pyarrow.compute as pc

SAT_COLUMN = "sa-t"
SP_COLUMN = "sa-t-sp"
COLD_DELTA_F = 5.0

def apply_faults_arrow(table, cfg, context=None):
    sat = pc.cast(table[SAT_COLUMN], "float64")
    sp = pc.cast(table[SP_COLUMN], "float64")
    return pc.less(pc.subtract(sat, sp), -COLD_DELTA_F)
```

138.2.3 Site rule stubs in the repo

Rule file	Use
bench_stat_zn-t_flatline_1h.py	Zone temp flatline —

Rule file	Use
	bind ZN-T per VAV
acme_duct_static_flatline_1h_gl36_duct_t_r_input.py	RTU duct static flatline
oat_vs_web_spread_1h.py	Local OAT vs OpenWeather JSON API

138.2.4 Local OA-T vs web weather (JSON API)

1. Control machine: workspace/json_api.env.local with OPENWEATHER_API_KEY
2. JSON API tab → **OpenWeatherMap** preset → **Register** (default poll **30 min**)
3. Bind BACnet local OAT in the model (oa-t historian column)
4. Enable oat_vs_web_spread_1h.py in Rule Lab — flags when |oa-t - web-oat-t| > 8 °F

138.3 Deploy / upgrade (Ansible, this lab only)

```
cd infra/ansible
OPENFDD_IMAGE_TAG=latest ./deploy.sh docker --limit
<inventory_host> -e enable_bacnet_poll_driver=true
```

Full UI + image upgrade from control machine:

```
OPENFDD_IMAGE_TAG=latest ./scripts/upgrade_edge_full.sh --limit
<inventory_host>
infra/ansible/scripts/post_deploy_check.sh --limit
<inventory_host> --full
```

Do **not** rsync dev auth.env.local to the edge (deploy_sync_auth: false in private host_vars). Regenerate credentials on the edge host if login fails after upgrade.

138.4 AI-assisted data modeling (lab workflow)

1. BACnet poll running (~340 GL36 points @ 60 s on this lab host).
2. **Model & assignments** → export commissioning JSON for LLM-assisted binding.
3. Import returned JSON via commissioning import.

4. Pin FDD rules on ZN-T, DA-T, duct-static, OAT, etc. — export shows readable names in `fdd_rules_linked`.

Model seed: `workspace/data/acme_gl36_model.json`.

139 Examples & lab notes

140 Examples & lab notes

Site-specific commissioning notes, GL36 examples, and maintainer lab workflows. These pages are **not** required for a generic edge deployment.

Page	Description
GL36 lab site (Acme)	Example real-building poll scope, Arrow rule stubs, Ansible deploy commands

For production IT operators, start with [Quick Start and Operations](#).

141 Central collection

142 Portfolio central collection

Edge Open-FDD sites emit **interval summaries** (not raw historian by default). Central portfolio ingests, stores, and aggregates for Dash and the AI tuning agent.

142.1 Interval summary schema

Defined in `portfolio/store/interval_schema.py` as `FaultIntervalSummary`:

- Identity: `site_id`, `building_id`, `equipment_id`, `equipment_family`
- Window: `timestamp_start`, `timestamp_end`
- Fault: `fault_code`, `canonical_id`, `severity`, `confidence`
- Activity: `active_minutes`, `percent_active`, `occurrence_count`
- Evidence: `evidence_json` (summarized, not full trend)

- Tuning: tuning_version, tuning_params_hash
- Operator: operator_status (new | acknowledged | resolved | false_positive | ...)

142.2 Edge export formats

- JSONL (append-only events)
- Parquet / Feather (batch intervals)

142.3 Collect today (rollup API)

```
source infra/ansible/secrets/acme.env.local
python3 scripts/portfolio_collect.py
```

142.4 Tuning proposals

Human-reviewable proposals use TuningProposal in the same module. The AI agent **never** auto-writes BACnet points — see docs/portfolio/ai-agent-tuning.md (3.0.19).

143 Arrow-native runtime

144 Arrow-native runtime

Open-FDD 3.0 executes Rule Lab rules on **PyArrow tables** end to end.

```
feather_store.read_site_table()
→ fdd_runner / playground.run_arrow_table()
→ apply_faults_arrow(table, cfg, context)
→ pyarrow.compute masks
```

144.1 Authoring contract

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["zone_temp"], cfg["max_zone_temp"])
```

Cookbook helpers: open_fdd.arrow_runtime.cookbook (flatline, spread, OOB, after-hours fan).

Script-mode analytics rules receive table (PyArrow) and cfg in the sandbox — not pandas DataFrames.

144.2 Package layout

Module	Role
open_fdd.arrow_runtime.backend	Execute rule code, batch chunks
open_fdd.arrow_runtime.cookbook	Shared fault masks
open_fdd.arrow_runtime.windows	Rolling min/max, consecutive-true
open_fdd.playground.arrow_templates	Rule Lab starter templates